Universidade Federal de Ciência e Inovação de Minas Gerais (UFCIMG)
Engenharia da Computação

Igor Peixoto Rodrigues, Pedro Henrique de Araújo Carvalho, Mariana Kaori Yano, Rita Mariê Amaral, Julya Gonçalves Pires, Samuel Pianetti Cordeiro

Heaps de Fibonacci e Árvores de van Emde Boas

Divinópolis – MG

2026

Igor Peixoto Rodrigues, Pedro Henrique de Araújo Carvalho, Mariana Kaori Yano, Rita Mariê Amaral, Julya Gonçalves Pires, Samuel Pianetti Cordeiro

Heaps de Fibonacci e Árvores de van Emde Boas

Estudo sobre heaps de Fibonacci e árvores de van Emde Boas, apresentado à disciplina de Algoritmos Avançados II do curso de Engenharia da Computação da Universidade Federal de Ciência e Inovação de Minas Gerais, como requisito parcial de avaliação.

Universidade Federal de Ciência e Inovação de Minas Gerais (UFCIMG)
Engenharia da Computação

Divinópolis – MG
2026

Resumo

Este estudo investiga formal e empiricamente duas estruturas de dados avançadas: os heaps de Fibonacci e as árvores de van Emde Boas (vEB). Analisam-se suas representações estruturais, invariantes matemáticos e complexidades assintóticas de tempo e espaço, destacando o método do potencial na obtenção do custo amortizado $O(1)$ para a diminuição de chave no heap de Fibonacci e a decomposição recursiva do universo de chaves para atingir operações em tempo $O(\log \log U)$ na árvore vEB. Conduzem-se dois estudos experimentais desenvolvidos em C++20: a comparação direta de operações de fila de prioridade sob cargas sintéticas de até 10^6 elementos e a avaliação comparativa frente a estruturas alternativas, incluindo heaps d-ários no algoritmo de Dijkstra e árvores balanceadas (AVL e rubro-negra) para conjuntos dinâmicos. Os resultados demonstram que, embora ambas as estruturas alcancem vantagens assintóticas teóricas expressivas, seus desempenhos práticos dependem diretamente da hierarquia de memória cache, do custo de ponteiros e da sobrecarga de alocação dinâmica. Heaps baseados em arranjos contíguos superam o heap de Fibonacci no algoritmo de Dijkstra nos cenários testados, enquanto a árvore vEB é mais rápida para consultas ordenadas sob universos compactos densamente ocupados, incorrendo em penalidade espacial quando o universo é esparso.

Palavras-chave: heap de Fibonacci; árvores de van Emde Boas; estruturas de dados; análise de algoritmos; filas de prioridade.

Sumário

1 INTRODUÇÃO 5
1.1 Objetivos 6
1.2 Roteiro de leitura 7

2 HEAP DE FIBONACCI 8
2.1 Representação estrutural e listas circulares 8
2.2 Invariantes estruturais e propriedades matemáticas 9
2.3 Operações preguiçosas em tempo constante: criação, mínimo, inserção e união 11
2.4 Extração do mínimo e consolidação por grau 13
2.5 Diminuição de chave, cortes em cascata e remoção 13
2.6 Análise amortizada 16

3 ÁRVORES DE VAN EMDE BOAS 19
3.1 Conceitos iniciais 19
3.1.1 Da busca por predecessores à estrutura proto-van Emde Boas 20
3.2 Definição da árvore vEB 24
3.3 Paradigma Algorítmico 29
3.3.1 Pertinência 29
3.3.2 Mínimo e máximo 31
3.3.3 Sucessor 32
3.3.4 Predecessor 34
3.3.5 Inserção 37
3.3.6 Remoção 39
3.3.7 Extração do mínimo 42
3.4 Análise Assintótica 43
3.4.1 Modelo de Custo e Recorrência 43
3.4.2 Tempo das Operações 44
3.4.3 Análise de Espaço e Aplicações Práticas 45

4 METODOLOGIA EXPERIMENTAL 46
4.1 Ambiente Computacional e Protocolo de Medição 46
4.2 Estudo 1: Comparação Direta das Filas de Prioridade 47
4.3 Estudo 2: Comparação com Estruturas Alternativas de Referência . 48

5 RESULTADOS 49
5.1 Comparação isolada das filas de prioridade 49

5.1.1 Tempos das operações equivalentes 49
5.1.2 Consumo de memória e influência do universo 52
5.1.3 Operações específicas de cada estrutura 53
5.2 Comparação com estruturas alternativas 54
5.2.1 Árvore vEB, AVL e rubro-negra 54
5.2.2 Heap de Fibonacci no algoritmo de Dijkstra 59
5.3 Síntese e discussão 61
5.3.1 Limitações e implicações práticas 62

6 CONCLUSÃO 64
6.1 Síntese dos resultados teóricos e práticos 64
6.1.1 Heaps de Fibonacci 64
6.1.2 Árvores de van Emde Boas 65
6.2 Diretrizes de engenharia de software 65

REFERÊNCIAS 67

APÊNDICE A – GUIA DE CONSULTA RÁPIDA (CHEAT SHEET) 68

1 Introdução

A busca por eficiência no processamento de grandes volumes de dados é um dos desafios centrais da computação. Em algoritmos de roteamento, sistemas operacionais e simulações científicas, o tempo de resposta depende diretamente da velocidade com que conjuntos dinâmicos são consultados e modificados. Para essa finalidade, as filas de prioridades consolidaram-se como estruturas essenciais, viabilizando a manutenção de dados com chaves mutáveis e a recuperação ágil do elemento de valor extremo. Na prática da engenharia de software, a solução hegemônica para essa demanda é o heap binário, que organiza os dados em arranjos contíguos de memória e assegura tempo $O(\log n)$ para inserções, remoções e atualizações. Do mesmo modo, árvores binárias balanceadas estendem essa cota logarítmica para a ordenação dinâmica a partir de comparações sucessivas entre chaves. No entanto, em aplicações de larga escala, essa dependência de $O(\log n)$ manifesta-se como um gargalo sensível. Um caso notório ocorre no algoritmo de Dijkstra (DIJKSTRA, 1959) aplicado a grafos densos ($|E| \gg |V|$), onde a exploração dos vértices exige milhões de chamadas à operação de diminuição de chave (*decrease-key*); sobre um heap binário convencional, cada ajuste consome $O(\log |V|)$, elevando o custo global de relaxamento para $O(|E| \log |V|)$ e restringindo a vazão computacional.

Para superar essa barreira, a teoria da computação desenvolveu alternativas capazes de subverter os limites do modelo comparativo tradicional. Fredman e Tarjan (FREDMAN; TARJAN, 1987) introduziram o Heap de Fibonacci, cuja estratégia reside em flexibilizar invariantes estruturais estritos e adotar o princípio da avaliação preguiçosa (*lazy evaluation*): inserções e reduções de chave são executadas em tempo constante estrito ou amortizado $O(1)$, postergando a consolidação das árvores para o momento da extração do mínimo, o que viabiliza a execução do algoritmo de Dijkstra no limite assintótico ótimo de $O(|E| + |V| \log |V|)$. Em paralelo, Peter van Emde Boas (van Emde Boas; KAAS; ZIJLSTRA, 1977) explorou uma abordagem distinta ao restringir o domínio das chaves a universos inteiros finitos $U = \{0, 1, \ldots, u - 1\}$. Ao invés de operar por comparações genéricas, a Árvore de van Emde Boas particiona recursivamente a representação posicional das chaves por raízes quadradas sucessivas, organizando os dados em coordenadas de *cluster* e deslocamento. Essa arquitetura permite realizar buscas, inserções, remoções e consultas de ordem em tempo de pior caso $O(\log \log U)$, um ritmo sublogarítmico que independe da quantidade de elementos ativos na estrutura.

Apesar da elegância matemática dessas formulações, sua eficácia prática no hardware contemporâneo é delimitada por restrições arquiteturais que transcendem a notação assintótica. Modelos puramente teóricos desconsideram constantes multiplicativas elevadas e assumem acesso homogêneo à memória, omitindo fatores críticos como as transferências

em blocos de 64 bytes nas linhas de cache (L1, L2 e L3), o impacto da localidade espacial e a penalidade por navegação em ponteiros dispersos na memória dinâmica (*pointer chasing*). Motivado por essa divergência entre as cotas ideais da literatura e o comportamento físico das CPUs, este trabalho propõe-se a implementar e avaliar experimentalmente o Heap de Fibonacci e a Árvore de van Emde Boas frente às estruturas clássicas, identificando sob quais cenários operacionais seus ganhos assintóticos de fato superam os sobrecustos de projeto e de memória.

1.1 Objetivos

O objetivo geral deste trabalho é caracterizar e comparar os heaps de Fibonacci e as árvores de van Emde Boas, relacionando seus fundamentos teóricos e matemáticos ao comportamento prático observado em implementações em C++20. Os objetivos específicos são:

1. Formalizar a representação em memória e os invariantes estruturais do heap de Fibonacci, demonstrando por indução matemática a cota inferior do tamanho das subárvores ($s_k \geq F_{k+2} \geq \varphi^k$) e o limite logarítmico do grau máximo ($D(n) \leq \lfloor \log_\varphi n \rfloor$);

2. Formalizar a organização hierárquica e os invariantes da árvore de van Emde Boas, detalhando a partição recursiva de universos discretos em estruturas de *summary* e *clusters* de ordens $\uparrow\sqrt{u}$ e $\downarrow\sqrt{u}$, bem como o isolamento dos extremos (min e max);

3. Apresentar as operações fundamentais e auxiliares de ambas as estruturas por meio de pseudocódigos formais, diagramas de organização de nós e fluxogramas de controle, destacando a consolidação e cortes em cascata no Fibonacci e as descidas recursivas e casos base na vEB;

4. Analisar formalmente as complexidades assintóticas de tempo e de espaço: para o heap de Fibonacci, derivar os custos amortizados via método do potencial com a função $\Phi(H) = t(H) + 2m(H)$; para a árvore vEB, resolver as relações de recorrência do tipo $T(u) = T(\sqrt{u}) + O(1)$, demonstrando o limite $O(\log \log U)$ e a ocupação espacial $\Theta(U)$;

5. Confrontar experimentalmente o heap de Fibonacci e a árvore vEB em operações equivalentes de fila de prioridade (`insert`, `findMin`, `extractMin` e `delete`) sob ordens de entrada aleatória, crescente e decrescente e diferentes razões de ocupação $\rho = n/U$;

6. Avaliar comparativamente o desempenho de ambas as estruturas frente a implementações de referência especializadas:

- o heap de Fibonacci frente a heaps binários e d-ários ($d \in \{4, 8\}$) em cargas sintéticas e no algoritmo de Dijkstra, examinando o impacto da hierarquia de cache de CPU e das estruturas de indexação (`std::vector` e `std::unordered_map`);
- a árvore vEB frente a árvores binárias balanceadas (AVL e rubro-negra) em operações sobre conjuntos ordenados (pertinência, extremos, sucessor e predecessor), caracterizando os compromissos de tempo de execução e pegada de memória medidos via Valgrind Massif.

1.2 Roteiro de leitura

O documento está estruturado em seis capítulos e um apêndice. O Capítulo 2 aborda o heap de Fibonacci, seus invariantes, algoritmos e análise amortizada via função potencial. O Capítulo 3 formaliza a árvore de van Emde Boas, sua decomposição hierárquica, operações e complexidades. O Capítulo 4 apresenta a metodologia experimental, os ambientes de teste e os protocolos de medição. O Capítulo 5 discute os resultados quantitativos obtidos nos dois estudos experimentais. O Capítulo 6 sintetiza as conclusões gerais, contrastando os compromissos práticos e teóricos das estruturas. Por fim, o Apêndice A fornece um resumo executivo com diretrizes práticas de uso.

O código-fonte das implementações e dos experimentos está disponível em: <https://github.com/peixotoigor/AEDS-AdvancedStructures.git>.

2 Heap de Fibonacci

Diferentemente do heap binário, que impõe uma árvore binária quase completa rígida mapeada em arranjo, ou do heap binomial, que exige no máximo uma árvore binomial de cada grau após toda inserção (VUILLEMIN, 1978), um Heap de Fibonacci relaxa drasticamente os invariantes de forma. Ele consiste em uma coleção (floresta) de árvores enraizadas não ordenadas que obedecem à propriedade de *min-heap*, mantendo a reorganização estrutural estritamente sob demanda (CORMEN et al., 2009).

2.1 Representação estrutural e listas circulares

Seja H um Heap de Fibonacci contendo $H.n$ elementos. Na representação adotada neste trabalho, cada elemento reside em um nó alocado dinamicamente x que armazena:

- $chave[x]$ e $valor[x]$: a prioridade numérica usada nas comparações e o identificador satélite associado ao elemento (por exemplo, o índice de um vértice $v \in V$);
- $grau[x] \in \mathbb{N}$: o número exato de filhos diretos na lista de filhos de x;
- $marca[x] \in \{\text{FALSO}, \text{VERDADEIRO}\}$: atributo booleano que registra se x já perdeu um filho desde a última vez em que se tornou filho de outro nó;
- Quatro ponteiros estruturais: $pai[x]$ (apontando para o nó ancestral imediato, ou NIL se x for raiz), $filho[x]$ (apontando para um filho arbitrário de x, ou NIL se $grau[x] = 0$), e os ponteiros laterais $esquerda[x]$ e $direita[x]$.

Os irmãos de uma mesma camada — tanto as raízes das árvores na *lista de raízes* quanto os filhos de qualquer nó x — são interligados em **listas circulares duplamente encadeadas** por meio de *esquerda* e *direita*. Quando um nó x encontra-se isolado em sua lista, vale $esquerda[x] = direita[x] = x$. O descritor global do heap armazena apenas dois campos: o inteiro $H.n$ e o ponteiro $H.min$, que referencia a raiz de menor chave na lista de raízes (NIL caso $H.n = 0$). A Figura 1 ilustra a disposição dos ponteiros em uma floresta com três raízes.

A escolha de listas circulares duplamente encadeadas responde diretamente por três propriedades algorítmicas essenciais (CORMEN et al., 2009): (i) inserir um nó adjacente a qualquer referência conhecida requer alterar apenas quatro ponteiros em $\Theta(1)$; (ii) remover um nó arbitrário x de sua lista de irmãos exige apenas religar $esquerda[x]$ e $direita[x]$ em $\Theta(1)$, sem busca linear; e (iii) concatenar duas listas circulares não vazias completa-se em $\Theta(1)$ trocando dois pares de ponteiros.

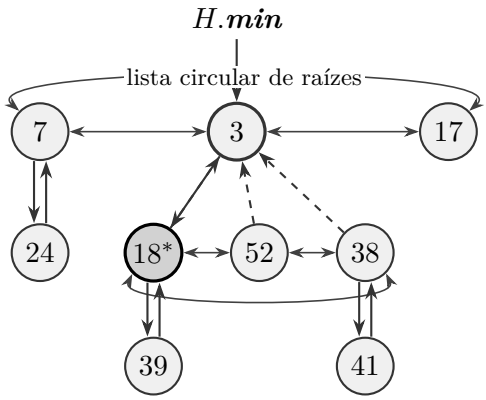


Figura 1 – Representação esquemática de um Heap de Fibonacci com $H.n = 9$ nós e $t(H) = 3$ árvores na lista de raízes. O nó 18^* encontra-se marcado (*marca* = VERDADEIRO). Elaborado pelos autores com base em Cormen et al. (2009).

2.2 Invariantes estruturais e propriedades matemáticas

Em qualquer estado válido entre operações públicas, um Heap de Fibonacci satisfaz cinco invariantes fundamentais:

1. **Propriedade de Min-Heap**: para todo nó x tal que $pai[x] \neq$ NIL, tem-se $chave[pai[x]] \leq chave[x]$;
2. **Ponteiro Mínimo**: se $H.n > 0$, $H.min$ aponta para um nó pertencente à lista de raízes cuja chave é mínima dentre todos os nós de H; se $H.n = 0$, $H.min =$ NIL;
3. **Consistência de Grau**: para todo nó x, $grau[x]$ coincide com a cardinalidade de sua lista circular de filhos diretos;
4. **Integridade Circular**: toda lista de raízes e de filhos forma um ciclo fechado biconectado ($direita[esquerda[u]] = u$ e $esquerda[direita[u]] = u$);
5. **Disciplina de Marcação**: raízes nunca acumulam cortes pendentes; um nó não raiz é marcado quando perde seu primeiro filho após ter sido vinculado a um pai, e é imediatamente cortado para a lista de raízes caso perca um segundo filho.

Toda a eficiência amortizada de Extract-Min e Decrease-Key repousa sobre o fato de que o grau máximo $D(n)$ de qualquer nó em um Heap de Fibonacci de n nós permanece logarítmico em n. Demonstramos formalmente esse resultado a seguir por indução matemática, seguindo Fredman e Tarjan (1987) e Cormen et al. (2009).

Lema 2.1 (Grau mínimo dos filhos ordenados)**.** *Seja x um nó qualquer de um Heap de Fibonacci com $k = grau[x]$, e sejam $y_1, y_2, \ldots, y_k$ seus filhos atuais listados na ordem cronológica em que foram vinculados a x. Então $grau[y_1] \geq 0$ e, para todo $i \in \{2, \ldots, k\}$, vale $grau[y_i] \geq i - 2$.*

Demonstração. Trivialmente $grau[y_1] \geq 0$. Considere $i \geq 2$. Na implementação do Heap de Fibonacci, a única operação que torna um nó y_i filho de x é o vínculo (HEAP-LINK) durante a consolidação, que só ocorre entre duas raízes que possuem exatamente o *mesmo grau* no instante da ligação. Quando y_i foi vinculado a x, os nós $y_1, y_2, \ldots, y_{i-1}$ já eram filhos atuais de x (eventuais outros filhos vinculados antes e já cortados só poderiam aumentar o grau de x naquele instante). Logo, no momento do vínculo de y_i, tínhamos $grau[x] \geq i-1$, o que implica que $grau[y_i] \geq i-1$ naquele instante, e $marca[y_i]$ foi definida como FALSO.

Desde então, y_i permaneceu filho de x. Pela disciplina de cortes em cascata, se y_i tivesse perdido dois ou mais filhos desde que se tornou filho de x, ele teria sido marcado na primeira perda e obrigatoriamente cortado de x na segunda perda, contradizendo o fato de ainda ser filho de x. Portanto, y_i perdeu no máximo um filho desde o vínculo, concluindo-se que seu grau atual satisfaz $grau[y_i] \geq (i-1)-1 = i-2$. □

Lema 2.2 (Identidade telescópica dos números de Fibonacci)**.** *Seja a sequência de Fibonacci definida por $F_0 = 0$, $F_1 = 1$ e $F_k = F_{k-1} + F_{k-2}$ para $k \geq 2$. Então, para todo inteiro $k \geq 0$:*

$$F_{k+2} = 1 + \sum_{i=0}^{k} F_i.$$

Demonstração. Procedemos por indução matemática sobre k. *Base ($k = 0$)*: $1 + \sum_{i=0}^{0} F_i = 1 + F_0 = 1 + 0 = 1 = F_2$, verificando a igualdade. *Hipótese indutiva*: suponha que $F_{m+2} = 1 + \sum_{i=0}^{m} F_i$ valha para um dado $m \geq 0$. *Passo indutivo ($k = m+1$)*: separando o último termo do somatório e aplicando a hipótese indutiva, obtemos:

$$1 + \sum_{i=0}^{m+1} F_i = \left(1 + \sum_{i=0}^{m} F_i\right) + F_{m+1} = F_{m+2} + F_{m+1} = F_{m+3} = F_{(m+1)+2},$$

o que estabelece a validade da identidade para todo $k \geq 0$. □

Lema 2.3 (Cota exponencial pela razão áurea)**.** *Para todo inteiro $k \geq 0$, o $(k+2)$-ésimo número de Fibonacci satisfaz $F_{k+2} \geq \varphi^k$, onde $\varphi = \frac{1+\sqrt{5}}{2} \approx 1{,}61803$ é a razão áurea.*

Demonstração. Usamos indução forte sobre k. *Casos base*: para $k = 0$, $F_2 = 1 \geq \varphi^0 = 1$. Para $k = 1$, $F_3 = 2 \geq \varphi^1 \approx 1{,}61803$. *Passo indutivo*: seja $k \geq 2$ e suponha que $F_{j+2} \geq \varphi^j$ para todo $0 \leq j < k$. Como φ é a raiz positiva do polinômio $x^2 - x - 1 = 0$, vale $\varphi^2 = \varphi + 1$. Pela recorrência de Fibonacci e pela hipótese indutiva aplicada a $k-1$ e $k-2$:

$$F_{k+2} = F_{k+1} + F_k \geq \varphi^{k-1} + \varphi^{k-2} = \varphi^{k-2}(\varphi + 1) = \varphi^{k-2} \cdot \varphi^2 = \varphi^k.$$ □

Teorema 2.4 (Tamanho da subárvore e grau máximo $D(n)$)**.** *Seja x qualquer nó de um Heap de Fibonacci com n nós, e seja $k = grau[x]$. Então a subárvore enraizada em x contém pelo menos $F_{k+2} \geq \varphi^k$ nós. Consequentemente, o grau máximo $D(n)$ de qualquer nó satisfaz:*

$$D(n) \leq \lfloor \log_\varphi n \rfloor.$$

Demonstração. Denote por s_k o menor número possível de nós em qualquer subárvore cuja raiz tenha grau k em qualquer Heap de Fibonacci. Claramente $s_0 = 1$ e $s_1 = 2$, pois uma raiz de grau 0 possui a si mesma e uma raiz de grau 1 possui ao menos a si mesma e seu único filho. Note que s_d é monotonicamente não decrescente em d.

Para $k \geq 2$, contabilizando o próprio nó x (1 nó) mais os tamanhos das subárvores de seus k filhos $y_1, \ldots, y_k$ e aplicando a cota de grau do Lema 2.1 ($grau[y_1] \geq 0$ e $grau[y_i] \geq i-2$ para $i \geq 2$), temos:

$$\text{tamanho}(x) \geq 1 + s_0 + \sum_{i=2}^{k} s_{i-2} \implies s_k \geq 2 + \sum_{i=2}^{k} s_{i-2}.$$

Provemos por indução forte em k que $s_k \geq F_{k+2}$. Para $k = 0$, $s_0 = 1 = F_2$; para $k = 1$, $s_1 = 2 = F_3$. Supondo $s_j \geq F_{j+2}$ para todo $0 \leq j < k$ (com $k \geq 2$) e usando o Lema 2.2 (lembrando que $F_1 = 1$ e $F_0 = 0$):

$$s_k \geq 2 + \sum_{i=2}^{k} F_{(i-2)+2} = 2 + \sum_{i=2}^{k} F_i = 1 + F_0 + F_1 + \sum_{i=2}^{k} F_i = 1 + \sum_{i=0}^{k} F_i = F_{k+2}.$$

Pelo Lema 2.3, se o heap possui n nós no total, então $n \geq \text{tamanho}(x) \geq s_k \geq F_{k+2} \geq \varphi^k$. Tomando o logaritmo na base $\varphi > 1$ em ambos os membros, resulta $k \leq \log_\varphi n$. Como k é inteiro, conclui-se que $D(n) \leq \lfloor \log_\varphi n \rfloor = O(\log n)$. □

2.3 Operações preguiçosas em tempo constante: criação, mínimo, inserção e união

As operações que não removem elementos nem violam a ordem ascendente das árvores são executadas de maneira estritamente constante ($\Theta(1)$ real). A criação (CRIAR-HEAP, encapsulada no construtor da classe `HeapFibonacci`) inicializa $H.n \leftarrow 0$ e $H.min \leftarrow$ NIL. A consulta ao menor elemento (MINIMO) limita-se a retornar o ponteiro $H.min$ sem percorrer a lista de raízes.

Na operação INSERIR (Algoritmo 1), aloca-se um novo nó x com $grau[x] = 0$, $marca[x] =$ FALSO e ponteiros laterais apontando para si mesmo. Em vez de consolidar árvores de mesmo grau como faria um heap binomial, o procedimento auxiliar INSERIR-LISTA-CIRCULAR apenas encaixa x imediatamente à esquerda de $H.min$ e atualiza $H.min$ caso $chave[x] < chave[H.min]$. De forma análoga, UNIR-HEAPS concatena as duas listas circulares de raízes H_1 e H_2 em $\Theta(1)$, elege o menor entre $H_1.min$ e $H_2.min$, soma as quantidades de nós e transfere a posse da memória para o novo heap resultante. A Figura 2 ilustra a atualização dos quatro ponteiros durante uma inserção preguiçosa.

Algoritmo 1: Inserção preguiçosa e união em listas circulares

Entrada: Heap H, par $(chave, valor)$ para inserção; ou dois heaps H_1, H_2 para união.

Saída: Ponteiro para o nó criado x (em INSERIR) ou heap unido H (em UNIR-HEAPS).

```
1  Procedimento INSERIR(H, chave, valor)
2      aloque x com chave[x] ← chave, valor[x] ← valor, grau[x] ← 0,
        marca[x] ← FALSO
3      pai[x] ← NIL;
4      filho[x] ← NIL;
5      esquerda[x] ← x;
6      direita[x] ← x
7      se H.min = NIL então
8          H.min ← x
9      senão
10         u ← esquerda[H.min]
11         esquerda[H.min] ← x;
12         direita[x] ← H.min
13         esquerda[x] ← u;
14         direita[u] ← x
15         se chave[x] < chave[H.min] então
16             H.min ← x
17     H.n ← H.n + 1
18     retorne x
19 Procedimento UNIR-HEAPS(H_1, H_2)
20     H ← CRIAR-HEAP()
21     se H_1.min = NIL então
22         retorne posse transferida de H_2 para H
23     se H_2.min = NIL então
24         retorne posse transferida de H_1 para H
25     u_1 ← esquerda[H_1.min];
26     u_2 ← esquerda[H_2.min]
27     direita[u_1] ← H_2.min;
28     esquerda[H_2.min] ← u_1
29     direita[u_2] ← H_1.min;
30     esquerda[H_1.min] ← u_2
31     H.min ← arg min_{r∈{H_1.min,H_2.min}} chave[r]
32     H.n ← H_1.n + H_2.n;
33     esvazie descritores de H_1 e H_2
34     retorne H
```

(a) Antes de Inserir$(H, 1)$**(b) Após encaixe circular e ajuste de** $H.min$

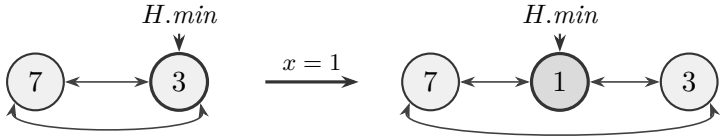


Figura 2 – Inserção preguiçosa na lista circular de raízes: o novo nó $x = 1$ é encaixado entre *esquerda*$[H.min]$ e $H.min$ em $\Theta(1)$, atualizando-se $H.min$ para 1. Elaborado pelos autores com base na implementação do grupo.

2.4 Extração do mínimo e consolidação por grau

A operação Extrair-Minimo (Algoritmo 2) é o único ponto em que o trabalho postergado pelas inserções anteriores é efetivamente realizado. Primeiro, cada filho da raiz mínima $z = H.min$ é desvinculado e promovido à lista de raízes com $pai \leftarrow$ NIL e $marca \leftarrow$ FALSO. Em seguida, remove-se z da lista circular; se z era o único nó do heap, $H.min$ torna-se NIL. Caso contrário, invoca-se o procedimento Consolidar.

Em nossa implementação, Consolidar copia inicialmente os ponteiros das raízes atuais para um vetor linear `listaRaizes`, garantindo iteração segura enquanto os ponteiros circulares são alterados, e aloca a tabela auxiliar de graus $A[0 \dots \lfloor \log_\varphi n \rfloor + 1]$ inicializada com NIL. Para cada raiz x, enquanto houver colisão $y = A[d] \neq$ NIL de mesmo grau d, compara-se $chave[x]$ com $chave[y]$, trocando as referências se necessário para manter x com a menor chave, e executa-se Heap-Link(y, x): y é removido da lista de raízes, inserido na lista circular de filhos de x com $marca[y] \leftarrow$ FALSO, e $d = grau[x]$ é incrementado em 1. Ao término das fusões, reconstrói-se a lista circular de raízes e o ponteiro $H.min$ a partir das entradas não nulas de A. A Figura 3 apresenta o fluxograma iterativo completo dessa etapa e o efeito estrutural do vínculo Heap-Link.

2.5 Diminuição de chave, cortes em cascata e remoção

A operação Diminuir-Chave (Algoritmo 4) sustenta a complexidade amortizada $O(1)$ exigida pelo relaxamento de arestas em Dijkstra. Dada uma referência direta ao nó x e uma nova chave $k \leq chave[x]$, atualiza-se $chave[x] \leftarrow k$. Se x for raiz ou se $chave[x] \geq chave[pai[x]]$, a propriedade de min-heap permanece intacta e basta testar se x tornou-se o novo $H.min$.

Se, contudo, $chave[x] < chave[y]$ para $y = pai[x]$, executa-se Corte(H, x, y), desvinculando x da lista circular de filhos de y, decrementando $grau[y]$ e movendo x diretamente para a lista de raízes com $pai[x] \leftarrow$ NIL e $marca[x] \leftarrow$ FALSO. Em seguida, aciona-se recursivamente Corte-Em-Cascata(H, y): caso y não seja raiz ($z = pai[y] \neq$ NIL), verifica-se $marca[y]$. Se y estava desmarcado, esta foi sua primeira perda de filho, bastando marcá-lo ($marca[y] \leftarrow$ VERDADEIRO) e encerrar a recursão; se já estava mar-

Algoritmo 2: Extração do mínimo e consolidação por arranjo de graus

Entrada: Heap de Fibonacci H.
Saída: Ponteiro z para o nó mínimo extraído (posse transferida ao chamador).

```
1  z ← H.min
2  se z ≠ NIL então
3      copie os filhos de z para o vetor auxiliar F
4      para cada w ∈ F faça
5          isole w (esquerda[w] ← w, direita[w] ← w) e insira w na lista de raízes de H
6          pai[w] ← NIL;
7          marca[w] ← FALSO
8      filho[z] ← NIL;
9      grau[z] ← 0
10     p ← direita[z];
11     remova z da lista circular de raízes
12     se z = p então
13         H.min ← NIL
14     senão
15         H.min ← p
16         Consolidar:
17         G_max ← ⌊log_2(H.n)/ log_2(φ)⌋ + 2
18         inicialize vetor A[0 . . . G_max − 1] com NIL e copie raízes atuais para R
19         para cada x ∈ R faça
20             d ← grau[x]
21             enquanto A[d] ≠ NIL faça
22                 y ← A[d]
23                 se chave[x] > chave[y] então
24                     troque x ↔ y
25                 remova y da lista de raízes e insira y na lista circular filho[x]
26                 pai[y] ← x;
27                 marca[y] ← FALSO;
28                 grau[x] ← grau[x] + 1
29                 A[d] ← NIL;
30                 d ← d + 1
31             A[d] ← x
32         H.min ← NIL
33         para i ← 0 até |A| − 1 faça
34             se A[i] ≠ NIL então
35                 insira A[i] na nova lista circular de raízes e atualize H.min se menor
36     H.n ← H.n − 1
37 retorne z
```

Algoritmo 3: Vínculo entre duas raízes de mesmo grau (HEAP-LINK)

Entrada: Heap H, raiz y e raiz x de mesmo grau com $x.key \leq y.key$
Saída: Nó y vinculado como filho de x

```
1 if x = NIL ou y = NIL then
2     return
3 remova y da lista circular de raízes de H
4 insira y na lista circular de filhos de x (x.child)
5 x.degree ← x.degree + 1
6 y.parent ← x
7 y.mark ← FALSE
```

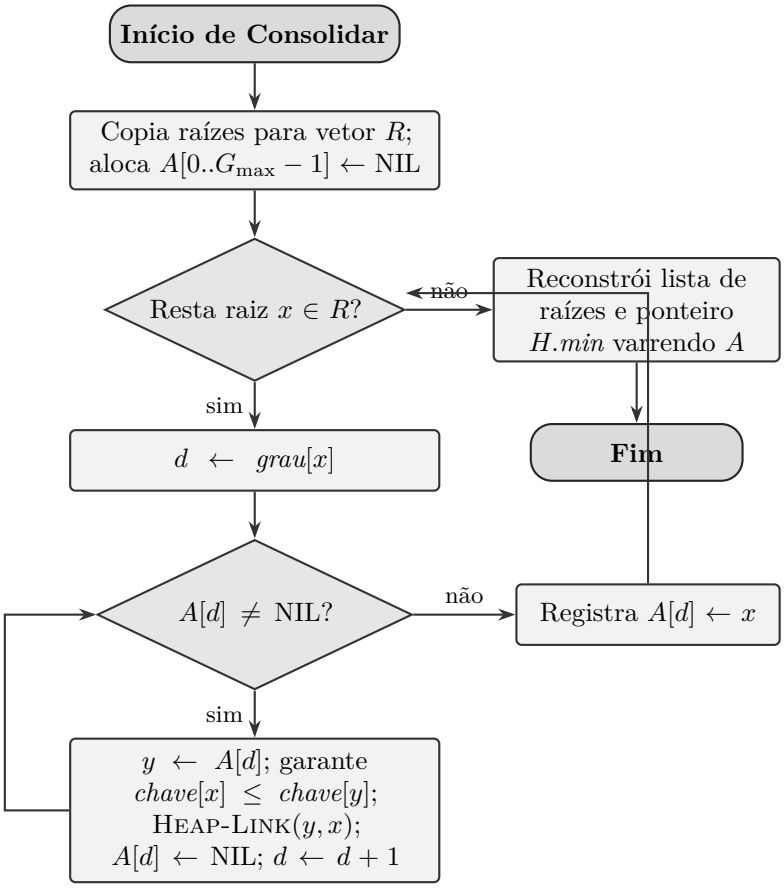


Figura 3 – Fluxograma iterativo do procedimento CONSOLIDAR: o laço interno resolve colisões de grau via HEAP-LINK até que cada posição $A[d]$ guarde no máximo uma árvore. Elaborado pelos autores com base na implementação do grupo.

cado, y também é cortado de z e a verificação propaga-se ao ancestral z. A operação Remover-No(H, x) é implementada pela composição Diminuir-Chave$(H, x, -\infty)$ seguida de Extrair-Minimo(H). Na implementação em C++, `INT_MIN` é reservado como sentinela interna e não pode ser uma chave válida; todas as chaves válidas devem ser estritamente maiores que esse valor. Sob essa precondição, a sentinela é menor que qualquer chave do heap e força x a se tornar raiz antes da extração. As Figuras 4 e 5 ilustram o estado de memória nos cortes em cascata e seu fluxograma recursivo.

Algoritmo 4: Diminuição de chave, corte simples e corte em cascata

Entrada: Heap H, ponteiro direto ao nó x e nova chave $k \leq \mathit{chave}[x]$.

```
1  Procedimento DIMINUIR-CHAVE(H, x, k)
2      se x = NIL ou k > chave[x] então
3          retorne
4      chave[x] ← k;
5      y ← pai[x]
6      se y ≠ NIL e chave[x] < chave[y] então
7          CORTE(H, x, y)
8          CORTE-EM-CASCATA(H, y)
9      se H.min = NIL ou chave[x] < chave[H.min] então
10         H.min ← x
11 Procedimento CORTE(H, x, y)
12     remova x da lista circular de filhos de y (ajustando filho[y] se necessário)
13     grau[y] ← grau[y] − 1
14     insira x na lista circular de raízes de H;
15     pai[x] ← NIL;
16     marca[x] ← FALSO
17 Procedimento CORTE-EM-CASCATA(H, y)
18     z ← pai[y]
19     se z ≠ NIL então
20         se marca[y] = FALSO então
21             marca[y] ← VERDADEIRO
22         senão
23             CORTE(H, y, z)
24             CORTE-EM-CASCATA(H, z)
```

2.6 Análise amortizada

Seja $t(H)$ o número de árvores na lista de raízes e $m(H)$ o número de nós marcados. Usamos a função potencial

$$\Phi(H) = t(H) + 2m(H),$$

que é não negativa. Para uma operação que transforma H em H', se c é seu custo real, definimos o custo amortizado por $\widehat{c} = c + \Phi(H') - \Phi(H)$. A análise a seguir complementa

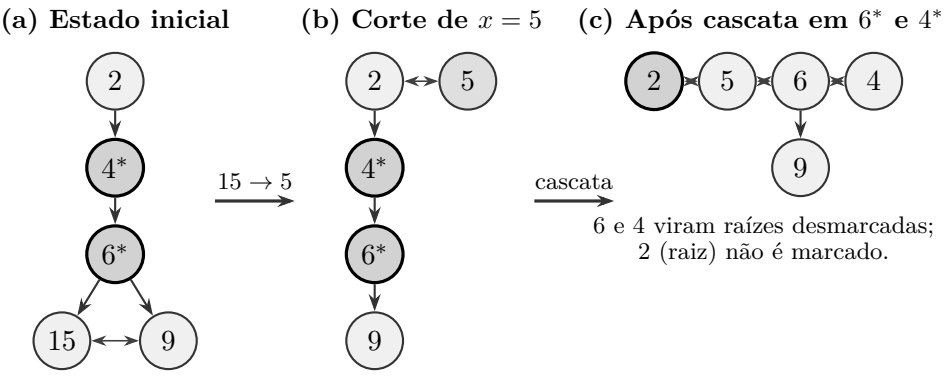


Figura 4 – Evolução da memória durante DIMINUIR-CHAVE$(H, x, 5)$: como $y = 6^*$ e $z = 4^*$ já estavam marcados, ambos sobem à lista de raízes com marca limpa (FALSO), liberando potencial acumulado para custear a cascata. Elaborado pelos autores com base em Cormen et al. (2009).

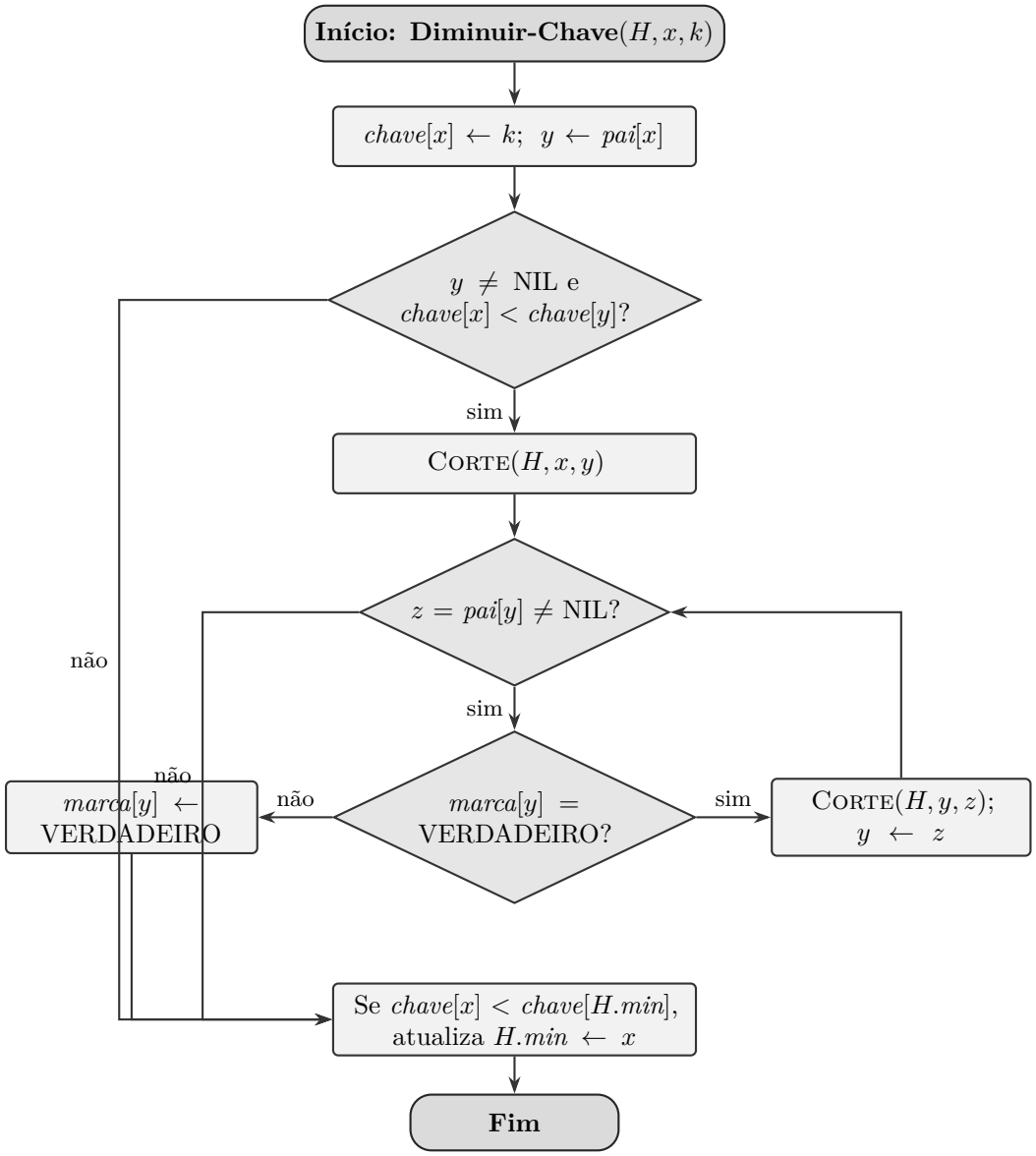


Figura 5 – Fluxograma de controle de DIMINUIR-CHAVE integrado à propagação recursiva de CORTE-EM-CASCATA. Elaborado pelos autores com base na implementação do grupo.

Algoritmo 5: Remoção de um nó em um heap de Fibonacci (DELETE)

Entrada: Heap H e nó $x \in H$; todas as chaves válidas são maiores que `INT_MIN`
Saída: Nó x removido, ou NIL se H estiver vazio ou $x = \text{NIL}$

```
1 if H.min = NIL ou x = NIL then
2     return NIL
3 DIMINUIR-CHAVE(H, x, −∞)
4 z ← EXTRAIR-MINIMO(H)
5 return z
```

a cota $D(n) = O(\log n)$ demonstrada anteriormente (CORMEN et al., 2009; FREDMAN; TARJAN, 1987).

A criação do heap e a consulta do mínimo têm custo real e amortizado $O(1)$, sem aumento de potencial. A inserção de um nó como raiz custa $O(1)$ e aumenta $t(H)$ em uma unidade, portanto também tem custo amortizado $O(1)$. Na união de dois heaps disjuntos, as listas de raízes são concatenadas em $O(1)$; como t e m no heap resultante são as somas dos valores dos heaps originais, a variação de potencial é zero e o custo amortizado permanece $O(1)$.

Na extração do mínimo, seja t o número inicial de raízes, d o grau do mínimo removido, L o número de vínculos realizados na consolidação e t' o número final de raízes. Ao remover o mínimo e adicionar seus filhos à lista de raízes, há $t + d - 1$ raízes antes dos vínculos; cada vínculo reduz essa quantidade em uma. Assim, $t' = t + d - 1 - L$ e $\Delta\Phi \leq t' - t = d - 1 - L$, pois o número de nós marcados não aumenta durante a operação. O custo real é $O(t + d)$, enquanto após a consolidação há no máximo $D(n) + 1$ raízes, uma por grau possível. A diminuição do potencial compensa o trabalho proporcional às raízes vinculadas, restando custo amortizado $O(D(n)) = O(\log n)$. O custo real de pior caso é $O(n)$.

Na diminuição de chave, se a violação da ordem de heap provoca $c \geq 1$ cortes, cada corte acrescenta uma raiz, enquanto os cortes em cascata removem nós marcados; no máximo um ancestral desmarcado é marcado ao fim da cascata. Portanto, $\Delta\Phi \leq c + 2(1 - (c - 1)) = 4 - c$. Como o custo real é $O(c + 1)$, a variação de potencial compensa a cascata e o custo amortizado é $O(1)$, embora o pior caso real seja $O(n)$. A remoção de um nó é a composição dessa operação com a extração do mínimo: seu custo amortizado é $O(\log n)$ e seu pior caso real é $O(n)$.

3 Árvores de van Emde Boas

Em sistemas computacionais, o processamento eficiente de dados é um desafio central no desenvolvimento de algoritmos e estruturas de dados. Aplicações como escalonamento de tarefas, simulação de eventos discretos e processamento em tempo real exigem conjuntos que possam ser atualizados por inserções e remoções sucessivas e consultados quanto aos seus valores e à ordem relativa dos elementos. Nesses cenários, o desempenho depende não apenas do armazenamento, mas também do custo de localizar, atualizar e recuperar elementos durante a execução.

Esses problemas podem ser modelados como conjuntos ordenados, nos quais é necessário identificar elementos extremos e determinar a posição relativa entre as chaves. Uma estrutura comum para esse fim é a fila de prioridade, que organiza os elementos por chave e permite inserções, consultas e remoções. Filas de prioridade podem ser implementadas, entre outras estruturas, por heaps binários, árvores rubro-negras e heaps de Fibonacci. Cada alternativa apresenta compromissos entre simplicidade de implementação e desempenho. Em particular, heaps binários e árvores balanceadas executam suas principais operações em tempo $O(\log n)$, em que n é o número de elementos armazenados. Embora adequado para muitas aplicações, esse custo motivou a busca por estruturas mais eficientes.

Nesse contexto, van Emde Boas, Kaas e Zijlstra (1977) apresentaram uma fila de prioridade, denominada *stratified tree*, baseada na decomposição recursiva de um universo finito de chaves. Em apresentações posteriores, essa organização passou a ser conhecida como árvore de van Emde Boas (vEB), que suporta busca, inserção, remoção, sucessor e predecessor (CORMEN et al., 2009). Essas operações são executadas em tempo $O(\log \log U)$, em que U representa a cardinalidade do universo de chaves. Esse desempenho torna a árvore vEB atraente para aplicações que exigem operações eficientes sobre conjuntos ordenados.

3.1 Conceitos iniciais

Antes de descrever a árvore de van Emde Boas, estabelecemos algumas definições e convenções. É importante distinguir a quantidade de elementos armazenados do tamanho do universo de chaves: n denota o número de elementos presentes no conjunto dinâmico; $\mathcal{U}$, o universo de chaves possíveis; e $U = |\mathcal{U}|$, sua cardinalidade. Assim, a complexidade $O(\log \log U)$ é expressa em função do tamanho do universo, e não da quantidade de elementos armazenados.

Assume-se que as chaves pertencem ao universo inteiro

$$\mathcal{U} = \{0, 1, 2, \ldots, U - 1\},$$

em que $U = |\mathcal{U}|$. Para simplificar a construção recursiva, considera-se geralmente que $U = 2^m$, para algum inteiro positivo m. Assim, cada chave pode ser representada por exatamente m bits, o que permite particionar o universo durante as operações. O objetivo é manter dinamicamente um subconjunto $S \subseteq \mathcal{U}$ e oferecer suporte a operações como inserção, remoção, consulta de pertencimento, obtenção do mínimo e do máximo e determinação do sucessor e do predecessor. Quando esse conjunto é usado como fila de prioridade, extrair o mínimo corresponde a remover o elemento devolvido pela consulta de mínimo.

Diversas estruturas de dados podem resolver esse problema. Árvores de busca binária balanceadas, como AVL e árvores rubro-negras, mantêm os elementos ordenados e oferecem suporte eficiente às operações consideradas. Outra alternativa é o endereçamento direto, em que cada chave possível é associada a uma posição de um vetor, permitindo acesso imediato. A árvore de van Emde Boas, por sua vez, explora diretamente a estrutura do universo de chaves e o decompõe recursivamente em subconjuntos menores. Assim, seus tempos de execução dependem do tamanho do universo U, e não apenas da quantidade de elementos armazenados.

3.1.1 Da busca por predecessores à estrutura proto-van Emde Boas

Para compreender a motivação das árvores de van Emde Boas, consideremos inicialmente um problema simples. Seja o conjunto

$$S = \{2, 3, 4, 5, 7, 14, 15\}$$

armazenado sobre o universo de chaves

$$\mathcal{U} = \{0, 1, \ldots, 15\},$$

cuja cardinalidade é $U = 16$.

chave x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$x \in S$	0	0	1	1	1	1	0	1	0	0	0	0	0	0	1	1

Figura 6 – Representação do subconjunto $S = \{2, 3, 4, 5, 7, 14, 15\}$ no universo $\mathcal{U} = \{0, 1, \ldots, 15\}$, cuja cardinalidade é $U = 16$. A linha superior mostra as chaves; na inferior, 1 indica que a chave pertence a S e 0 indica que não pertence. As células sombreadas destacam os elementos armazenados. Retirado e adaptado de (CORMEN et al., 2009).

Suponha que desejamos determinar o predecessor da chave 14, isto é, o maior elemento de S estritamente menor que 14. Uma forma direta de representar esse conjunto

consiste em utilizar endereçamento direto. Nesse modelo, cada chave do universo é associada a uma posição de um vetor de bits A. Uma posição recebe valor 1 quando a chave correspondente pertence ao conjunto e valor 0 caso contrário. A correspondência entre o universo de chaves e o subconjunto armazenado está representada na Figura 6.

Embora essa estrutura permita verificar a presença de uma chave em tempo constante, operações relacionadas à ordenação dos elementos permanecem custosas. Para encontrar o predecessor de 14, por exemplo, seria necessário percorrer o vetor em direção à esquerda até localizar a primeira posição contendo valor 1. Em um universo de tamanho U, esse procedimento pode exigir a inspeção de até U posições. Uma forma de reduzir esse custo consiste em armazenar informações resumidas sobre intervalos do universo. Para isso, pode-se sobrepor uma árvore binária completa ao vetor de bits. As folhas da árvore correspondem às posições do vetor A, enquanto cada nó interno representa um intervalo do universo e armazena o resultado da operação lógica OR aplicada a todos os elementos contidos em sua subárvore. Assim, um nó assume valor 1 sempre que existe pelo menos uma chave armazenada no intervalo representado e valor 0 caso contrário. A Figura 7 ilustra essa construção para o conjunto considerado. Os nós internos resumem informações sobre intervalos progressivamente maiores do universo, permitindo identificar rapidamente regiões que contêm elementos.

A busca pelo predecessor de 14 pode então ser realizada explorando essa hierarquia. Partindo da folha correspondente à chave 14, o algoritmo sobe pela árvore até encontrar um ancestral a partir do qual seja possível alcançar um intervalo à esquerda contendo

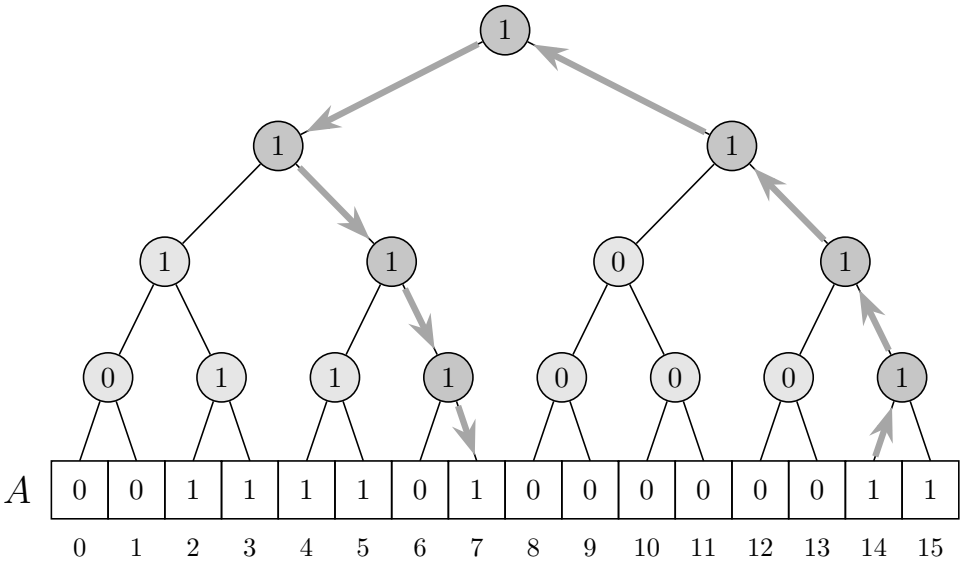


Figura 7 – Representação hierárquica do conjunto por endereçamento direto. O vetor de bits A, na base, indica a presença (1) ou ausência (0) das chaves de $\mathcal{U} = \{0, \ldots, 15\}$. Cada nó interno resume um intervalo e vale 1 quando esse intervalo contém ao menos uma chave do conjunto. As setas mostram a busca do predecessor de 14, que termina na chave 7. Retirado e adaptado de (CORMEN et al., 2009).

algum elemento armazenado. Em seguida, realiza uma descida guiada pelos valores dos nós internos até encontrar a maior chave presente naquele intervalo. No exemplo, esse percurso conduz à chave 7, que corresponde ao predecessor procurado. Essa estratégia substitui uma busca linear no vetor por uma navegação em uma árvore de altura $\Theta(\log U)$. Consequentemente, operações como predecessor e sucessor podem ser executadas em tempo $O(\log U)$. Mais importante do que a melhoria assintótica obtida é a mudança de perspectiva introduzida pela estrutura: o universo deixa de ser interpretado como um conjunto de posições independentes e passa a ser visto como uma hierarquia de intervalos.

Entretanto, mesmo após essa transformação, a altura da árvore continua crescendo proporcionalmente a $\log U$. Surge então uma questão natural: seria possível reduzir ainda mais a profundidade da estrutura? Uma possibilidade consiste em aumentar o grau de ramificação da árvore. Suponha que o universo possua tamanho

$$U = 2^{2k},$$

de modo que $\sqrt{U}$ seja inteiro. Em vez de dividir cada intervalo em apenas duas partes, podemos particionar o universo em $\sqrt{U}$ grupos, cada um contendo $\sqrt{U}$ chaves. No exemplo em estudo, em que $U = 16$, obtém-se

$$\sqrt{U} = 4.$$

O universo passa então a ser dividido nos grupos

$$\{0, 1, 2, 3\}, \quad \{4, 5, 6, 7\}, \quad \{8, 9, 10, 11\}, \quad \{12, 13, 14, 15\}.$$

Cada grupo é denominado *cluster*. Para o conjunto armazenado, os clusters 0, 1 e 3 contêm elementos, enquanto o cluster 2 permanece vazio. Para registrar quais clusters possuem elementos armazenados, introduz-se uma estrutura auxiliar denominada *summary*. Essa estrutura contém uma posição para cada cluster e armazena valor 1 quando o cluster correspondente não está vazio.

No exemplo considerado, o vetor *summary* é representado por

$$[1, 1, 0, 1].$$

A decomposição ilustrada na Figura 8 reduz significativamente a altura da estrutura. Em vez de percorrer diversos níveis de uma árvore binária, torna-se possível localizar rapidamente o cluster relevante e utilizar o *summary* para identificar quais blocos do universo contêm elementos.

A introdução dos conceitos de *summary* e *cluster* constitui o primeiro passo em direção às estruturas proto-van Emde Boas. Essa primeira decomposição, porém, não resolve por si só o problema: cada *cluster* continua representando um conjunto ordenado sobre um universo de tamanho $\sqrt{U}$ e precisa suportar as mesmas operações de consulta,

sucessor e predecessor. Para evitar que essas operações exijam uma nova busca linear dentro de cada cluster, aplica-se recursivamente a decomposição ao *summary* e a cada *cluster*.

A Figura 9 ilustra uma instância proto-vEB de universo com cardinalidade u: o *summary* registra os clusters não vazios, enquanto cada posição do vetor *cluster* aponta para uma subestrutura responsável pela faixa correspondente de chaves.

Para formalizar o roteamento das chaves, denotemos por U a cardinalidade do universo original. Supondo $U = 2^{2k}$, para algum inteiro positivo k, temos $\sqrt{U}$ inteiro; para cada chave $x \in \{0, \ldots, U-1\}$, definem-se as coordenadas

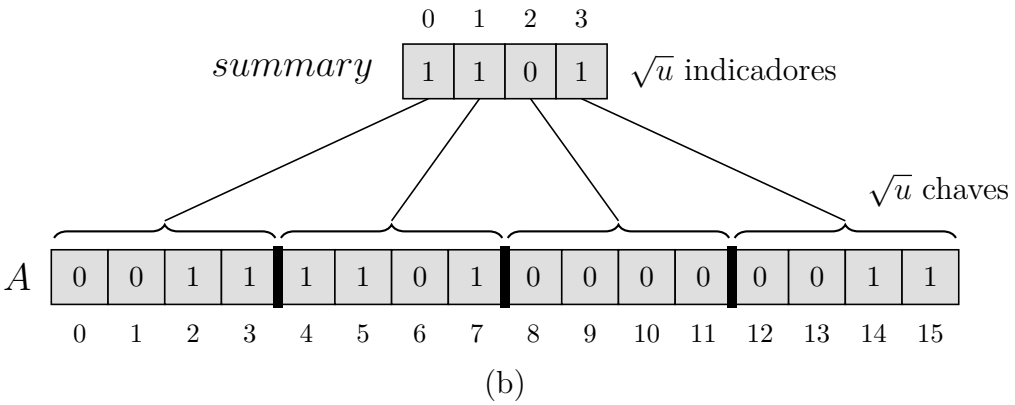
$$\text{high}(x) = \left\lfloor \frac{x}{\sqrt{U}} \right\rfloor, \qquad \text{low}(x) = x \bmod \sqrt{U}.$$

O quociente $\text{high}(x)$ seleciona o cluster, enquanto o resto $\text{low}(x)$ determina a posição local dentro dele; em conjunto, eles satisfazem $x = \text{high}(x)\sqrt{U} + \text{low}(x)$. Essa decomposição explica como percorrer os ponteiros representados na figura: primeiro se escolhe uma entrada do vetor *cluster* e, em seguida, a posição dentro da subestrutura apontada por ela.

Com esse roteamento, tanto o *summary* quanto cada *cluster* operam sobre universos de tamanho $\sqrt{U}$. O *summary* armazena os índices dos clusters não vazios; cada cluster armazena as posições locais das chaves de sua faixa. Embora tenham funções distintas, ambos mantêm subconjuntos dinâmicos de universos menores e, portanto, podem ser representados pela mesma estrutura recursiva.

Para estimar a profundidade, considere primeiro o caso idealizado em que cada nível reduz exatamente o universo pela raiz quadrada: após um nível, obtém-se $\sqrt{U}$; após dois, $U^{1/4}$; e, após t níveis, $U^{1/2^t}$. A recursão termina quando o universo alcança o caso-base de tamanho 2. Assim,

$$U^{1/2^t} = 2 \quad \Longrightarrow \quad \frac{\log_2 U}{2^t} = 1 \quad \Longrightarrow \quad t = \log_2(\log_2 U).$$



(b)

Figura 8 – Decomposição do universo de cardinalidade $U = 16$ em quatro *clusters* de quatro chaves. O vetor *summary* indica quais clusters estão ocupados. Retirado e adaptado de (CORMEN et al., 2009).

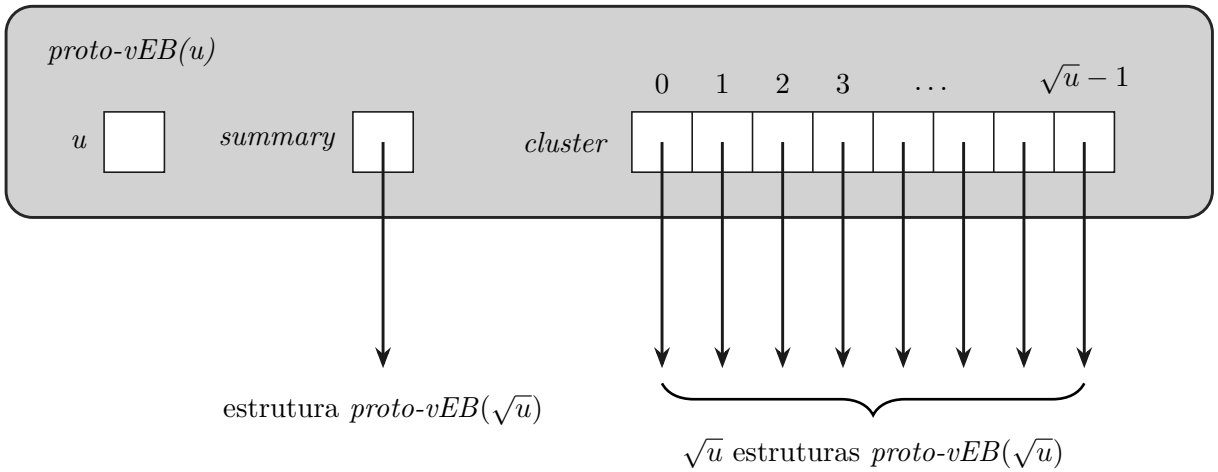


Figura 9 – Organização genérica de uma instância *proto-vEB* sobre um universo de cardinalidade u. O *summary* aponta para uma subestrutura que registra os clusters ocupados. Cada uma das $\sqrt{u}$ entradas do vetor *cluster* aponta para uma subestrutura responsável pela faixa correspondente de chaves. Retirado e adaptado de (CORMEN et al., 2009).

Logo, nesse caso idealizado, a profundidade é $\Theta(\log \log U)$. Para universos cujo número de bits não permite divisões exatamente iguais em todos os níveis, a implementação divide os bits entre as partes com comprimentos que diferem por no máximo uma unidade; a redução continua ocorrendo aproximadamente pela metade, preservando a mesma ordem de profundidade. Esse resultado explica a escala da recursão; a análise do custo de cada operação também depende do trabalho realizado em cada nível.

A Figura 10 mostra essa decomposição para o universo de 16 chaves e o conjunto $S = \{2, 3, 4, 5, 7, 14, 15\}$. Na raiz, o universo é dividido em quatro *clusters*, cada um responsável por quatro chaves; o *summary* registra quais deles não estão vazios — neste exemplo, os clusters 0, 1 e 3. O percurso da chave 14 ilustra o roteamento: como $\text{high}(14) = 3$ e $\text{low}(14) = 2$, consulta-se o cluster 3 e, nele, a posição local 2. Essa posição é decomposta novamente: no universo de tamanho 4, $\text{high}(2) = 1$ e $\text{low}(2) = 0$, levando ao segundo subcluster e, por fim, à posição 0 do vetor de bits da subestrutura de caso-base, onde a chave 14 está marcada como presente. O *summary* desse cluster também indica que o subcluster 1 está ocupado; assim, as células e setas da figura mostram tanto onde os elementos são armazenados quanto como os resumos orientam a navegação entre as subestruturas.

3.2 Definição da árvore vEB

As estruturas proto-van Emde Boas introduzem a decomposição recursiva do universo em um *summary* e em *clusters*. A árvore de van Emde Boas, ou árvore vEB, conserva essa organização para manter conjuntos ordenados e responder a consultas sobre seus elementos. Sua principal diferença é armazenar explicitamente o mínimo e o máximo

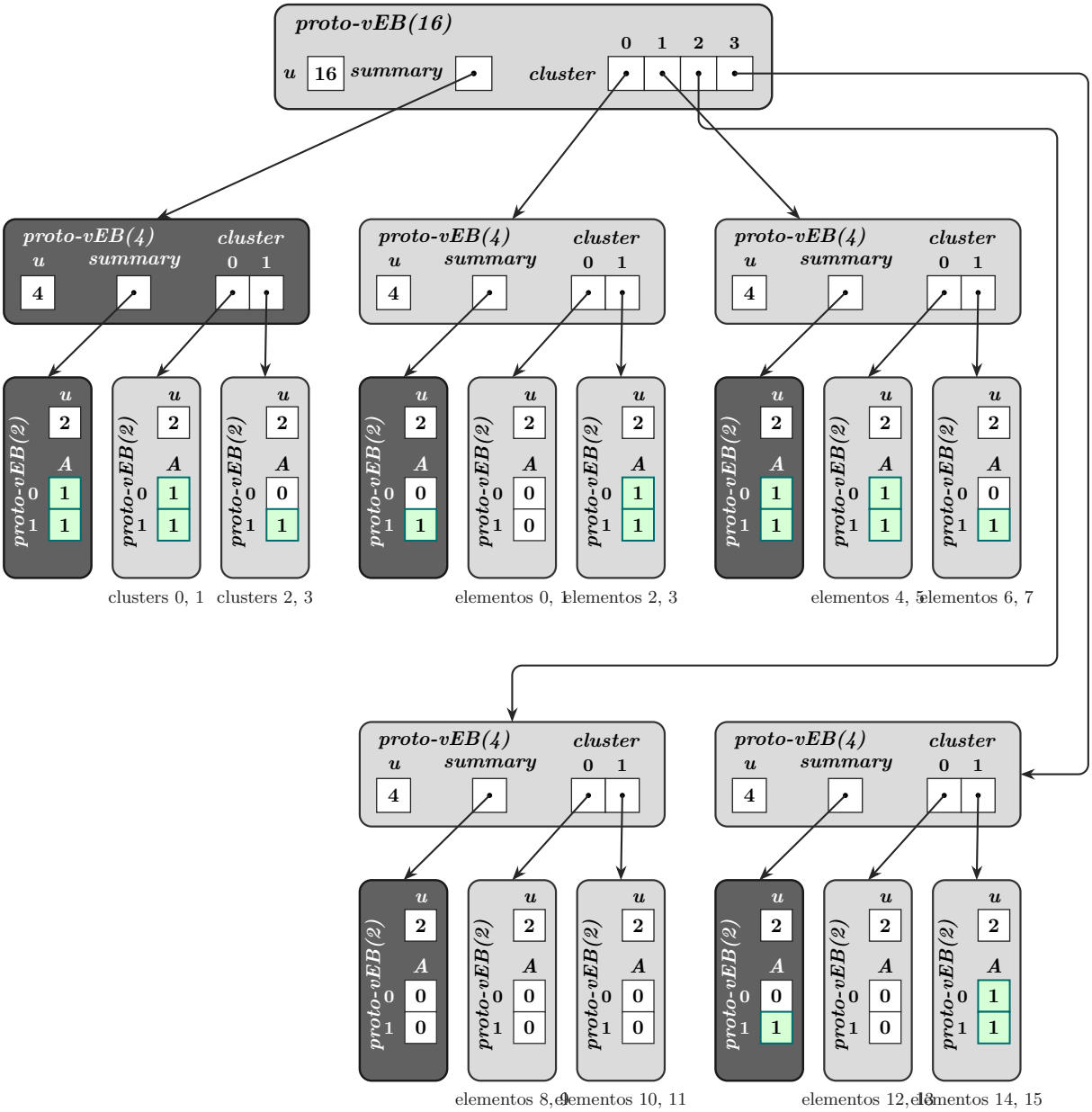


Figura 10 – Estrutura *proto-vEB*(16) que representa $S = \{2, 3, 4, 5, 7, 14, 15\}$. A raiz aponta para quatro estruturas *proto-vEB*(4), uma por cluster, e para um *summary* do mesmo tipo. Cada estrutura *proto-vEB*(4) aponta para duas estruturas de cluster e uma de *summary*, todas *proto-vEB*(2), com vetores de dois bits. Nas folhas sob "elementos i, j", os bits indicam a presença dessas chaves em S; sob "clusters i, j", indicam quais clusters da raiz estão ocupados. O sombreado escuro destaca as estruturas de resumo, cuja organização é igual à das demais estruturas de mesmo universo. Retirado e adaptado de (CORMEN et al., 2009).

de cada subestrutura e manter o mínimo fora da recursão. Assim, esse elemento não precisa ser redistribuído nem redescoberto nos níveis inferiores, enquanto os demais são organizados recursivamente. A descrição a seguir considera universos inteiros cuja cardinalidade é uma potência de dois e segue a apresentação de Cormen et al. (2009). Nessa descrição, u denota o universo de uma instância genérica, e U, o universo da raiz.

Uma instância vEB(u) representa um subconjunto de $\{0, 1, \ldots, u-1\}$. No caso-base $u = 2$, os campos min e max bastam para representar o conjunto, sem necessidade de criar *summary* ou *clusters*. Para $u > 2$, a instância mantém, além desses campos, um ponteiro *summary* e um vetor *cluster*. O campo min guarda o menor elemento, que não é incluído nas subestruturas; o campo max registra o maior. Cada chave restante é encaminhada ao cluster correspondente, e o *summary* registra quais clusters não estão vazios.

A Figura 11 apresenta essa organização para um universo genérico. Essa forma geral substitui a restrição $U = 2^{2k}$ usada acima. Se $u = 2^m$, definem-se $\sqrt[\uparrow]{u} = 2^{\lceil m/2 \rceil}$ e $\sqrt[\downarrow]{u} = 2^{\lfloor m/2 \rfloor}$. O vetor *cluster* possui $\sqrt[\uparrow]{u}$ entradas, cada uma apontando para uma subárvore de universo $\sqrt[\downarrow]{u}$; o *summary* representa os índices dos clusters ocupados em um universo de tamanho $\sqrt[\uparrow]{u}$. Para uma chave x, o índice do cluster e a posição local são dados por $\text{high}(x) = \lfloor x / \sqrt[\downarrow]{u} \rfloor$ e $\text{low}(x) = x \bmod \sqrt[\downarrow]{u}$, respectivamente. Quando $u = 16$, ambos os valores são 4, como na decomposição proto-vEB apresentada anteriormente.

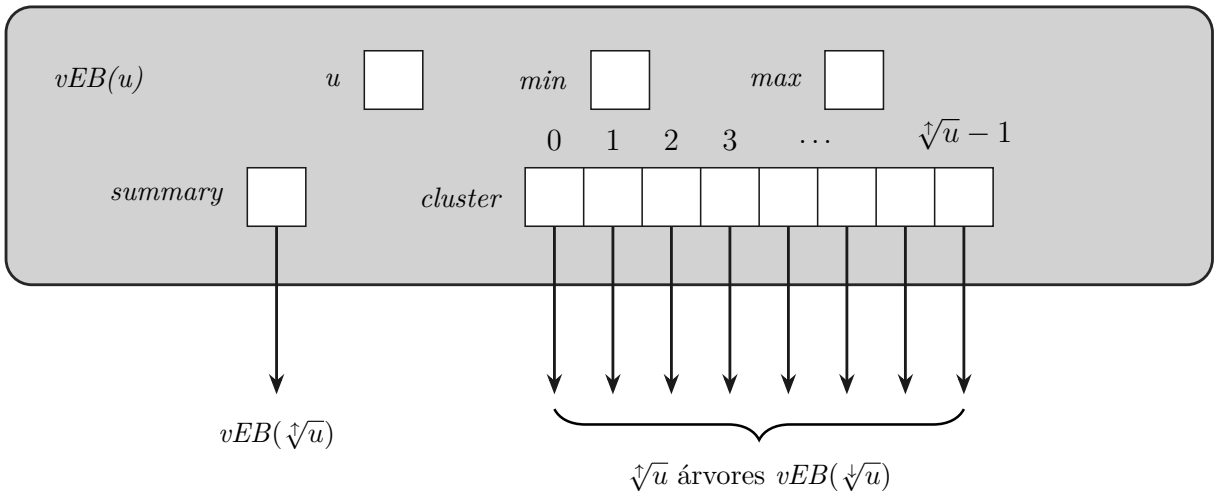


Figura 11 – Campos de uma árvore vEB(u), para $u > 2$. A estrutura armazena o tamanho do universo u, os campos *min* e *max*, um ponteiro *summary* para uma árvore $vEB(\sqrt[\uparrow]{u})$ e um vetor *cluster* com $\sqrt[\uparrow]{u}$ ponteiros para subárvores $vEB(\sqrt[\downarrow]{u})$. Manter *min* fora da recursão evita processamento redundante e permite operações em $O(\lg \lg u)$. Retirado e adaptado de (CORMEN et al., 2009).

Essa organização permite explorar a hierarquia sem percorrer todas as chaves. As consultas de mínimo e máximo usam diretamente os campos correspondentes e, por isso, levam tempo $O(1)$. Para localizar uma chave, a árvore usa $\text{high}(x)$ para escolher um cluster e $\text{low}(x)$ para consultar a posição local. Para determinar o sucessor ou o predecessor, consulta primeiro o cluster correspondente; se ele não contiver a resposta, o *summary*

orienta a busca ao próximo cluster ocupado ou ao anterior. Inserções e remoções atualizam o cluster afetado e, quando sua ocupação muda, também atualizam o *summary*. Como cada nível reduz aproximadamente pela metade o número de bits do universo, a recursão tem profundidade $O(\log \log u)$; busca, sucessor, predecessor, inserção e remoção têm essa mesma ordem de complexidade no pior caso.

Para ilustrar essa organização em um caso concreto, a Figura 12 apresenta a árvore vEB para o conjunto $S = \{2, 3, 4, 5, 7, 14, 15\}$, definido sobre o universo $\mathcal{U} = \{0, 1, \ldots, 15\}$, cuja cardinalidade é $U = 16$. O exemplo permite acompanhar como os campos da raiz, os índices dos clusters e as posições locais representam as chaves armazenadas.

Na raiz da Figura 12, os campos indicam min = 2 e max = 15. Como o mínimo fica fora da recursão, os clusters representam apenas os elementos de $S \setminus \{2\}$: o cluster 0 contém a chave 3; o cluster 1 contém 4, 5 e 7; o cluster 2 está vazio; e o cluster 3 contém 14 e 15. Assim, o *summary* da raiz representa os índices $\{0, 1, 3\}$, correspondentes aos clusters não vazios.

Em cada cluster, a chave é decomposta em um índice de cluster e uma posição local. Como $U = 16$, temos $\text{high}(x) = \lfloor x/4 \rfloor$ e $\text{low}(x) = x \bmod 4$. Assim, as chaves 4, 5 e 7 pertencem ao cluster 1 e ocupam, nessa subárvore, as posições locais 0, 1 e 3. A subárvore mantém seus próprios campos min e max, além de um *summary* e de clusters para representar recursivamente as demais posições. Na base da figura, as subárvores de universo 2 encerram a decomposição.

Esse exemplo também permite acompanhar a busca pelo predecessor de 14, cujo resultado foi apresentado anteriormente. Na raiz, $\text{high}(14) = 3$ identifica o cluster 3, e $\text{low}(14) = 2$ indica a posição local consultada. Como 2 coincide com o mínimo local desse cluster, não há predecessor dentro dele. A consulta recorre, então, ao *summary* da raiz para localizar o cluster ocupado anterior, que é o cluster 1. O maior valor local desse cluster é 3; ao recompor a chave, obtemos $1 \cdot 4 + 3 = 7$. Dessa maneira, o *summary* permite saltar entre clusters ocupados, sem examinar as posições vazias intermediárias.

Em síntese, a árvore vEB combina a decomposição recursiva da proto-vEB com o armazenamento explícito dos extremos e a exclusão do mínimo das subestruturas. Para um universo inteiro limitado, essa organização permite consultar mínimo e máximo em $O(1)$ e executar busca, sucessor, predecessor, inserção e remoção em $O(\log \log U)$, no pior caso. Esse ganho temporal deve ser considerado junto ao custo de espaço: na representação densa ilustrada pelas Figuras 11 e 12, a alocação das subestruturas pode exigir espaço proporcional a U, mesmo quando poucos elementos estão armazenados.

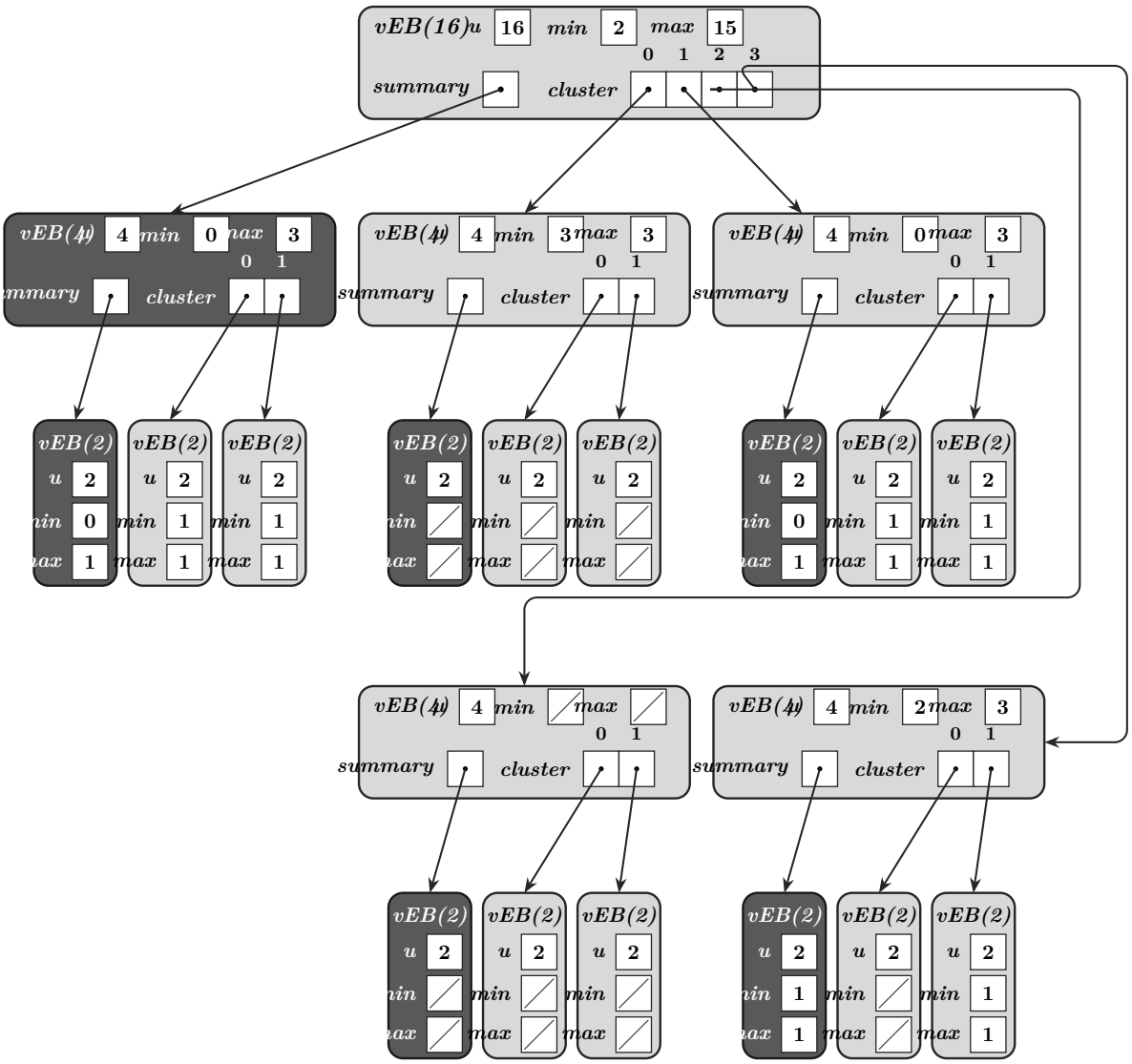


Figura 12 – Árvore de van Emde Boas $vEB(16)$ para $S = \{2, 3, 4, 5, 7, 14, 15\}$. O mínimo (2) é armazenado apenas no campo *min* da raiz, fora das subestruturas recursivas, permitindo consultá-lo em $O(1)$. Os blocos escuros representam as estruturas *summary*, que registram os índices dos clusters não vazios; os traços diagonais indicam campos vazios (*NIL*). Retirado e adaptado de (CORMEN et al., 2009).

3.3 Paradigma Algorítmico

A árvore vEB realiza operações de pertinência, mínimo, máximo, sucessor, predecessor, inserção e remoção, que abordaremos nesta ordem. Com elas, também construímos a extração do mínimo, operação básica de uma fila de prioridade, tratada ao final. Para facilitar o entendimento, detalharemos cada operação em quatro passos: explicação do conceito, algoritmo, fluxograma (quando útil para visualizar as decisões) e um teste utilizando o conjunto $S = \{2, 3, 4, 5, 7, 14, 15\}$, ilustrado na Figura 12. A análise de custo matemático ficará para a Seção 3.4.

Para a instância V de universo $V.u = 2^m$, cada cluster tem tamanho $s = 2^{\lfloor m/2 \rfloor} = \sqrt[\downarrow]{V.u}$. Assim, a fórmula $\text{index}(j, l) = j \cdot s + l$ reconstrói a chave a partir do índice do cluster j e da posição local l.

3.3.1 Pertinência

Para descobrir se um elemento x pertence ao conjunto S, a operação verifica primeiro as informações do próprio nó onde está, antes de descer para os níveis inferiores. Se $V.min = \text{NIL}$, significa que a estrutura está vazia, então a resposta é imediatamente negativa. Se x for igual a $V.min$ ou a $V.max$, a busca termina com sucesso ali mesmo, sem precisar consultar nenhum cluster. Checar o mínimo é obrigatório, já que ele não fica guardado dentro dos clusters; já checar o máximo funciona como um atalho para poupar processamento. No caso mais simples (quando $u = 2$), esses dois campos representam tudo o que o conjunto possui. Se x não for igual a nenhum dos dois, é porque não faz parte do conjunto. Para todos os outros casos, o algoritmo calcula $\text{high}(x)$ para saber em qual cluster entrar e $\text{low}(x)$ para saber a posição local que deve procurar. Se esse cluster destino estiver vazio, a recursão já retorna falso logo no primeiro teste.

O Algoritmo 6 descreve esse passo a passo formalmente, enquanto a Figura 13 ilustra essas decisões em um fluxograma.

Algoritmo 6: Pertinência de x em uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

Entrada: Estrutura V de universo u e chave x, com $0 \leq x < u$

```
1 if V.min = NIL then
2     return Falso
3 if x = V.min ou x = V.max then
4     return Verdadeiro
5 if u = 2 then
6     return Falso
7 return MEMBER(c_high(x), low(x))
```

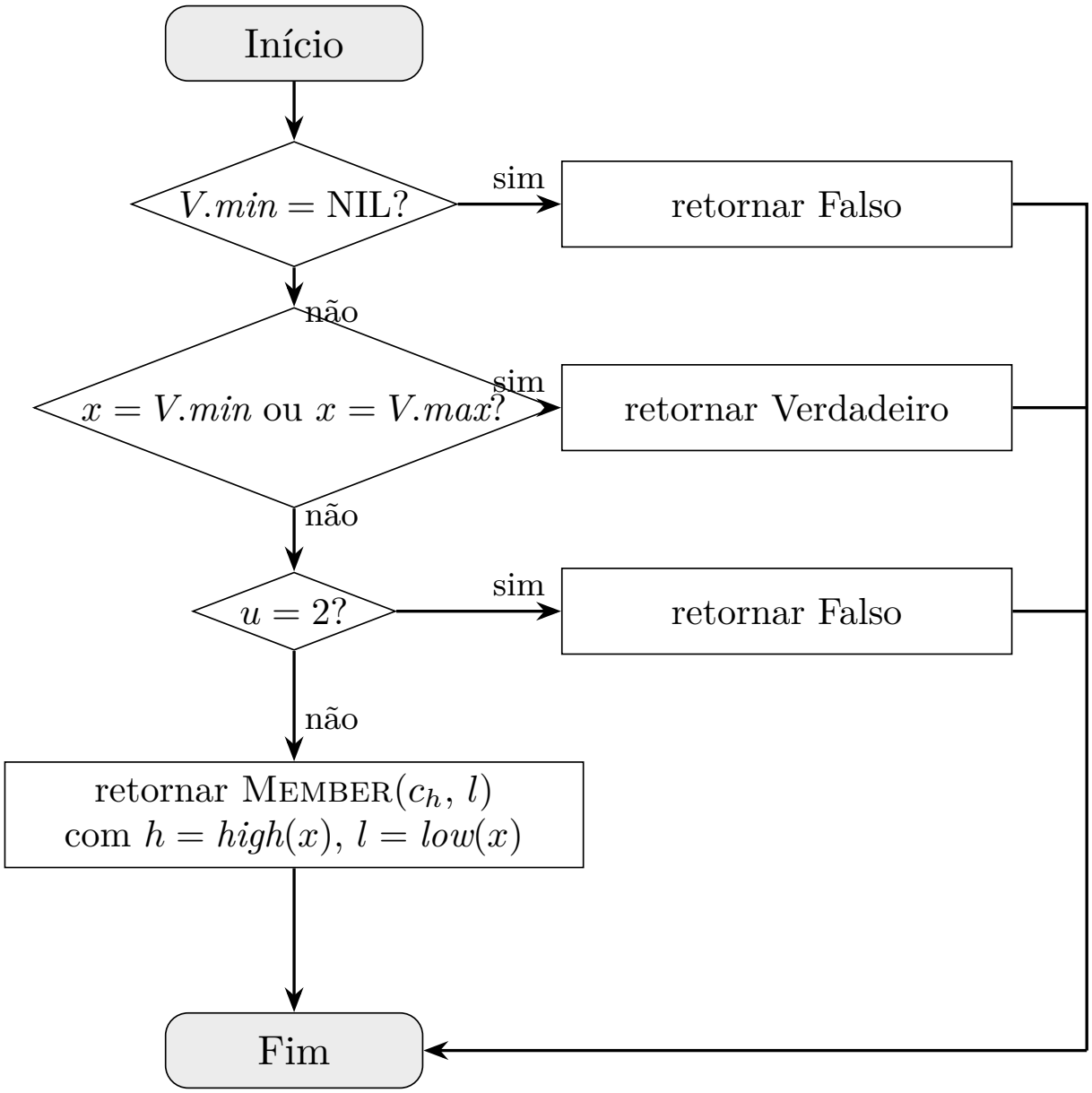


Figura 13 – Fluxograma da pertinência em vEB(u). Os ramos sem chamada recursiva terminam com resposta imediata, obtida dos campos *min* e *max*. Retirado e adaptado de (CORMEN et al., 2009).

Para ilustrar como o algoritmo funciona na prática, vamos acompanhar um teste buscando o número 5 (que pertence ao conjunto) e o 6 (que não pertence) na vEB(16) ilustrada na Figura 12. A Tabela 1 registra cada passo dessas buscas. Repare que, nas chamadas intermediárias, o valor procurado (x) não coincide nem com o *min* nem com o *max* local, o que força o algoritmo a descer para o cluster $h = \text{high}(x)$ buscando a posição $l = \text{low}(x)$. Quando entramos em um nível menor, como uma vEB(4), a chave local l é novamente decomposta utilizando as operações $\text{high}(l) = \lfloor l/2 \rfloor$ e $\text{low}(l) = l \bmod 2$.

Tabela 1 – Execução da pertinência sobre a vEB(16) da Figura 12.

Chamada	u	*min*	*max*	Decisão
Busca por $x = 5$				
MEMBER$(V, 5)$	16	2	15	$h = 1$, $l = 1$; desce
MEMBER$(c_1, 1)$	4	0	3	$h = 0$, $l = 1$; desce
MEMBER$(c_0, 1)$	2	1	1	$x = min$: Verdadeiro
Busca por $x = 6$				
MEMBER$(V, 6)$	16	2	15	$h = 1$, $l = 2$; desce
MEMBER$(c_1, 2)$	4	0	3	$h = 1$, $l = 0$; desce
MEMBER$(c_1, 0)$	2	1	1	$x \neq min$, $u = 2$: Falso

Vale notar que algumas buscas terminam muito rápido, sem exigir descidas na árvore. Por exemplo, ao buscar $x = 2$ ou $x = 15$, os próprios campos *min* e *max* da raiz já validam a presença do número de imediato. Por outro lado, uma busca por $x = 8$ tentaria acessar o cluster 2, que está vazio, o que faria a chamada recursiva retornar 'Falso' logo na primeira verificação.

3.3.2 Mínimo e máximo

Os menores e maiores valores do conjunto já ficam salvos direto nos campos $V.min$ e $V.max$ de cada nó. Por conta disso, descobrir o mínimo ou o máximo é muito rápido: basta ler o valor do campo correspondente, sem precisar de repetições (laços) ou de descidas na árvore (recursão). Se a estrutura estiver vazia, ambos os campos estarão marcados como `NIL`, que é exatamente o que a operação vai retornar. Como o fluxograma se resumiria a um único bloco de resposta, optamos por omiti-lo.

O Algoritmo 7 detalha como essas leituras funcionam.

Algoritmo 7: Mínimo e máximo de uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

Entrada: Estrutura V
1 MINIMUM(V)
2 **return** $V.min$
3 MAXIMUM(V)
4 **return** $V.max$

Usando a vEB(16) da Figura 12 como exemplo, o comando MINIMUM(V) retorna 2 e o MAXIMUM(V) retorna 15, pegando a informação diretamente da raiz. A Tabela 2 traz exemplos de consultas feitas dentro de clusters específicos. Nesses casos internos, a operação devolve apenas uma posição local. Para descobrir qual é o valor correspondente no conjunto original S, recompomos a chave usando a fórmula index(j, l), lembrando que aqui o tamanho é $s = 4$.

Tabela 2 – Execução de mínimo e máximo sobre a vEB(16) da Figura 12.

Chamada	u	Retorno	Chave em S
MINIMUM(V)	16	2	2
MAXIMUM(V)	16	15	15
MINIMUM(c_1)	4	0	index$(1, 0) = 1 \cdot 4 + 0 = 4$
MAXIMUM(c_3)	4	3	index$(3, 3) = 3 \cdot 4 + 3 = 15$
MINIMUM(c_2)	4	NIL	cluster vazio

3.3.3 Sucessor

O sucessor de um número x é simplesmente o próximo elemento válido do conjunto (ou seja, o menor valor que seja maior que x). A busca por esse "vizinho"é dividida em alguns cenários práticos, e a estrutura do algoritmo garante que ele fará, no máximo, uma descida recursiva por nível.

Primeiro, se a estrutura estiver vazia, não há sucessor. No caso mais básico ($u = 2$), o sucessor só existe se estivermos buscando a partir do 0 e o máximo armazenado for 1. Se o x procurado for menor que o $V.min$ atual, a resposta é o próprio mínimo. Afinal, ele é o menor elemento daquele nível e está logo ali, guardado fora dos clusters.

Se o valor procurado for maior que o mínimo, a busca precisa olhar os clusters. O algoritmo verifica o cluster destino high(x). Se esse cluster tiver algum elemento com posição local maior que low(x), sabemos que o sucessor está ali dentro, e a busca desce para ele (Vale notar que, se o cluster estiver vazio, seu máximo será `NIL` e essa opção é logo descartada).

Por outro lado, se o sucessor não estiver no mesmo cluster, precisamos procurar nos clusters seguintes. Para isso, o algoritmo usa o *summary* para descobrir qual é o próximo cluster ocupado, por exemplo, o cluster j. A resposta será exatamente o valor mínimo guardado dentro desse cluster j. Se o *summary* avisar que não há mais nenhum cluster ocupado à frente, significa que não existe sucessor para x.

O Algoritmo 8 formaliza esse passo a passo, e a Figura 14 mostra o caminho das decisões em um fluxograma.

Para visualizar o algoritmo funcionando, vamos simular a busca pelo sucessor de 5 e de 7 na vEB(16) que usamos na Figura 12. A Tabela 3 registra as chamadas. Nela, cada etapa SUCCESSOR(V', x) foi abreviada como (V', x), e a palavra *summary* virou *sum.* Em

Algoritmo 8: Sucessor de x em uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

Entrada: Estrutura V de universo u e chave x, com $0 \leq x < u$

```
1  if V.min = NIL then
2      return NIL
3  if u = 2 then
4      if x = 0 e V.max = 1 then
5          return 1
6      return NIL
7  if x < V.min then
8      return V.min
9  h ← high(x); l ← low(x)
10 if c_h.max ≠ NIL e l < c_h.max then
11     o ← SUCCESSOR(c_h, l)
12     return index(h, o)
13 j ← SUCCESSOR(V.summary, h)
14 if j = NIL then
15     return NIL
16 return index(j, c_j.min)
```

cada linha, extraímos o $h = \text{high}(x)$ e o $l = \text{low}(x)$, e a grande decisão é feita comparando a posição local l com o máximo disponível naquele cluster.

Tabela 3 – Execução do sucessor sobre a vEB(16) da Figura 12.

(V', x)	u	min	max	Decisão
Sucessor de $x = 5$				
$(V, 5)$	16	2	15	$h = 1$, $l = 1$; $1 < 3$: desce
$(c_1, 1)$	4	0	3	$h = 0$, $l = 1$; $1 \geq 1$: vai ao *sum*
$(c_1.sum, 0)$	2	0	1	$x = 0$, $max = 1$: devolve 1
Sucessor de $x = 7$				
$(V, 7)$	16	2	15	$h = 1$, $l = 3$; $3 \geq 3$: vai ao *sum*
$(V.sum, 1)$	4	0	3	$h = 0$, $l = 1$; $1 \geq 1$: vai ao *sum*
$(V.sum.sum, 0)$	2	0	1	$x = 0$, $max = 1$: devolve 1

Ao terminar a primeira busca e desfazer as chamadas, o cluster c_1 devolve o valor reconstituído $\text{index}(1, 1) = 1 \cdot 2 + 1 = 3$. Chegando de volta à raiz, o valor final é remontado fazendo $\text{index}(1, 3) = 1 \cdot 4 + 3 = 7$. Logo, o sucessor de 5 é 7. Na segunda busca, o *summary* devolve $\text{index}(1, 1) = 3$, o que indica que o próximo cluster com dados é o cluster 3. Com isso, a raiz pega o valor mínimo desse cluster 3 e reconstrói o número: $\text{index}(3, 2) = 3 \cdot 4 + 2 = 14$. O sucessor de 7 é o 14.

Outras buscas se resolvem de forma muito mais direta. O sucessor de 1, por exemplo, é avaliado rapidamente: como 1 é menor que o $min = 2$ da raiz, a função devolve 2 na mesma hora. Já o sucessor de 15 não existe, e a busca retorna `NIL`, pois o *summary*

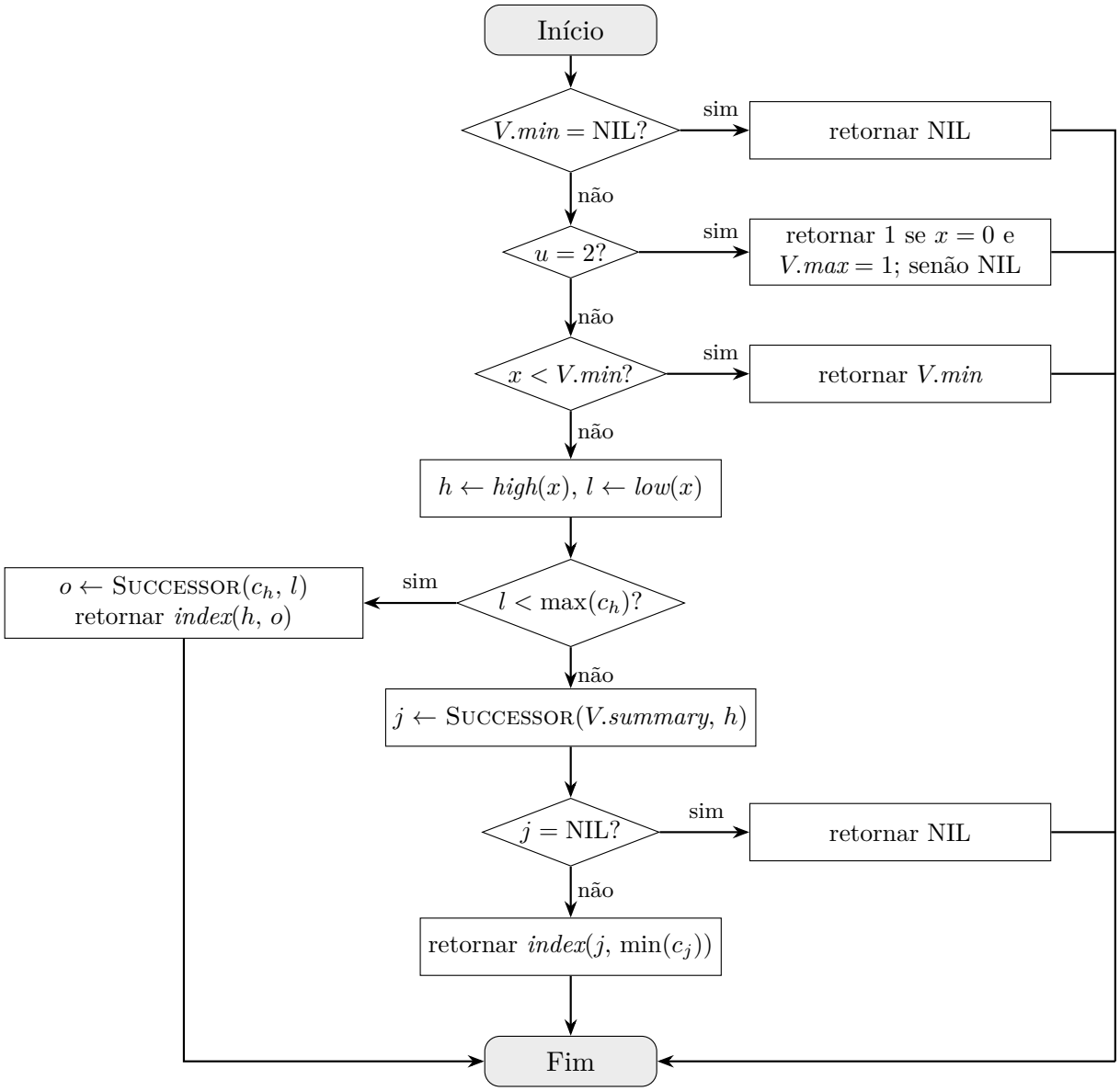


Figura 14 – Fluxograma do sucessor em $vEB(u)$. Usa-se c_h para $V.cluster[h]$. Quando c_h é vazio, $\max(c_h)$ é NIL e a condição de descida é falsa. Retirado e adaptado de (CORMEN et al., 2009).

não tem nenhum cluster ocupado anotado depois do cluster 3.

3.3.4 Predecessor

O predecessor de um número x funciona como o inverso do sucessor: é o maior elemento do conjunto que seja menor que x. A lógica do algoritmo é praticamente um espelho da busca pelo sucessor, com duas diferenças principais. A primeira é um atalho: se o valor de x que você está procurando for maior do que o $V.max$ do nó atual, então a resposta é o próprio máximo (pois nenhum outro número será maior que ele). A segunda diferença é o tratamento do mínimo: se a busca não encontrar nenhum cluster anterior com dados, a resposta ainda pode ser o valor $V.min$ que fica guardado solto no nó (desde

que ele seja menor que x, claro).

O restante da lógica é bem parecido. Se a estrutura estiver vazia, não há predecessor. No caso básico ($u = 2$), o predecessor só existe se estivermos buscando a partir do 1 e o mínimo armazenado for 0. Se o cluster destino ($\text{high}(x)$) tiver algum valor guardado antes da posição local $\text{low}(x)$, a busca desce para ele. Se esse cluster estiver vazio, a verificação falha na hora. Caso contrário, a busca pergunta ao *summary* qual é o último cluster ocupado antes do atual, e a resposta será o valor máximo dentro desse cluster anterior.

O Algoritmo 9 detalha as etapas, e a Figura 15 transforma as decisões em um fluxograma de fácil leitura.

Algoritmo 9: Predecessor de x em uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

```
Entrada: Estrutura V de universo u e chave x, com 0 ≤ x < u
1  if V.min = NIL then
2      return NIL
3  if u = 2 then
4      if x = 1 e V.min = 0 then
5          return 0
6      return NIL
7  if x > V.max then
8      return V.max
9  h ← high(x); l ← low(x)
10 if c_h.min ≠ NIL e l > c_h.min then
11     o ← PREDECESSOR(c_h, l)
12     return index(h, o)
13 j ← PREDECESSOR(V.summary, h)
14 if j ≠ NIL then
15     return index(j, c_j.max)
16 if x > V.min then
17     return V.min
18 return NIL
```

Já mostramos um exemplo do predecessor na Seção 3.2. Para treinar outros cenários, vamos acompanhar na vEB(16) a busca pelo predecessor de 5 e de 3. A Tabela 4 registra os passos, com $\textsc{Predecessor}(V', x)$ abreviado para (V', x) e *summary* para *sum.*

Analisando a primeira busca: a chamada desce para c_1, mas lá dentro o *summary* avisa com um `NIL` que não há clusters ocupados antes da posição atual. Mesmo assim, como o valor procurado ($x = 1$) é maior que o mínimo solto no nó ($V.min = 0$), a função devolve esse 0. Voltando para a raiz, a posição local é convertida usando $\text{index}(1, 0) = 1 \cdot 4 + 0 = 4$. Ou seja, o predecessor de 5 é 4.

Na segunda busca ($x = 3$), o *summary* da raiz também retorna `NIL` porque não

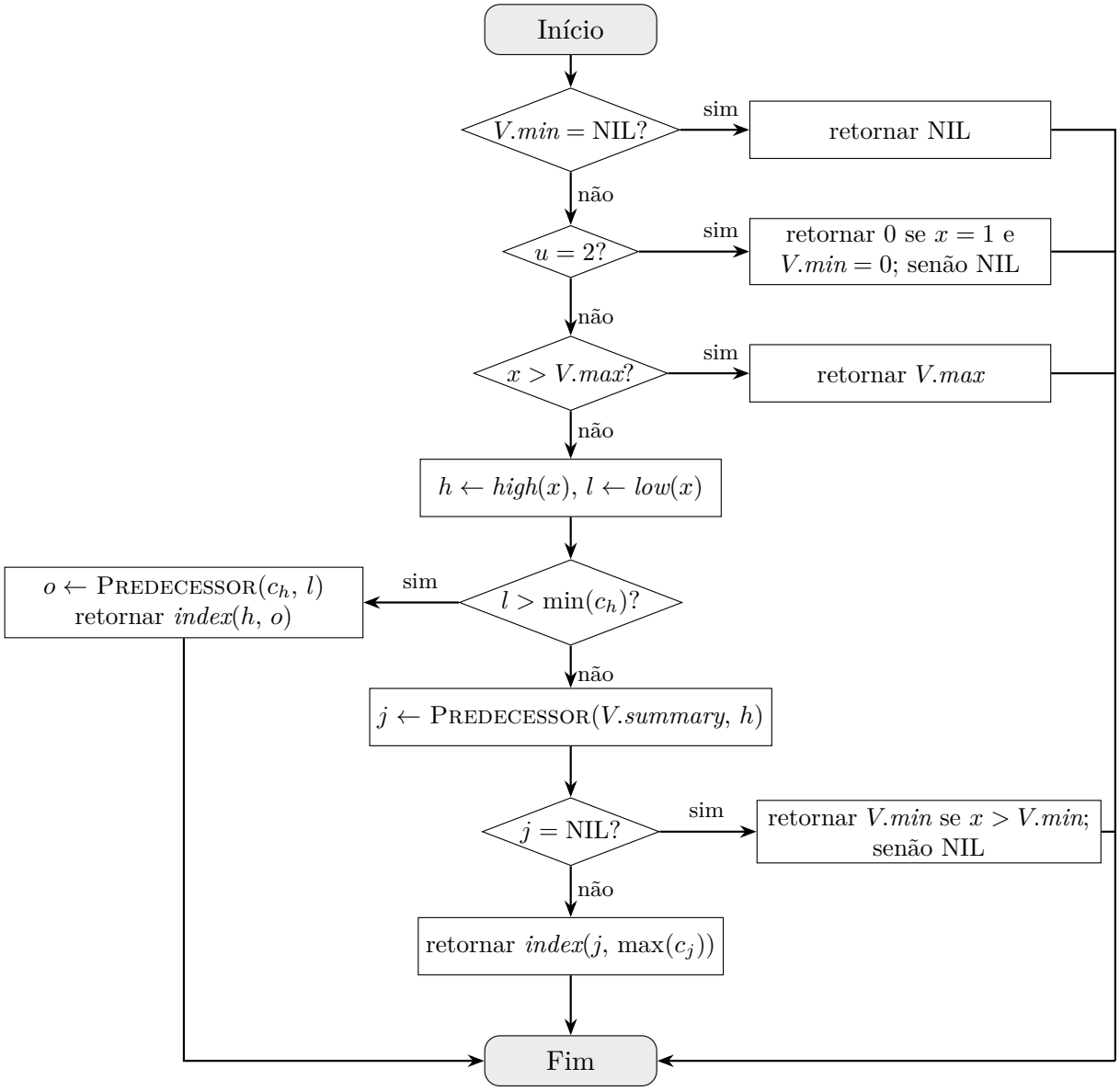


Figura 15 – Fluxograma do predecessor em $vEB(u)$. Usa-se c_h para $V.cluster[h]$. Quando c_h é vazio, $\min(c_h)$ é NIL e a condição de descida é falsa. Retirado e adaptado de (CORMEN et al., 2009).

Tabela 4 – Execução do predecessor sobre a vEB(16) da Figura 12.

(V', x)	u	min	max	Decisão
Predecessor de $x = 5$				
$(V, 5)$	16	2	15	$h = 1$, $l = 1$; $1 > 0$: desce
$(c_1, 1)$	4	0	3	$h = 0$, $l = 1$; $1 \leq 1$: vai ao *sum*
$(c_1.sum, 0)$	2	0	1	$x \neq 1$: NIL
Predecessor de $x = 3$				
$(V, 3)$	16	2	15	$h = 0$, $l = 3$; $3 \leq 3$: vai ao *sum*
$(V.sum, 0)$	4	0	3	$h = 0$, $l = 0$; $0 \leq 1$: vai ao *sum*
$(V.sum.sum, 0)$	2	0	1	$x \neq 1$: NIL

encontra nada antes dele. Mas a raiz checa seu próprio mínimo ($x = 3$ é maior que $V.min = 2$) e devolve o 2. Logo, o predecessor de 3 é 2. Já se tentássemos achar o predecessor de 2, a busca falharia: não há clusters antes ocupados e x não é maior que o mínimo do conjunto, resultando em um `NIL` definitivo.

3.3.5 Inserção

O processo para inserir um novo número x em uma estrutura vEB(u) passa por até três etapas bem definidas. O primeiro caso é o mais simples: se a estrutura inteira estiver vazia, o novo número x será, ao mesmo tempo, o mínimo e o máximo. Ele é salvo nos dois campos e o trabalho termina ali mesmo.

Se a estrutura não estiver vazia, verificamos se o x que queremos inserir é menor que o $V.min$ atual. Se for, os papéis se invertem: o novo x assume o posto de mínimo oficial do nó, e o antigo mínimo é que será empurrado para dentro dos clusters. Esse truque de trocar os valores garante a regra de ouro da vEB: o menor elemento do conjunto sempre fica guardado de fora dos clusters.

O próximo passo acontece se a estrutura for maior que o caso básico ($u > 2$). A chave (que pode ser o x original ou o antigo mínimo que acabou de ser trocado) precisa ser guardada no cluster correspondente, chamado de high(x). Aqui, o algoritmo lida com duas possibilidades:

- **1.** O cluster estava vazio: A nova chave se torna o mínimo e o máximo desse cluster (na posição local low(x)), sem precisar de chamadas recursivas para dentro dele. O único trabalho extra é avisar ao *summary* (por meio de uma recursão) que esse cluster agora está ocupado.
- **2.** O cluster já tinha elementos: A chave local low(x) é inserida recursivamente dentro dele. Nesse caso, não precisamos avisar o *summary*, pois ele já sabia que o cluster estava em uso.

Para finalizar, o campo $V.max$ é atualizado caso a chave inserida seja maior que o máximo atual. Com essa lógica, a inserção garante que só haverá uma única chamada recursiva custosa por nível.

O Algoritmo 10 detalha essa sequência de eventos, e a Figura 16 mostra o caminho no fluxograma (as setas que retornam para a linha central indicam que quase todos os caminhos terminam conferindo e atualizando o máximo).

Para visualizar essa dinâmica, vamos simular a inserção do 1 e do 9 na vEB(16) ilustrada na Figura 12. A Tabela 5 detalha as chamadas INSERT(V', x), mostrando os valores de *min* e *max* antes da inserção. (Abreviamos *summary* para *sum*).

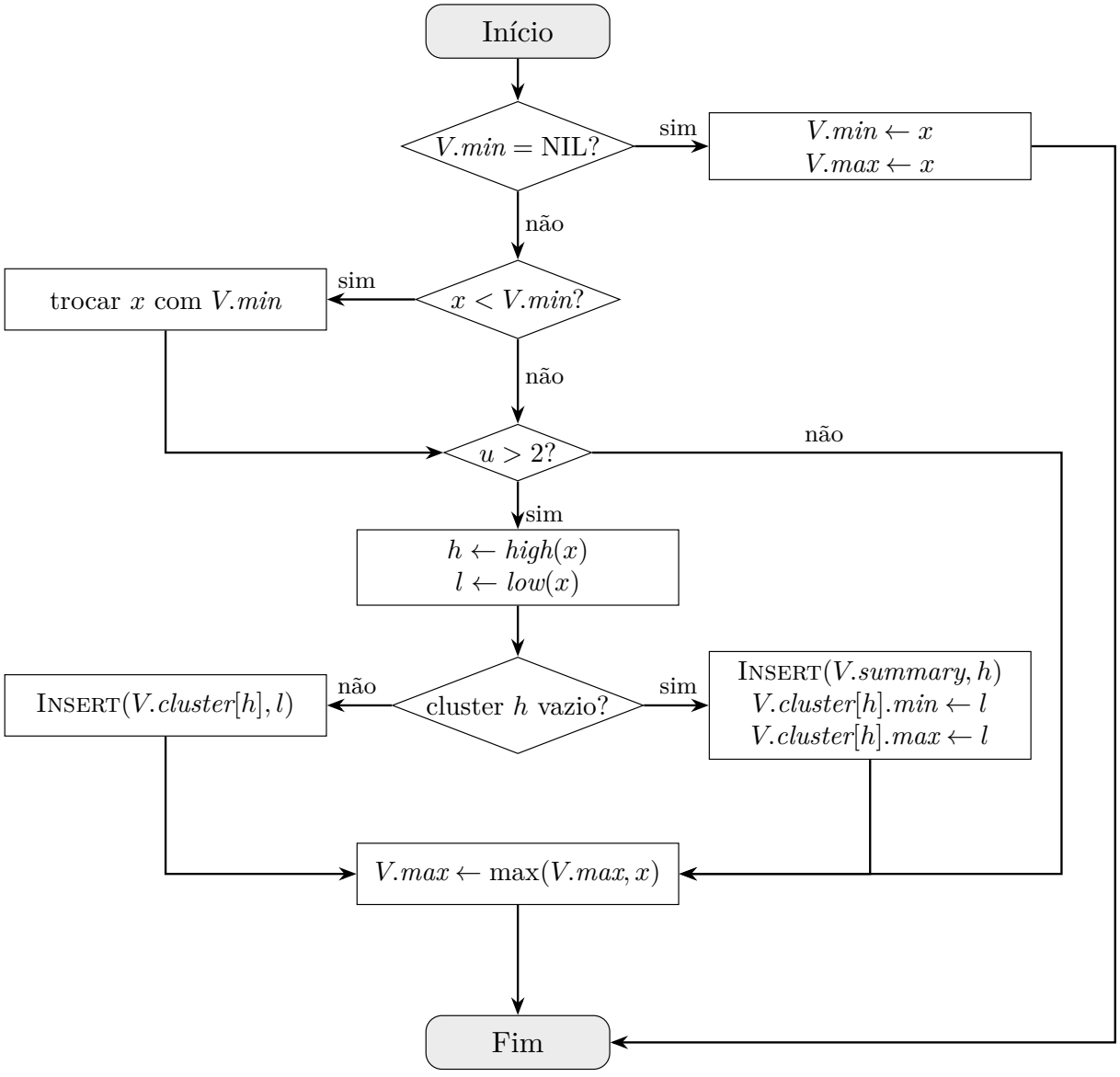


Figura 16 – Fluxograma da inserção em $vEB(u)$. Depois da troca do mínimo, a chave restante gera no máximo uma chamada recursiva: ao *summary*, quando o cluster estava vazio, ou ao próprio cluster, caso contrário. Retirado e adaptado de (CORMEN et al., 2009).

Algoritmo 10: Inserção de x em uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

Entrada: Estrutura V de universo $u = 2^m$ e chave x, com $0 \leq x < u$

```
1  if V.min = NIL then
2      V.min ← x; V.max ← x
3      return
4  if x < V.min then
5      trocar x com V.min
6  if u > 2 then
7      h ← high(x); l ← low(x)
8      if c_h.min = NIL then
9          INSERT(V.summary, h)
10         c_h.min ← l; c_h.max ← l
11     else
12         INSERT(c_h, l)
13 if x > V.max then
14     V.max ← x
```

Tabela 5 – Execução da inserção sobre a vEB(16) da Figura 12.

(V', x)	u	min	max	Decisão
Inserção de $x = 1$				
$(V, 1)$	16	2	15	$1 < 2$: troca ($min \leftarrow 1$, $x \leftarrow 2$); $h = 0$, $l = 2$; desce
$(c_0, 2)$	4	3	3	$2 < 3$: troca ($min \leftarrow 2$, $x \leftarrow 3$); $h = 1$, $l = 1$; c_1 vazio
$(c_0.sum, 1)$	2	NIL	NIL	vazia: $min, max \leftarrow 1$
Inserção de $x = 9$				
$(V, 9)$	16	2	15	$h = 2$, $l = 1$; c_2 vazio: insere 2 no sum; $c_2.min, c_2.max \leftarrow 1$
$(V.sum, 2)$	4	0	3	$h = 1$, $l = 0$; c_1 não vazio; desce
$(V.sum.c_1, 0)$	2	1	1	$0 < 1$: troca; $min \leftarrow 0$

Na primeira inserção, o 1 é menor que o mínimo atual (2), então ele rouba o lugar na raiz. O 2, que foi expulso, desce para o cluster c_0. Lá embaixo, ele é menor que o mínimo local (3) e ocorre uma nova troca. Ao final do processo, o cluster c_0 fica com $min = 2$ e $max = 3$, e o *summary* de c_0 é atualizado para indicar a ocupação.

Na segunda inserção (chave 9), a raiz não altera seu min nem o max, pois o 9 fica no meio do caminho. Ao calcular high(9) = 2 e low(9) = 1, a chave é enviada para o cluster c_2. Como ele estava vazio, os campos min e max de c_2 recebem diretamente o valor 1 (que recomposto dá $2 \cdot 4 + 1 = 9$), e a única chamada recursiva vai para o *summary* da raiz, que passa a registrar que os clusters $\{0, 1, 2, 3\}$ estão ocupados.

3.3.6 Remoção

Remover um elemento x do conjunto traz três desafios novos em comparação com a inserção. Primeiro, lembre-se de que o mínimo fica guardado fora dos clusters. Se

apagarmos justamente ele, precisaremos "promover"o próximo menor valor (buscando-o nos clusters via *summary*) para assumir o posto de novo mínimo. Depois, teremos que apagar esse valor promovido de dentro do cluster onde ele estava. Segundo, se a remoção de um elemento deixar um cluster completamente vazio, precisamos avisar o *summary* para apagar o registro daquele cluster. Terceiro, se apagarmos o valor máximo, o novo máximo geral deverá ser o maior elemento do último cluster que ainda tiver dados (ou, se não sobrar nenhum cluster, o novo máximo será igual ao mínimo restante).

O algoritmo resolve tudo isso numa ordem específica, partindo do princípio de que o elemento x realmente existe. Se a estrutura inteira tiver apenas um elemento, remover x zera tudo (mínimo e máximo viram `NIL`). No caso mais básico ($u = 2$), que guarda apenas as opções 0 ou 1, sobra apenas o valor que não foi apagado ($1 - x$).

Fora desses cenários extremos, o algoritmo checa se x é o mínimo. Se for, ele encontra o novo mínimo, coloca-o no campo $V.min$, e passa a tratar essa nova chave promovida como o elemento que precisa ser removido lá embaixo. Em seguida, a busca desce para o cluster high(x) para apagar a posição local low(x). Se esse cluster secar (ficar vazio), o índice dele é apagado do *summary*. Nesse cenário de cluster vazio, se o valor removido também era o máximo da estrutura, o novo máximo é recalculado pegando o maior valor do último cluster ocupado. Por outro lado, se o cluster *não* secou, mas o elemento removido era o máximo geral, basta atualizar o $V.max$ buscando o maior valor que restou dentro desse mesmo cluster.

O Algoritmo 11 formaliza essas regras, e a Figura 17 organiza essas decisões em um fluxograma.

Para ilustrar o processo na prática, vamos simular a remoção dos números 2 e 15 a partir da vEB(16) mostrada na Figura 12. O primeiro caso forçará a promoção de um novo mínimo e deixará um cluster vazio; o segundo exigirá a atualização do máximo geral. A Tabela 6 registra o passo a passo. Para encurtar, cada chamada DELETE(V', x) está apenas como (V', x), usamos *sum* no lugar de *summary*, e $V'.c_i$ representa o cluster i de V'. Os valores de *min* e *max* mostrados referem-se ao estado do nó antes da operação.

Analisando a remoção do 2: como ele era o mínimo oficial da raiz, o próximo valor, o 3, é promovido para assumir esse lugar, deixando o conjunto principal como $\{3, 4, 5, 7, 14, 15\}$. Ao ir lá no cluster 0 para apagar o 3, descobrimos que o cluster esvaziou totalmente. O índice 0 é, então, removido do *summary*, que passa a apontar apenas para os clusters $\{1, 3\}$ (agora com o mínimo sendo 1). O máximo geral da estrutura permanece igual (15), pois a chave removida foi o mínimo.

Já na remoção do 15: ele não é o mínimo, então a função desce direto para apagá-lo. Ao fazer isso, o pequeno subcluster $c_3.c_1$ seca, e seu índice sai do *summary* local de c_3. Como o 15 era o máximo de c_3 e o *summary* local ficou vazio, o novo máximo de c_3 passa a ser o seu próprio mínimo, que é o 2. Voltando para a raiz principal, o cluster c_3 ainda

Algoritmo 11: Remoção de x de uma $vEB(u)$, supondo $x \in S$. Retirado e adaptado de (CORMEN et al., 2009).

```
   Entrada: Estrutura V de universo u = 2^m e chave x ∈ S
 1 if V.min = V.max then
 2 |   V.min ← NIL; V.max ← NIL
 3 |   return
 4 if u = 2 then
 5 |   V.min ← 1 − x; V.max ← 1 − x
 6 |   return
 7 if x = V.min then
 8 |   j ← V.summary.min
 9 |   x ← index(j, c_j.min); V.min ← x
10 h ← high(x); l ← low(x)
11 DELETE(c_h, l)
12 if c_h.min = NIL then
13 |   DELETE(V.summary, h)
14 |   if x = V.max then
15 |   |   if V.summary.min = NIL then
16 |   |   |   V.max ← V.min
17 |   |   else
18 |   |   |   j' ← V.summary.max
19 |   |   |   V.max ← index(j', c_j'.max)
20 else
21 |   if x = V.max then
22 |   |   V.max ← index(h, c_h.max)
```

Tabela 6 – Execução da remoção sobre a vEB(16) da Figura 12.

(V', x)	u	min	max	Decisão
Remoção de $x = 2$				
$(V, 2)$	16	2	15	$x = min$: $j = 0$, $x \leftarrow 3$, $min \leftarrow 3$; $h = 0$, $l = 3$; desce
$(c_0, 3)$	4	3	3	$min = max$: fica vazio
$(V.sum, 0)$	4	0	3	c_0 vazio: remove 0 do sum; $x = min$: $x \leftarrow 1$, $min \leftarrow 1$; desce
$(V.sum.c_0, 1)$	2	1	1	$min = max$: fica vazio
$(V.sum.sum, 0)$	2	0	1	$u = 2$: $min, max \leftarrow 1$
Remoção de $x = 15$				
$(V, 15)$	16	2	15	$x \neq min$; $h = 3$, $l = 3$; desce
$(c_3, 3)$	4	2	3	$x \neq min$; $h = 1$, $l = 1$; desce
$(c_3.c_1, 1)$	2	1	1	$min = max$: fica vazio
$(c_3.sum, 1)$	2	1	1	$min = max$: fica vazio

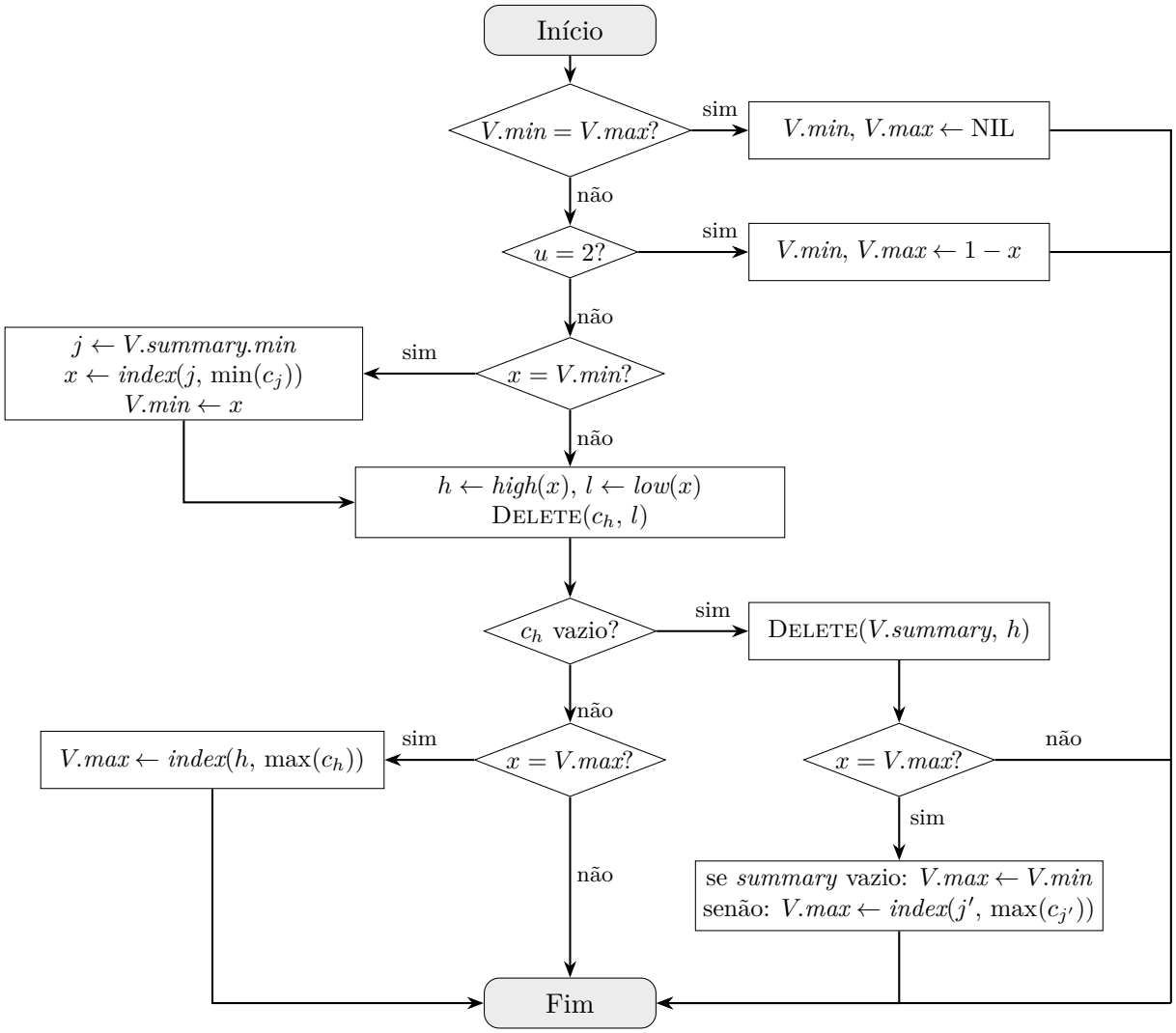


Figura 17 – Fluxograma da remoção de $x \in S$ em $vEB(u)$. Ao remover o mínimo, o novo mínimo é promovido dos clusters e removido deles em seguida. Usa-se c_h para $V.cluster[h]$ e j' para o último cluster ocupado. Retirado e adaptado de (CORMEN et al., 2009).

não está totalmente vazio, mas como o 15 era o máximo de toda a estrutura, precisamos atualizar o $V.max$. A raiz puxa esse novo máximo de c_3 (que é 2) e reconstrói a chave final: $\text{index}(3, 2) = 3 \cdot 4 + 2 = 14$. O conjunto final fica sendo $\{2, 3, 4, 5, 7, 14\}$.

3.3.7 Extração do mínimo

Extrair o valor mínimo é a função mais importante quando usamos a estrutura como uma fila de prioridade. Na prática, a operação não tem mistério: ela lê o valor guardado no campo $V.min$, apaga esse número usando o processo de remoção que acabamos de estudar e, por fim, devolve o valor lido. Se a árvore estiver totalmente vazia, a resposta é apenas `NIL`. Como essa etapa não toma nenhuma decisão nova complexa (é literalmente ler e depois remover), optamos por não incluir um fluxograma aqui.

Voltando à nossa vEB(16) da Figura 12, a primeira vez que chamamos a extração, ela

Algoritmo 12: Extração do mínimo de uma $vEB(u)$. Retirado e adaptado de (CORMEN et al., 2009).

```
   Entrada: Estrutura V
1  if V.min = NIL then
2      return NIL
3  x ← V.min
4  DELETE(V, x)
5  return x
```

nos entrega o 2. Como vimos na Tabela 6, ao remover o 2, o 3 é promovido automaticamente para assumir o posto de novo mínimo da estrutura. Se executarmos uma segunda extração, ela devolve o 3, e logo em seguida o 4 é "puxado"do cluster 1 para preencher a vaga no campo $V.min$ da raiz. A Tabela 7 resume como o conjunto reage a esses passos.

Tabela 7 – Extrações sucessivas do mínimo a partir da vEB(16) da Figura 12.

Chamada	Retorno	Conjunto depois
EXTRACTMIN(V)	2	$\{3, 4, 5, 7, 14, 15\}$
EXTRACTMIN(V)	3	$\{4, 5, 7, 14, 15\}$

3.4 Análise Assintótica

O grande diferencial de desempenho da árvore vEB é que o custo de suas operações depende estritamente do tamanho do universo de chaves (u), e não da quantidade de elementos (n) armazenados. Isso garante previsibilidade assintótica mesmo para conjuntos muito esparsos. Nesta seção, avaliamos o tempo de execução e o consumo de memória no pior cenário.

3.4.1 Modelo de Custo e Recorrência

Consideramos como custo o número de passos elementares (leitura, escrita, comparações e operações aritméticas). Assumindo que $u = 2^m$ e que as chaves cabem em uma palavra de memória do computador, as funções que decomputam e reconstroem as chaves — high(x), low(x) e index(j, l) — executam apenas deslocamentos e máscaras de bits, operando em tempo constante $O(1)$.

A cada nível da árvore de universo u, o problema é dividido em subestruturas de tamanho aproximado $\sqrt{u}$. Em termos de bits ($m = \log_2 u$), isso equivale a cortar o número de bits pela metade a cada etapa ($\lceil m/2 \rceil$). Como cada nível executa um número fixo de operações c e faz no máximo uma chamada recursiva custosa, a função de tempo $T(m)$

obedece à relação:

$$T(m) \leq T\left(\left\lceil \frac{m}{2} \right\rceil\right) + c \quad (m \geq 2), \tag{3.1}$$
$$T(1) \leq c.$$

Essa redução sucessiva do universo de u para $\sqrt{u}$ atinge o caso-base $u = 2$ em $O(\log m)$ passos (CORMEN et al., 2009). Como $m = \log_2 u$, a solução direta dessa relação estabelece que as operações recursivas possuem complexidade de tempo de $O(\log \log u)$.

3.4.2 Tempo das Operações

As operações da árvore vEB dividem-se em três padrões de comportamento recursivo:

1. **Acesso Direto ($\Theta(1)$):** As consultas MINIMUM e MAXIMUM leem diretamente os campos $V.min$ e $V.max$ do nó raiz, operando de forma instantânea sem laços ou recursão.

2. **Caminho Único ($O(\log \log u)$):** As operações MEMBER, SUCCESSOR, PREDECESSOR e INSERT realizam no máximo uma chamada recursiva por nível. Na inserção, a estratégia de manter o elemento mínimo solto fora dos clusters é fundamental: ao inserir em um cluster vazio, ele é ocupado em $O(1)$ e a única chamada recursiva vai para o *summary*; se o cluster já possui elementos, a recursão entra no cluster e o *summary* nem é alterado.

3. **Remoção e Operações Compostas ($O(\log \log u)$):** Embora a remoção (DELETE) apresente chamadas ao cluster e ao *summary*, elas nunca são custosas ao mesmo tempo. Se o cluster esvazia, a remoção local levou tempo constante; se continua ocupado, o *summary* não é consultado. As operações compostas EXTRACTMIN (remoção precedida de leitura) e DECREASEKEY (remoção seguida de inserção) mantêm essa mesma classe de custo.

Essa complexidade de $O(\log \log u)$ é justa no pior caso ($\Theta(\log \log u)$), pois existem sequências de chaves que forçam o algoritmo a percorrer a árvore até o caso-base $u = 2$. A Tabela 8 sintetiza esses custos.

Tabela 8 – Custo assintótico das operações sobre a vEB(u).

Operação	Custo (Pior Caso)
MINIMUM, MAXIMUM	$\Theta(1)$
MEMBER, SUCCESSOR, PREDECESSOR	$\Theta(\log \log u)$
INSERT, DELETE	$\Theta(\log \log u)$
EXTRACTMIN, DECREASEKEY	$\Theta(\log \log u)$

3.4.3 Análise de Espaço e Aplicações Práticas

Na representação densa, todas as subestruturas são pré-alocadas durante a criação da árvore. Um nó de universo u guarda seus campos próprios, um vetor de $\sqrt{u}$ ponteiros de clusters e suas subestruturas filhas, gerando a recorrência:

$$P(u) = P(\sqrt{u}) + \sqrt{u} \cdot P(\sqrt{u}) + O(\sqrt{u})$$

A resolução dessa relação demonstra que o consumo de memória é de $\Theta(u)$ palavras de máquina.

Otimizações e Uso Recomendado: O consumo de espaço $\Theta(u)$ independe da quantidade de elementos n inseridos. Quando a estrutura guarda poucos elementos ($n \ll u$), essa alocação prévia pode ser ineficiente. Uma alternativa é a representação esparsa, que aloca clusters dinamicamente em tabelas hash à medida que são ocupados. Isso reduz a memória para $O(n \log \log u)$, embora troque as garantias determinísticas de pior caso por tempos esperados. Em suma, a versão densa original compensa bastante quando o universo de chaves é proporcional ao volume de dados ($u = O(n)$).

4 Metodologia Experimental

Este capítulo descreve o ambiente computacional, as escolhas de implementação e os protocolos experimentais empregados para avaliar o heap de Fibonacci e a árvore de van Emde Boas frente a estruturas alternativas. Os ensaios foram organizados em dois estudos: a comparação direta de operações de filas de prioridade e a avaliação de árvores balanceadas e filas de prioridade no algoritmo de Dijkstra. A análise considera os tempos e a memória observados; efeitos de cache, previsibilidade de desvios e custos de alocação são hipóteses de interpretação, não efeitos isolados pelos ensaios.

4.1 Ambiente Computacional e Protocolo de Medição

Todas as estruturas avaliadas foram desenvolvidas do zero em C++20 e compiladas com `g++ 13.3.0`, usando `-O3 -std=c++20 -march=native -pipe`. Os ensaios de tempo e perfilamento de memória foram executados em Ubuntu Linux 24.04 LTS, kernel 6.8.0, em uma estação com processador Intel Core i7-13620H, 10 núcleos físicos e 16 threads, e 24 GB de memória DDR5 a 5200 MT/s. O processador tem frequência turbo máxima informada de 4,90 GHz, cache L1D de 48 KB e L2 de 1,25 MB por núcleo de desempenho, além de cache L3 compartilhado de 24 MB. O ambiente foi configurado em modo monousuário, com o governador de frequência no perfil *performance* e os estados de economia de energia desativados.

O tempo foi medido com `std::chrono::high_resolution_clock`. Como o padrão C++ não garante que esse relógio seja monotônico em todas as implementações, essa propriedade depende da biblioteca usada no ambiente de execução. Os resultados das consultas foram acumulados em variáveis auxiliares `volatile` para reduzir o risco de eliminação de código morto durante a otimização; essa técnica, por si só, não substitui a verificação do trecho efetivamente cronometrado e do código gerado. Antes das medições foi executada uma rodada de aquecimento com cargas reduzidas, com o objetivo de inicializar páginas de memória e linhas de cache. Cada cenário foi repetido dez vezes de forma independente; entre as repetições, as estruturas foram reconstruídas e a memória dinâmica foi liberada. Reportam-se a média amostral (μ) e o desvio padrão (σ).

A medição de memória foi conduzida separadamente da cronometragem, com Valgrind Massif e as opções `-tool=massif -pages-as-heap=yes -detailed-freq=1`. O perfilamento registra o pico de memória no escopo definido por essa configuração, mas não separa automaticamente as alocações da estrutura daquelas destinadas à entrada ou a outros componentes do processo. Por isso, os valores de Massif são apresentados separadamente das medidas de memória do benchmark e não devem ser interpretados

como consumo exclusivo da estrutura sem essa decomposição.

4.2 Estudo 1: Comparação Direta das Filas de Prioridade

O primeiro estudo compara o heap de Fibonacci e a árvore de van Emde Boas (vEB) em operações de fila de prioridades: inserção, consulta ao mínimo, extração do mínimo e remoção arbitrária. A Tabela 9 apresenta o mapeamento funcional usado na comparação.

Tabela 9 – Mapeamento funcional das operações equivalentes entre o heap de Fibonacci e a árvore vEB.

Operação Avaliada	**Heap de Fibonacci**	**Árvore vEB**
Inserção de nova chave	`insert(x)`	`insert(x)`
Consulta ao menor elemento	`findMin()`	`minimum()`
Extração do menor elemento	`extractMin()`	`delete(minimum())`
Remoção de chave arbitrária	`delete(x)`	`delete(x)`

A consulta de vazio, `isEmpty()`, é registrada como verificação auxiliar, mas não integra as quatro operações principais. Na remoção de um elemento específico, o heap de Fibonacci recebe uma referência ao nó e combina diminuição de chave com extração do mínimo; a vEB recebe a chave e a localiza durante a exclusão. Quando aplicável, distinguem-se o custo da operação estrutural e o custo de localizar ou obter a referência ao elemento.

Para avaliar a sensibilidade ao padrão de entrada, foram gerados três arquivos com 1.000.000 de chaves inteiras distintas no intervalo $[0, U-1]$: `cresce`, em ordem estritamente crescente; `decresce`, em ordem estritamente decrescente; e `aleatorio`, em ordem pseudoaleatória produzida por `std::mt19937_64`. As sementes são mantidas fixas. Não está documentado se os arquivos crescente, decrescente e aleatório são permutações do mesmo conjunto; portanto, diferenças entre eles não isolam necessariamente o efeito da ordem de inserção. Para a vEB, o universo foi fixado em $U = 2^{20} = 1.048.576$. As cargas variam n em $\{10^2, 10^3, 10^4, 10^5, 10^6\}$, correspondendo a razões de ocupação $\rho = n/U$ de aproximadamente $9{,}5 \times 10^{-5}$ a 0,954.

Para cada tamanho e padrão, o script seleciona as primeiras n linhas do arquivo correspondente; assim, cada tamanho constitui um prefixo da carga. Dentro de cada cenário, a mesma sequência é fornecida às duas estruturas. A consulta de pertinência, sucessor, predecessor e máximo é avaliada na vEB como operações adicionais; diminuição de chave e união são avaliadas no heap de Fibonacci. Essas operações não são tratadas como comparações diretas quando não há operação equivalente na outra estrutura.

A descrição disponível não especifica, para cada métrica, a quantidade e a sequência de chamadas cronometradas, os limites do intervalo nem se a construção da estrutura está

incluída. Assim, os tempos reportados não podem ser interpretados como latência por chamada sem recuperar e documentar o protocolo exato de cada medição.

4.3 Estudo 2: Comparação com Estruturas Alternativas de Referência

O segundo estudo contém dois cenários. No primeiro, a vEB é comparada com árvores AVL e rubro-negras na manutenção de conjuntos ordenados, mantendo-se $U = 2^{20}$ para a vEB. A AVL mantém o balanceamento de altura com fatores $\Delta h \in \{-1, 0, +1\}$; a implementação rubro-negra utiliza coloração, rotações e o sentinela `T_NIL`. As três estruturas recebem as mesmas sequências para cada fatia e são avaliadas em inserção, busca por pertinência, consulta de mínimo e máximo, sucessor, predecessor e remoção. Também se mede a memória. Para interpretar os resultados da vEB, são considerados n, U e ρ, uma vez que a representação depende do universo além da quantidade de chaves.

No segundo cenário, o heap de Fibonacci é usado como fila de prioridades no algoritmo de Dijkstra e comparado com heaps binários ($d = 2$) e d-ários ($d \in \{4, 8\}$) implementados em vetores contíguos. Nas filas vetoriais, um vetor de indexação direta associa cada vértice à sua posição no heap e permite localizar o elemento para a operação de diminuição de chave. Os grafos são direcionados, têm pesos não negativos e são gerados com a semente fixa `seed=42`. Avaliam-se um grafo esparso com $|V| = 100.000$ e $|E| = 300.000$ e um grafo denso com $|V| = 5.000$ e $|E| = 4.999.000$. Para cada perfil, comparam-se o tempo total de Dijkstra e o pico de memória reportado. A carga densa foi incluída para exercitar cenários com muitas relaxações; contudo, sem contadores de hardware, os resultados não isolam a contribuição da localidade de cache nem permitem atribuir causalmente diferenças à estrutura da fila.

As variantes são executadas sobre os mesmos grafos e usam a mesma representação, origem e procedimentos auxiliares. Para as chaves pseudoaleatórias do Estudo 1, os valores das sementes não estão informados. Em Dijkstra, embora `seed=42` esteja registrada, a regra de geração, a distribuição e a faixa dos pesos e o identificador da origem não estão especificados no material disponível; esses itens, assim como os arquivos e registros brutos, são necessários para a reprodução independente. Cada combinação de estrutura, tamanho e distribuição segue dez repetições, com apresentação da média e do desvio padrão.

5 Resultados

Este capítulo apresenta os resultados na ordem dos estudos descritos no Capítulo 4. As tabelas de tempo reportam a média e o desvio padrão de dez execuções; as figuras complementam a comparação entre tamanhos e padrões de entrada. Os dados de memória são apresentados separadamente dos tempos, pois o perfilamento com Valgrind Massif não foi usado durante a cronometragem. Para quantificar as diferenças, a redução percentual é calculada em relação à maior média, enquanto o aumento percentual da estrutura mais lenta é calculado em relação à menor média. Os percentuais são derivados das médias tabuladas, arredondados a duas casas decimais e descrevem somente as cargas medidas; não demonstram, por si só, causalidade nem significância estatística.

5.1 Comparação isolada das filas de prioridade

5.1.1 Tempos das operações equivalentes

A Tabela 10 reúne os tempos reportados para as quatro operações, organizados por estrutura, padrão de entrada e cardinalidade. Os valores são médias ± desvio padrão em microssegundos (μs), apresentados em notação científica para $n \in \{10^2, 10^3, 10^4, 10^5, 10^6\}$. O universo da vEB foi fixado em $U = 2^{20} = 1.048.576$. Como os registros não esclarecem quantas chamadas compõem cada medição nem sua normalização, esses valores não devem ser lidos como latências unitárias ou usados para inferir a complexidade assintótica a partir da inclinação dos gráficos.

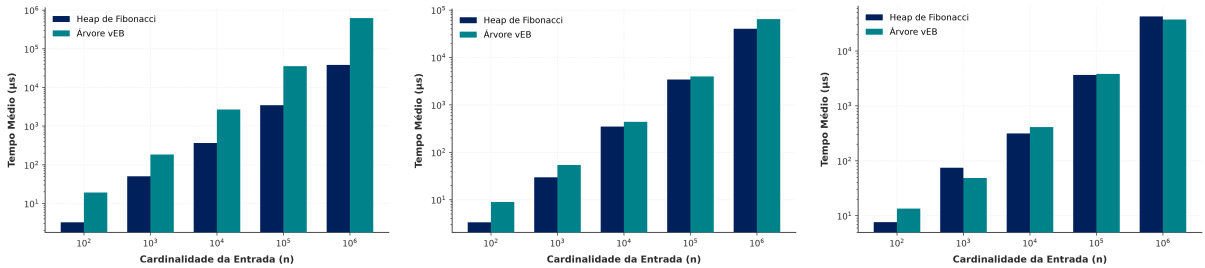
As Figuras 18 a 21 mostram que o resultado depende da operação, do padrão de entrada e de n. Para $n = 10^6$ e entrada aleatória, o heap de Fibonacci levou 38,06 ms na inserção, contra 623,35 ms para a vEB, uma redução de 93,89% em relação à média da vEB. Com entrada crescente, os tempos foram 40,20 ms e 64,17 ms, respectivamente, redução de 37,36% para o heap de Fibonacci; com entrada decrescente, a vEB levou 37,21 ms contra 42,42 ms do heap, redução de 12,30%. Portanto, não há uma única ordenação de desempenho que valha para os três padrões.

Na consulta do mínimo (Figura 19), os tempos reportados variam conforme a cardinalidade e o padrão da entrada. Para $n = 10^6$ e padrão aleatório, foram registrados 2.409,4 μs para o heap de Fibonacci e 3.535,0 μs para a vEB, redução de 31,84% para o heap. Com entrada crescente, a redução foi de 7,36% (2.412,2 contra 2.603,9 μs); com entrada decrescente, a vEB registrou redução de 42,58% em relação ao heap (2.375,8 contra 4.137,6 μs). Essas diferenças devem ser interpretadas junto aos desvios padrão e às demais cargas; uma medição pontual não basta para inferir uma complexidade assintótica.

Tabela 10 – Desempenho comparativo condensado em notação científica (μs) entre Heap de Fibonacci e Árvore vEB.

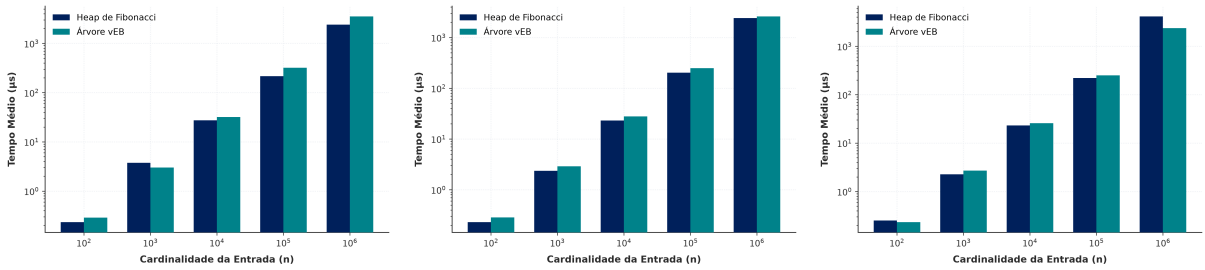
Padrão	n	Inserção	Mínimo	Extrair Mín	Remoção
Heap de Fibonacci					
Aleatório	10^2	$(3,25 \pm 0,46) \times 10^0$	$(2,33 \pm 0,21) \times 10^{-1}$	$(5,22 \pm 0,46) \times 10^1$	$(2,89 \pm 0,32) \times 10^1$
	10^3	$(5,01 \pm 1,24) \times 10^1$	$(3,75 \pm 0,85) \times 10^0$	$(1,12 \pm 0,22) \times 10^3$	$(5,82 \pm 1,39) \times 10^2$
	10^4	$(3,66 \pm 0,29) \times 10^2$	$(2,73 \pm 0,39) \times 10^1$	$(1,03 \pm 0,02) \times 10^4$	$(4,81 \pm 0,30) \times 10^3$
	10^5	$(3,44 \pm 0,43) \times 10^3$	$(2,17 \pm 0,32) \times 10^2$	$(1,06 \pm 0,06) \times 10^5$	$(4,09 \pm 0,14) \times 10^4$
	10^6	$(3,81 \pm 0,47) \times 10^4$	$(2,41 \pm 0,31) \times 10^3$	$(2,15 \pm 0,33) \times 10^6$	$(5,98 \pm 2,36) \times 10^5$
Crescente	10^2	$(3,32 \pm 0,24) \times 10^0$	$(2,29 \pm 0,23) \times 10^{-1}$	$(4,42 \pm 0,16) \times 10^1$	$(4,51 \pm 0,21) \times 10^1$
	10^3	$(2,96 \pm 0,17) \times 10^1$	$(2,35 \pm 0,15) \times 10^0$	$(5,38 \pm 0,30) \times 10^2$	$(5,49 \pm 0,36) \times 10^2$
	10^4	$(3,48 \pm 0,75) \times 10^2$	$(2,31 \pm 0,45) \times 10^1$	$(6,90 \pm 1,52) \times 10^3$	$(6,69 \pm 1,17) \times 10^3$
	10^5	$(3,41 \pm 0,13) \times 10^3$	$(2,03 \pm 0,17) \times 10^2$	$(7,24 \pm 1,41) \times 10^4$	$(7,00 \pm 0,60) \times 10^4$
	10^6	$(4,02 \pm 0,90) \times 10^4$	$(2,41 \pm 0,44) \times 10^3$	$(7,11 \pm 0,45) \times 10^5$	$(7,22 \pm 0,31) \times 10^5$
Decrescente	10^2	$(7,58 \pm 0,18) \times 10^0$	$(2,52 \pm 0,52) \times 10^{-1}$	$(1,03 \pm 0,02) \times 10^2$	$(5,34 \pm 0,11) \times 10^1$
	10^3	$(7,42 \pm 0,56) \times 10^1$	$(2,27 \pm 0,19) \times 10^0$	$(1,26 \pm 0,08) \times 10^3$	$(6,59 \pm 1,50) \times 10^2$
	10^4	$(3,14 \pm 0,64) \times 10^2$	$(2,31 \pm 0,57) \times 10^1$	$(6,05 \pm 0,77) \times 10^3$	$(3,05 \pm 0,67) \times 10^3$
	10^5	$(3,64 \pm 0,50) \times 10^3$	$(2,22 \pm 0,41) \times 10^2$	$(7,08 \pm 0,96) \times 10^4$	$(3,42 \pm 0,49) \times 10^4$
	10^6	$(4,24 \pm 0,81) \times 10^4$	$(4,14 \pm 5,28) \times 10^3$	$(8,32 \pm 1,05) \times 10^5$	$(5,09 \pm 2,72) \times 10^5$
Árvore de van Emde Boas					
Aleatório	10^2	$(1,91 \pm 0,31) \times 10^1$	$(2,90 \pm 0,73) \times 10^{-1}$	$(1,15 \pm 1,14) \times 10^1$	$(7,85 \pm 1,11) \times 10^0$
	10^3	$(1,83 \pm 0,09) \times 10^2$	$(3,00 \pm 0,37) \times 10^0$	$(9,50 \pm 1,25) \times 10^1$	$(9,31 \pm 0,43) \times 10^1$
	10^4	$(2,66 \pm 0,37) \times 10^3$	$(3,20 \pm 0,73) \times 10^1$	$(2,03 \pm 0,33) \times 10^3$	$(2,68 \pm 0,37) \times 10^3$
	10^5	$(3,53 \pm 0,17) \times 10^4$	$(3,20 \pm 0,85) \times 10^2$	$(1,82 \pm 0,25) \times 10^4$	$(5,74 \pm 0,79) \times 10^4$
	10^6	$(6,23 \pm 0,67) \times 10^5$	$(3,53 \pm 0,77) \times 10^3$	$(6,43 \pm 1,38) \times 10^4$	$(7,68 \pm 0,70) \times 10^5$
Crescente	10^2	$(8,93 \pm 1,85) \times 10^0$	$(2,86 \pm 0,90) \times 10^{-1}$	$(6,01 \pm 0,94) \times 10^0$	$(6,71 \pm 2,84) \times 10^0$
	10^3	$(5,38 \pm 1,55) \times 10^1$	$(2,89 \pm 0,53) \times 10^0$	$(5,11 \pm 1,62) \times 10^1$	$(5,29 \pm 2,27) \times 10^1$
	10^4	$(4,37 \pm 1,21) \times 10^2$	$(2,79 \pm 0,88) \times 10^1$	$(4,17 \pm 0,19) \times 10^2$	$(4,72 \pm 1,44) \times 10^2$
	10^5	$(3,98 \pm 0,42) \times 10^3$	$(2,48 \pm 0,28) \times 10^2$	$(6,43 \pm 1,37) \times 10^3$	$(5,66 \pm 0,77) \times 10^3$
	10^6	$(6,42 \pm 3,36) \times 10^4$	$(2,60 \pm 0,46) \times 10^3$	$(1,14 \pm 0,48) \times 10^5$	$(8,94 \pm 4,36) \times 10^4$
Decrescente	10^2	$(1,34 \pm 0,14) \times 10^1$	$(2,32 \pm 0,15) \times 10^{-1}$	$(9,65 \pm 0,78) \times 10^0$	$(6,78 \pm 0,43) \times 10^0$
	10^3	$(4,83 \pm 0,95) \times 10^1$	$(2,71 \pm 1,02) \times 10^0$	$(4,99 \pm 1,79) \times 10^1$	$(3,43 \pm 1,13) \times 10^1$
	10^4	$(4,10 \pm 0,96) \times 10^2$	$(2,59 \pm 0,66) \times 10^1$	$(4,55 \pm 0,49) \times 10^2$	$(3,37 \pm 1,03) \times 10^2$
	10^5	$(3,79 \pm 0,44) \times 10^3$	$(2,50 \pm 0,51) \times 10^2$	$(5,22 \pm 0,87) \times 10^3$	$(3,90 \pm 0,47) \times 10^3$
	10^6	$(3,72 \pm 0,15) \times 10^4$	$(2,38 \pm 0,30) \times 10^3$	$(5,32 \pm 0,32) \times 10^4$	$(3,94 \pm 0,29) \times 10^4$

Figura 18 – Tempo médio de inserção, em μs, no heap de Fibonacci e na árvore vEB em função de n. Cada painel corresponde a um padrão de entrada, identificado abaixo do gráfico.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

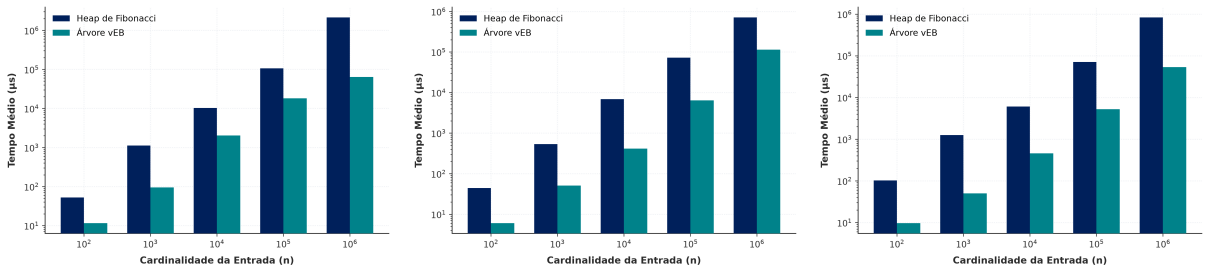
Figura 19 – Tempo médio de consulta do mínimo, em μs, no heap de Fibonacci e na árvore vEB em função de n. Cada painel corresponde a um padrão de entrada, identificado abaixo do gráfico.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

Na extração do mínimo (Figura 20), para $n = 10^6$ e padrão aleatório, a média do heap de Fibonacci foi 2.145,75 ms e a da vEB foi 64,33 ms, redução de 97,00% para a vEB. Com entradas crescente e decrescente, as reduções da vEB foram, respectivamente, 83,98% (113,95 contra 711,44 ms) e 93,60% (53,24 contra 832,00 ms). As diferenças são expressivas nesses cenários específicos. São compatíveis com o fato de a extração no heap de Fibonacci ter custo amortizado $O(\log n)$ e na vEB depender de U; os tempos, porém, também refletem a implementação e a carga medidas, e não identificam isoladamente a causa do resultado.

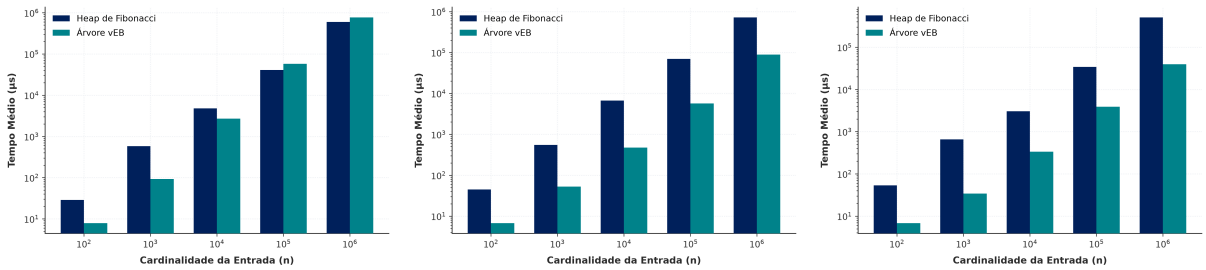
Figura 20 – Tempo médio de extração do mínimo, em μs, no heap de Fibonacci e na árvore vEB em função de n. Cada painel corresponde a um padrão de entrada, identificado abaixo do gráfico.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

A remoção de elemento específico, apresentada na Figura 21, requer cautela adicional. No heap de Fibonacci, a remoção depende de uma referência ao nó e é implementada por diminuição de chave seguida de extração do mínimo; na vEB, a operação recebe a chave. Como as interfaces e os custos para localizar o elemento não são iguais, esses tempos descrevem as implementações avaliadas, mas não constituem uma comparação estritamente equivalente de ponta a ponta. Para $n = 10^6$ e padrão aleatório, foram registrados 597,89 ms no heap e 768,37 ms na vEB, redução de 22,19% para o heap. Nos padrões crescente e decrescente, a vEB apresentou médias 87,61% e 92,27% menores que as do heap, respectivamente; esses percentuais também comparam as interfaces implementadas e não eliminam a ressalva de equivalência.

Figura 21 – Tempo médio de remoção de elemento específico nas estruturas avaliadas, em μs, em função de n. Cada painel corresponde a um padrão de entrada; as implementações recebem identificadores diferentes conforme a interface descrita no texto.



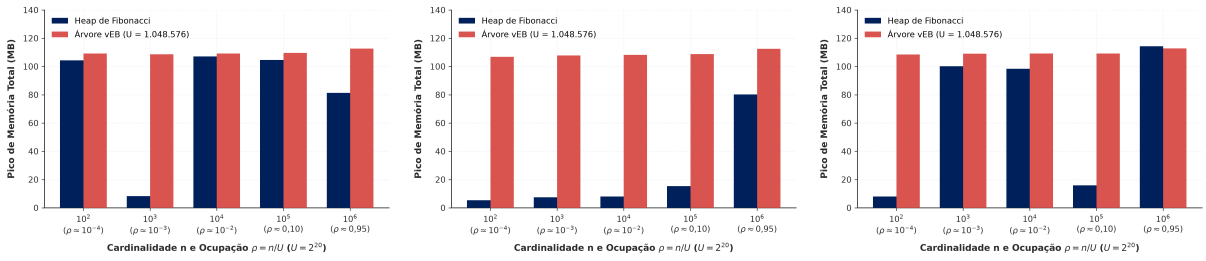
(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

A verificação `isEmpty()` foi definida como métrica auxiliar e não integra as quatro operações principais. O material experimental registra um valor inferior a 1 ns para essa métrica; sem a série de medições e o procedimento de agregação correspondentes, não é possível confirmar que represente a latência de uma chamada isolada nem sustentar precisão subnanosegundo.

5.1.2 Consumo de memória e influência do universo

A Figura 22 mostra os picos de memória para $U = 1.048.576$ e n variando de 10^2 a 10^6. As razões de ocupação correspondentes são, aproximadamente, $9{,}54 \times 10^{-5}$, $9{,}54 \times 10^{-4}$, $9{,}54 \times 10^{-3}$, 0,0954 e 0,9537. Esse parâmetro é relevante porque o espaço da vEB depende do universo, enquanto a memória do heap de Fibonacci cresce com os elementos armazenados.

Figura 22 – Pico de memória, em MB, do heap de Fibonacci e da árvore vEB em função de n, com universo fixo $U = 1.048.576$. Os painéis distinguem a ordem de entrada e a ocupação é dada por $\rho = n/U$.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

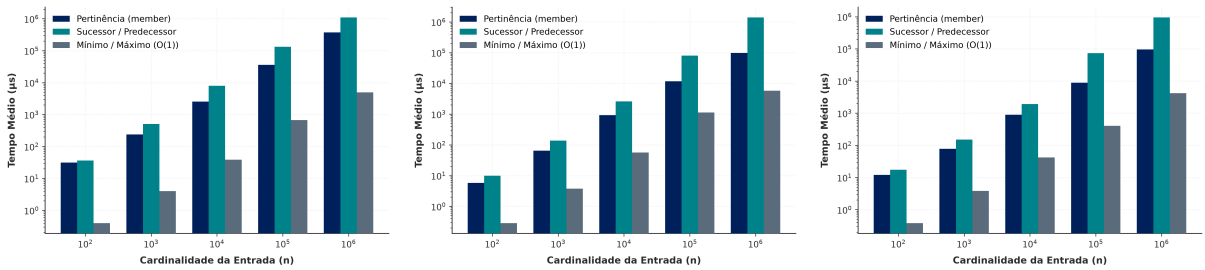
Nos dados reportados, a vEB permanece na faixa aproximada de 106,8 a 109,3 MB nas menores cargas e chega a cerca de 112,7 MB em $n = 10^6$. O heap de Fibonacci parte de aproximadamente 5,5 a 8,3 MB nas cargas pequenas e alcança valores entre 80 e 114 MB na maior carga. Assim, para o universo fixado, a vEB tem maior custo inicial quando

ρ é baixo; na maior cardinalidade, as faixas se aproximam. Como esses intervalos são aproximados e não apresentam os pares por cardinalidade e padrão, não é possível calcular uma porcentagem única de diferença sem pressupor correspondências não informadas. Esse resultado também não deve ser generalizado para outros valores de U, que não foram variados nesta medição.

5.1.3 Operações específicas de cada estrutura

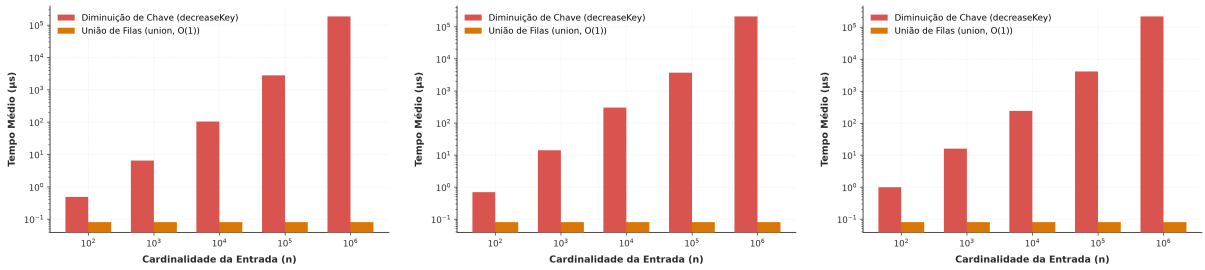
Busca por pertinência, sucessor, predecessor e máximo da vEB são exibidos separadamente das operações específicas do heap de Fibonacci. Para a vEB, o material reporta, em $n = 10^6$, médias aproximadas de 0,37 μs para pertinência e 1,10 μs para sucessor/predecessor; a consulta de máximo é descrita como inferior a 0,05 μs. Para o heap, são apresentados valores reportados de 0,04 a 0,20 μs para diminuição de chave em grandes volumes e de cerca de 0,08 μs para união. Como o protocolo e a normalização desses valores não foram recuperados, eles não devem ser interpretados como latências por chamada. Além disso, como as operações não são equivalentes entre as estruturas, não se estabelece uma classificação cruzada. A Figura 23 reúne as operações específicas da vEB; a Figura 24 apresenta as medições de diminuição de chave e união no heap de Fibonacci.

Figura 23 – Tempos das operações específicas da árvore vEB, em μs: busca por pertinência, sucessor/predecessor e consulta do máximo, em função de n. Cada painel indica o padrão de entrada.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

Figura 24 – Tempos das operações específicas do heap de Fibonacci, em μs: diminuição de chave e união, em função de n. Cada painel indica o padrão de entrada.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

5.2 Comparação com estruturas alternativas

5.2.1 Árvore vEB, AVL e rubro-negra

As Figuras 25 a 29 e as Tabelas 11 e 12 apresentam os tempos de inserção, pertinência, mínimo/máximo, sucessor/predecessor e remoção para AVL, rubro-negra e vEB. Foram reutilizados os padrões de entrada e as cardinalidades do primeiro estudo. Para a vEB, $U = 1.048.576$; logo, os valores de ρ informados na análise anterior se aplicam.

Tabela 11 – Tempos de inserção, busca e consulta de mínimo/máximo para AVL, rubro-negra e vEB (média ± desvio padrão, em μs).

Padrão	n	**AVL**	**RN**	**vEB**
Inserção				
Aleatório	10^2	$(1{,}70 \pm 1{,}81) \times 10^1$	$(1{,}44 \pm 0{,}23) \times 10^1$	$(3{,}40 \pm 0{,}73) \times 10^1$
	10^3	$(2{,}11 \pm 0{,}12) \times 10^2$	$(1{,}86 \pm 0{,}20) \times 10^2$	$(4{,}34 \pm 4{,}87) \times 10^2$
	10^4	$(3{,}27 \pm 0{,}29) \times 10^3$	$(3{,}14 \pm 0{,}50) \times 10^3$	$(4{,}91 \pm 2{,}39) \times 10^3$
	10^5	$(8{,}10 \pm 0{,}94) \times 10^4$	$(7{,}62 \pm 0{,}96) \times 10^4$	$(6{,}84 \pm 3{,}06) \times 10^4$
	10^6	$(2{,}38 \pm 0{,}40) \times 10^6$	$(1{,}82 \pm 0{,}22) \times 10^6$	$(7{,}93 \pm 1{,}60) \times 10^5$
Crescente	10^2	$(9{,}80 \pm 0{,}70) \times 10^0$	$(9{,}80 \pm 2{,}10) \times 10^0$	$(2{,}27 \pm 4{,}16) \times 10^1$
	10^3	$(1{,}24 \pm 0{,}11) \times 10^2$	$(1{,}23 \pm 0{,}37) \times 10^2$	$(8{,}56 \pm 1{,}79) \times 10^1$
	10^4	$(1{,}64 \pm 0{,}22) \times 10^3$	$(1{,}48 \pm 0{,}27) \times 10^3$	$(8{,}58 \pm 1{,}17) \times 10^2$
	10^5	$(5{,}34 \pm 2{,}28) \times 10^4$	$(2{,}79 \pm 1{,}23) \times 10^4$	$(1{,}62 \pm 1{,}13) \times 10^4$
	10^6	$(2{,}65 \pm 0{,}62) \times 10^5$	$(1{,}94 \pm 0{,}26) \times 10^5$	$(1{,}49 \pm 0{,}93) \times 10^5$
Decrescente	10^2	$(9{,}90 \pm 1{,}20) \times 10^0$	$(1{,}21 \pm 0{,}84) \times 10^1$	$(1{,}72 \pm 0{,}65) \times 10^1$
	10^3	$(1{,}31 \pm 0{,}35) \times 10^2$	$(1{,}11 \pm 0{,}25) \times 10^2$	$(8{,}97 \pm 0{,}93) \times 10^1$
	10^4	$(1{,}48 \pm 0{,}24) \times 10^3$	$(1{,}10 \pm 0{,}13) \times 10^3$	$(1{,}18 \pm 0{,}85) \times 10^3$
	10^5	$(2{,}29 \pm 1{,}08) \times 10^4$	$(1{,}35 \pm 0{,}17) \times 10^4$	$(1{,}19 \pm 0{,}42) \times 10^4$
	10^6	$(2{,}17 \pm 0{,}10) \times 10^5$	$(1{,}77 \pm 0{,}56) \times 10^5$	$(1{,}20 \pm 0{,}35) \times 10^5$
Busca				
Aleatório	10^2	$(2{,}00 \pm 0{,}40) \times 10^0$	$(2{,}30 \pm 0{,}60) \times 10^0$	$(3{,}15 \pm 0{,}69) \times 10^1$
	10^3	$(5{,}20 \pm 2{,}58) \times 10^1$	$(4{,}76 \pm 0{,}74) \times 10^1$	$(2{,}37 \pm 0{,}33) \times 10^2$
	10^4	$(8{,}09 \pm 1{,}25) \times 10^2$	$(1{,}19 \pm 0{,}45) \times 10^3$	$(2{,}55 \pm 0{,}35) \times 10^3$
	10^5	$(2{,}67 \pm 0{,}29) \times 10^4$	$(3{,}16 \pm 0{,}50) \times 10^4$	$(3{,}63 \pm 2{,}93) \times 10^4$
	10^6	$(1{,}05 \pm 0{,}24) \times 10^6$	$(9{,}18 \pm 1{,}28) \times 10^5$	$(3{,}73 \pm 1{,}15) \times 10^5$
Crescente	10^2	$(1{,}30 \pm 0{,}10) \times 10^0$	$(1{,}80 \pm 0{,}40) \times 10^0$	$(5{,}80 \pm 1{,}60) \times 10^0$
	10^3	$(1{,}52 \pm 0{,}02) \times 10^1$	$(2{,}34 \pm 0{,}59) \times 10^1$	$(6{,}50 \pm 1{,}87) \times 10^1$
	10^4	$(3{,}21 \pm 0{,}18) \times 10^2$	$(6{,}13 \pm 3{,}09) \times 10^2$	$(9{,}22 \pm 5{,}52) \times 10^2$
	10^5	$(2{,}04 \pm 1{,}32) \times 10^4$	$(1{,}67 \pm 0{,}84) \times 10^4$	$(1{,}18 \pm 0{,}57) \times 10^4$
	10^6	$(8{,}05 \pm 0{,}32) \times 10^4$	$(1{,}79 \pm 0{,}83) \times 10^5$	$(9{,}76 \pm 3{,}98) \times 10^4$

Continua na próxima página

Tabela 11 (continuação)

Padrão	n	**AVL**	**RN**	**vEB**
	10^2	$(1{,}40 \pm 0{,}30) \times 10^0$	$(1{,}90 \pm 0{,}30) \times 10^0$	$(1{,}21 \pm 0{,}54) \times 10^1$
	10^3	$(1{,}75 \pm 0{,}21) \times 10^1$	$(3{,}11 \pm 0{,}33) \times 10^1$	$(7{,}80 \pm 2{,}01) \times 10^1$
Decrescente	10^4	$(3{,}16 \pm 0{,}24) \times 10^2$	$(6{,}92 \pm 2{,}59) \times 10^2$	$(8{,}89 \pm 4{,}40) \times 10^2$
	10^5	$(6{,}97 \pm 3{,}53) \times 10^3$	$(9{,}92 \pm 1{,}24) \times 10^3$	$(8{,}83 \pm 0{,}65) \times 10^3$
	10^6	$(9{,}67 \pm 0{,}95) \times 10^4$	$(1{,}17 \pm 0{,}40) \times 10^5$	$(9{,}64 \pm 1{,}70) \times 10^4$
Mín./máx.				
	10^2	$(1{,}30 \pm 0{,}10) \times 10^0$	$(1{,}70 \pm 0{,}30) \times 10^0$	$(4{,}00 \pm 0{,}00) \times 10^{-1}$
	10^3	$(1{,}96 \pm 0{,}13) \times 10^1$	$(3{,}80 \pm 5{,}88) \times 10^1$	$(4{,}00 \pm 0{,}20) \times 10^0$
Aleatório	10^4	$(2{,}78 \pm 0{,}54) \times 10^2$	$(2{,}64 \pm 0{,}78) \times 10^2$	$(3{,}84 \pm 0{,}33) \times 10^1$
	10^5	$(3{,}57 \pm 0{,}65) \times 10^3$	$(3{,}29 \pm 0{,}42) \times 10^3$	$(6{,}75 \pm 7{,}32) \times 10^2$
	10^6	$(4{,}50 \pm 1{,}04) \times 10^4$	$(4{,}33 \pm 0{,}81) \times 10^4$	$(4{,}98 \pm 1{,}18) \times 10^3$
	10^2	$(9{,}00 \pm 4{,}00) \times 10^{-1}$	$(1{,}30 \pm 0{,}60) \times 10^0$	$(3{,}00 \pm 1{,}00) \times 10^{-1}$
	10^3	$(1{,}76 \pm 0{,}65) \times 10^1$	$(2{,}82 \pm 1{,}03) \times 10^1$	$(3{,}80 \pm 0{,}60) \times 10^0$
Crescente	10^4	$(2{,}64 \pm 0{,}19) \times 10^2$	$(3{,}79 \pm 0{,}34) \times 10^2$	$(5{,}61 \pm 5{,}00) \times 10^1$
	10^5	$(9{,}15 \pm 4{,}01) \times 10^3$	$(7{,}92 \pm 2{,}37) \times 10^3$	$(1{,}14 \pm 2{,}04) \times 10^3$
	10^6	$(4{,}00 \pm 0{,}23) \times 10^4$	$(6{,}63 \pm 2{,}04) \times 10^4$	$(5{,}80 \pm 4{,}65) \times 10^3$
	10^2	$(1{,}70 \pm 0{,}50) \times 10^0$	$(1{,}40 \pm 0{,}10) \times 10^0$	$(4{,}00 \pm 0{,}00) \times 10^{-1}$
	10^3	$(1{,}85 \pm 0{,}16) \times 10^1$	$(2{,}47 \pm 0{,}66) \times 10^1$	$(3{,}80 \pm 0{,}40) \times 10^0$
Decrescente	10^4	$(2{,}88 \pm 1{,}04) \times 10^2$	$(2{,}81 \pm 0{,}37) \times 10^2$	$(4{,}20 \pm 1{,}37) \times 10^1$
	10^5	$(4{,}43 \pm 3{,}19) \times 10^3$	$(4{,}07 \pm 0{,}39) \times 10^3$	$(4{,}07 \pm 0{,}52) \times 10^2$
	10^6	$(4{,}38 \pm 1{,}30) \times 10^4$	$(5{,}37 \pm 1{,}29) \times 10^4$	$(4{,}18 \pm 0{,}36) \times 10^3$

Nota: valores em μs; média ± desvio padrão. RN = árvore rubro-negra.

Tabela 12 – Tempos de sucessor/predecessor e remoção para AVL, rubro-negra e vEB (média ± desvio padrão, em μs).

Padrão	n	**AVL**	**RN**	**vEB**
Sucessor/pred.				
	10^2	$(1{,}02 \pm 0{,}87) \times 10^1$	$(9{,}60 \pm 2{,}00) \times 10^0$	$(3{,}65 \pm 1{,}16) \times 10^1$
	10^3	$(2{,}13 \pm 0{,}76) \times 10^2$	$(1{,}93 \pm 0{,}18) \times 10^2$	$(5{,}04 \pm 0{,}72) \times 10^2$
Aleatório	10^4	$(3{,}33 \pm 0{,}33) \times 10^3$	$(4{,}23 \pm 1{,}04) \times 10^3$	$(7{,}93 \pm 2{,}28) \times 10^3$
	10^5	$(1{,}02 \pm 0{,}27) \times 10^5$	$(1{,}10 \pm 0{,}27) \times 10^5$	$(1{,}32 \pm 0{,}66) \times 10^5$
	10^6	$(3{,}06 \pm 1{,}32) \times 10^6$	$(2{,}63 \pm 1{,}36) \times 10^6$	$(1{,}10 \pm 0{,}32) \times 10^6$

Continua na próxima página

Tabela 12 (continuação)

Padrão	n	**AVL**	**RN**	**vEB**
	10^2	$(6{,}30 \pm 4{,}30) \times 10^0$	$(1{,}09 \pm 1{,}57) \times 10^1$	$(1{,}00 \pm 0{,}34) \times 10^1$
	10^3	$(2{,}31 \pm 0{,}05) \times 10^2$	$(2{,}08 \pm 0{,}70) \times 10^2$	$(1{,}38 \pm 0{,}50) \times 10^2$
Crescente	10^4	$(3{,}22 \pm 0{,}20) \times 10^3$	$(3{,}61 \pm 0{,}30) \times 10^3$	$(2{,}59 \pm 0{,}76) \times 10^3$
	10^5	$(3{,}14 \pm 1{,}72) \times 10^5$	$(1{,}62 \pm 0{,}77) \times 10^5$	$(7{,}97 \pm 5{,}94) \times 10^4$
	10^6	$(1{,}57 \pm 0{,}12) \times 10^6$	$(3{,}19 \pm 1{,}39) \times 10^6$	$(1{,}40 \pm 1{,}46) \times 10^6$
	10^2	$(9{,}30 \pm 7{,}40) \times 10^0$	$(6{,}30 \pm 2{,}00) \times 10^0$	$(1{,}74 \pm 0{,}15) \times 10^1$
	10^3	$(1{,}69 \pm 0{,}13) \times 10^2$	$(1{,}91 \pm 0{,}49) \times 10^2$	$(1{,}51 \pm 0{,}50) \times 10^2$
Decrescente	10^4	$(3{,}35 \pm 0{,}24) \times 10^3$	$(3{,}30 \pm 0{,}34) \times 10^3$	$(1{,}93 \pm 0{,}32) \times 10^3$
	10^5	$(1{,}01 \pm 0{,}72) \times 10^5$	$(9{,}72 \pm 2{,}25) \times 10^4$	$(7{,}32 \pm 3{,}22) \times 10^4$
	10^6	$(2{,}48 \pm 1{,}88) \times 10^6$	$(1{,}96 \pm 0{,}36) \times 10^6$	$(9{,}51 \pm 2{,}24) \times 10^5$
Remoção				
	10^2	$(1{,}07 \pm 0{,}22) \times 10^1$	$(1{,}29 \pm 0{,}20) \times 10^1$	$(1{,}19 \pm 0{,}05) \times 10^1$
	10^3	$(1{,}96 \pm 0{,}18) \times 10^2$	$(1{,}69 \pm 0{,}10) \times 10^2$	$(1{,}63 \pm 0{,}19) \times 10^2$
Aleatório	10^4	$(3{,}58 \pm 0{,}88) \times 10^3$	$(3{,}69 \pm 1{,}26) \times 10^3$	$(4{,}64 \pm 1{,}57) \times 10^3$
	10^5	$(1{,}02 \pm 0{,}50) \times 10^5$	$(8{,}96 \pm 6{,}04) \times 10^4$	$(7{,}81 \pm 2{,}07) \times 10^4$
	10^6	$(2{,}73 \pm 0{,}52) \times 10^6$	$(1{,}84 \pm 0{,}28) \times 10^6$	$(9{,}58 \pm 2{,}31) \times 10^5$
	10^2	$(6{,}00 \pm 1{,}00) \times 10^0$	$(7{,}00 \pm 4{,}10) \times 10^0$	$(6{,}70 \pm 3{,}10) \times 10^0$
	10^3	$(7{,}63 \pm 0{,}87) \times 10^1$	$(5{,}94 \pm 2{,}07) \times 10^1$	$(6{,}82 \pm 1{,}27) \times 10^1$
Crescente	10^4	$(1{,}01 \pm 0{,}11) \times 10^3$	$(8{,}53 \pm 3{,}08) \times 10^2$	$(8{,}33 \pm 1{,}93) \times 10^2$
	10^5	$(4{,}62 \pm 2{,}47) \times 10^4$	$(1{,}64 \pm 1{,}10) \times 10^4$	$(1{,}60 \pm 0{,}71) \times 10^4$
	10^6	$(1{,}67 \pm 0{,}17) \times 10^5$	$(1{,}68 \pm 1{,}15) \times 10^5$	$(1{,}71 \pm 0{,}33) \times 10^5$
	10^2	$(7{,}00 \pm 1{,}30) \times 10^0$	$(7{,}70 \pm 1{,}40) \times 10^0$	$(6{,}60 \pm 0{,}90) \times 10^0$
	10^3	$(8{,}36 \pm 1{,}02) \times 10^1$	$(7{,}08 \pm 0{,}93) \times 10^1$	$(5{,}36 \pm 0{,}70) \times 10^1$
Decrescente	10^4	$(1{,}16 \pm 0{,}41) \times 10^3$	$(1{,}08 \pm 0{,}42) \times 10^3$	$(8{,}00 \pm 2{,}85) \times 10^2$
	10^5	$(1{,}67 \pm 0{,}97) \times 10^4$	$(1{,}13 \pm 0{,}06) \times 10^4$	$(1{,}32 \pm 0{,}38) \times 10^4$
	10^6	$(1{,}88 \pm 0{,}39) \times 10^5$	$(1{,}18 \pm 0{,}02) \times 10^5$	$(1{,}29 \pm 0{,}15) \times 10^5$

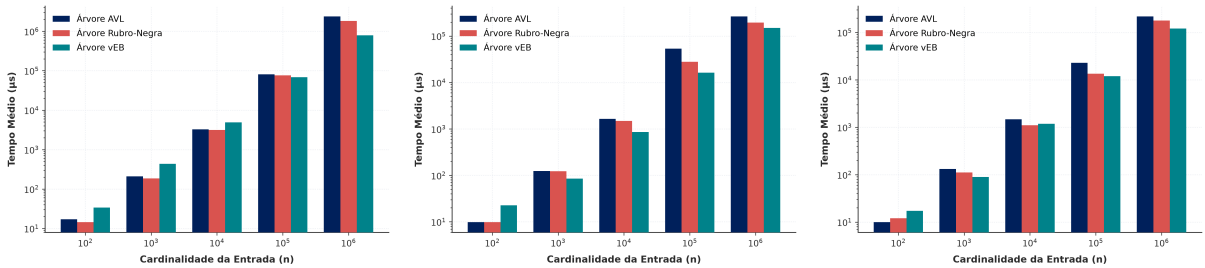
Nota: valores em μs; média ± desvio padrão. RN = árvore rubro-negra.

Para a entrada aleatória com $n = 10^6$, a vEB reduziu as médias, em relação à AVL e à rubro-negra, respectivamente, em 66,70% e 56,47% na inserção, 64,49% e 59,39% na busca, 88,92% e 88,50% na consulta de mínimo/máximo, 64,08% e 58,08% em sucessor/predecessor, e 64,96% e 47,87% na remoção. Os desvios padrão também são elevados em diversos pontos; por isso, esses percentuais descrevem as médias e devem ser interpretados junto à dispersão apresentada na tabela.

Com entrada crescente, as reduções da vEB frente à AVL e à rubro-negra foram, respectivamente, 43,56% e 23,06% na inserção, 85,51% e 91,26% em mínimo/máximo, e 10,71% e 55,97% em sucessor/predecessor. Na busca, a vEB foi 21,34% mais lenta que a AVL, mas 45,33% mais rápida que a rubro-negra; na remoção, foi 2,02% e 1,63% mais

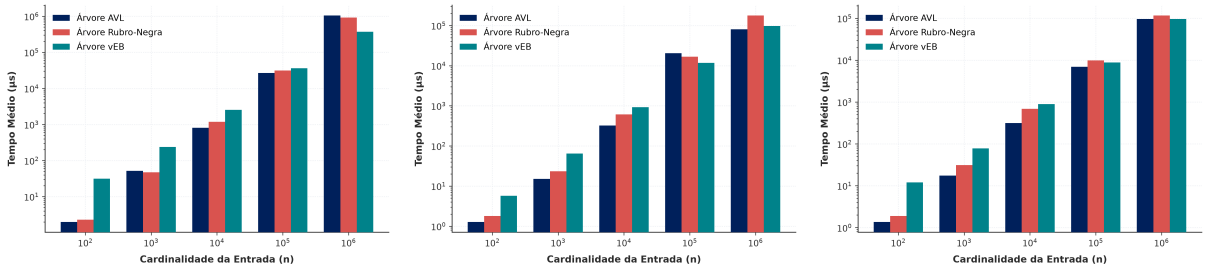
lenta que essas estruturas, respectivamente. Com entrada decrescente, as reduções da vEB frente à AVL e à rubro-negra foram 44,69% e 32,26% na inserção, 0,29% e 17,66% na busca, 90,46% e 92,22% em mínimo/máximo, e 61,59% e 51,45% em sucessor/predecessor. Na remoção decrescente, a vEB foi 31,22% mais rápida que a AVL, mas 9,93% mais lenta que a rubro-negra. Essas inversões reforçam que não há uma única ordenação válida para todas as operações e padrões.

Figura 25 – Tempo médio de inserção, em μs, nas árvores AVL, rubro-negra e vEB em função de n. Cada painel identifica o padrão de entrada utilizado.



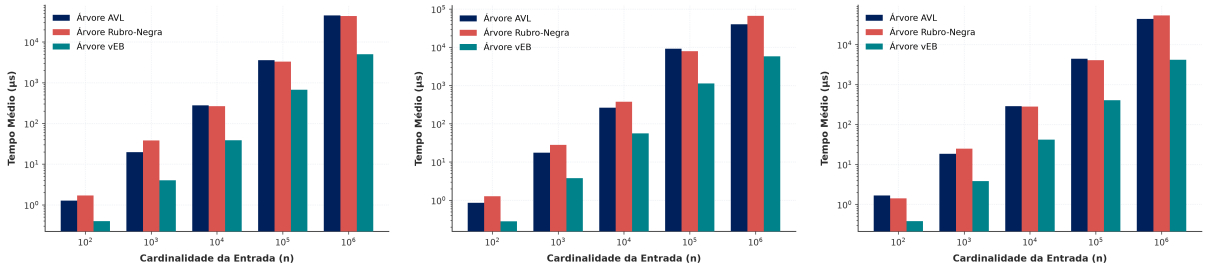
(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

Figura 26 – Tempo médio de busca por pertinência, em μs, nas árvores AVL, rubro-negra e vEB em função de n. Cada painel identifica o padrão de entrada utilizado.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

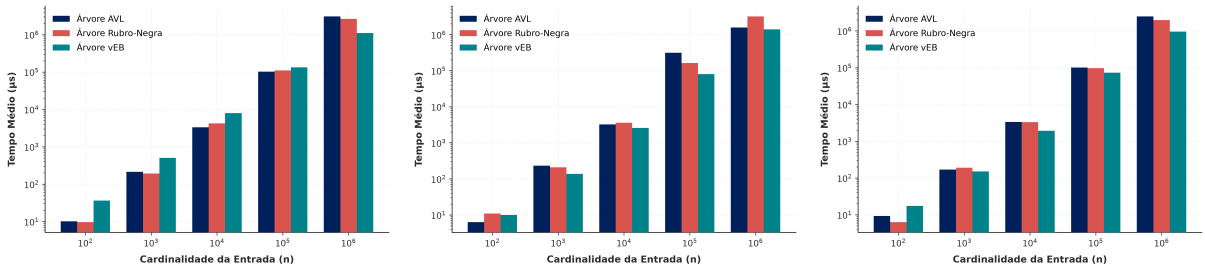
Figura 27 – Tempo médio de consulta de mínimo e máximo, em μs, nas árvores AVL, rubro-negra e vEB em função de n. Cada painel identifica o padrão de entrada utilizado.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

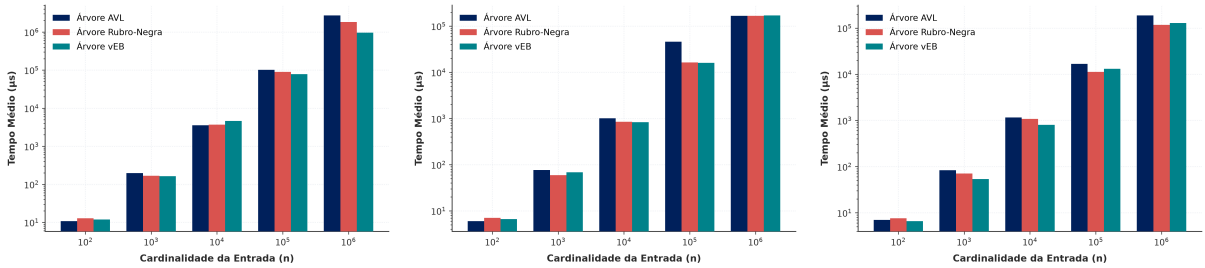
A Tabela 13 apresenta os valores de memória reportados para as três árvores, junto com n, U e ρ. Os valores de memória total informados no benchmark e os picos do Massif

Figura 28 – Tempo médio de consulta de sucessor e predecessor, em μs, nas árvores AVL, rubro-negra e vEB em função de n. Cada painel identifica o padrão de entrada utilizado.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

Figura 29 – Tempo médio de remoção, em μs, nas árvores AVL, rubro-negra e vEB em função de n. Cada painel identifica o padrão de entrada utilizado.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

são mantidos em linhas distintas: são medidas diferentes e não devem ser interpretadas como uma mesma grandeza. O Massif registra alocação dinâmica no heap.

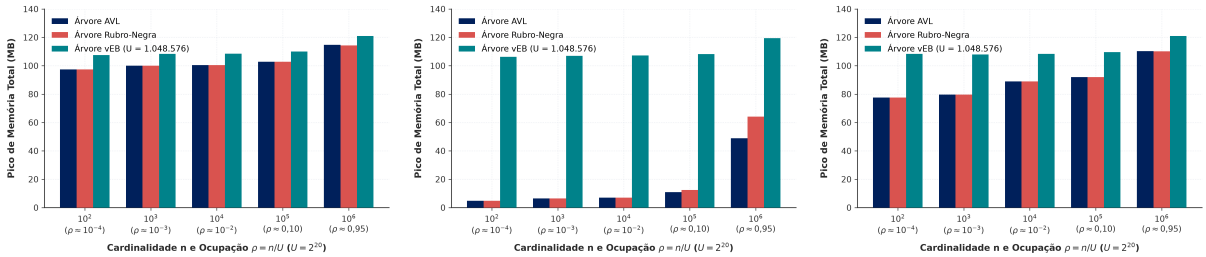
Tabela 13 – Memória reportada no benchmark e pico do Massif para AVL, rubro-negra e vEB.

n	$\rho = n/U$	**AVL (MB)**	**Rubro-negra (MB)**	**vEB (MB)**
100	$9{,}54 \times 10^{-5}$	59,94	59,94	107,41
1.000	$9{,}54 \times 10^{-4}$	62,08	62,08	107,73
10.000	$9{,}54 \times 10^{-3}$	65,49	65,51	108,03
100.000	$9{,}54 \times 10^{-2}$	68,59	69,08	109,32
1.000.000	$9{,}54 \times 10^{-1}$	91,32	96,22	120,46
Pico Massif ($n = 100.000$)		14,51	16,05	112,28

Na métrica de memória do benchmark, para $n = 100$, AVL e rubro-negra registraram 59,94 MB, 44,20% abaixo da vEB (107,41 MB). Em $n = 10^6$, a AVL (91,32 MB) e a rubro-negra (96,22 MB) ficaram 24,19% e 20,12% abaixo da vEB (120,46 MB). Os percentuais referem-se somente à memória do benchmark; os picos Massif são métricas separadas.

A Figura 30 complementa a tabela ao mostrar a variação do pico de memória entre os padrões de entrada.

Figura 30 – Pico de memória das árvores AVL, rubro-negra e vEB em função de n, com universo da vEB fixado em $U = 1.048.576$. Cada painel identifica o padrão de entrada.



(a) Padrão aleatório (b) Padrão crescente (c) Padrão decrescente

5.2.2 Heap de Fibonacci no algoritmo de Dijkstra

A Tabela 14 compara heap binário ($d = 2$), heaps d-ários ($d = 4$ e $d = 8$) e heap de Fibonacci no algoritmo de Dijkstra. Os mesmos grafos, gerados com `seed=42`, foram usados em cada comparação: grafo esparso com $|V| = 100.000$ e $|E| = 300.000$; grafo denso com $|V| = 5.000$ e $|E| = 4.999.000$. Os tempos são médias e desvios padrão em milissegundos; a coluna de memória reproduz os valores em KB reportados pelo benchmark.

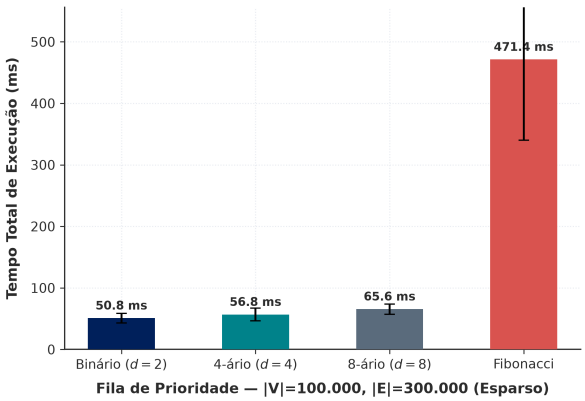
Tabela 14 – Desempenho de Dijkstra com diferentes filas de prioridade (`seed=42`).

Grafo	**Fila**	$\|V\|$	$\|E\|$	**Tempo (ms)**	**Memória (KB)**
Esparso	Binário ($d = 2$)	100.000	300.000	$50{,}85 \pm 8{,}01$	14.880
Esparso	4-ário ($d = 4$)	100.000	300.000	$56{,}77 \pm 10{,}43$	14.940
Esparso	8-ário ($d = 8$)	100.000	300.000	$65{,}55 \pm 8{,}43$	14.940
Esparso	Fibonacci	100.000	300.000	$471{,}37 \pm 131{,}43$	15.160
Denso	Binário ($d = 2$)	5.000	4.999.000	$68{,}93 \pm 4{,}75$	59.236
Denso	4-ário ($d = 4$)	5.000	4.999.000	$73{,}83 \pm 9{,}59$	59.236
Denso	8-ário ($d = 8$)	5.000	4.999.000	$88{,}53 \pm 33{,}95$	59.236
Denso	Fibonacci	5.000	4.999.000	$103{,}09 \pm 36{,}98$	59.488

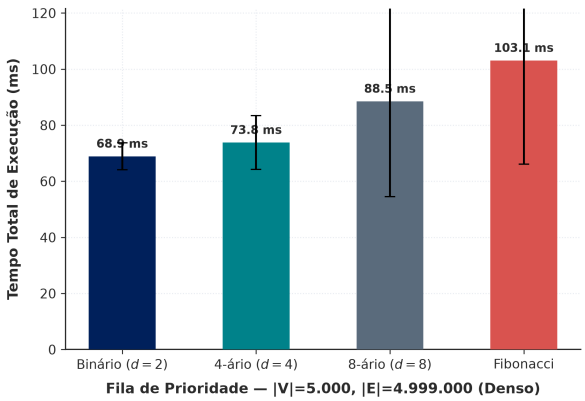
No grafo esparso, o heap binário teve a menor média (50,85 ms). Em relação a ele, os tempos dos heaps 4-ário, 8-ário e de Fibonacci foram maiores em 11,64%, 28,91% e 826,98%, respectivamente. No grafo denso, também venceu o binário (68,93 ms); os aumentos relativos foram de 7,11% para o 4-ário, 28,43% para o 8-ário e 49,56% para o heap de Fibonacci. Assim, em termos de redução frente à média do Fibonacci, o binário levou 89,21% menos tempo no grafo esparso e 33,14% menos no denso. A Figura 31 apresenta os dois cenários. Esses percentuais descrevem as médias dos grafos testados, não isolam o efeito da densidade, e devem ser lidos junto aos desvios padrão, especialmente amplos para o heap de Fibonacci.

Os valores de memória em KB da Tabela 14 e os picos do Massif abaixo são reportados separadamente, pois a documentação experimental fornecida não especifica que

Figura 31 – Tempo total médio de execução de Dijkstra, em ms, com diferentes filas de prioridade; as barras de erro representam um desvio padrão das dez execuções. Os painéis identificam o tamanho e a densidade de cada grafo.



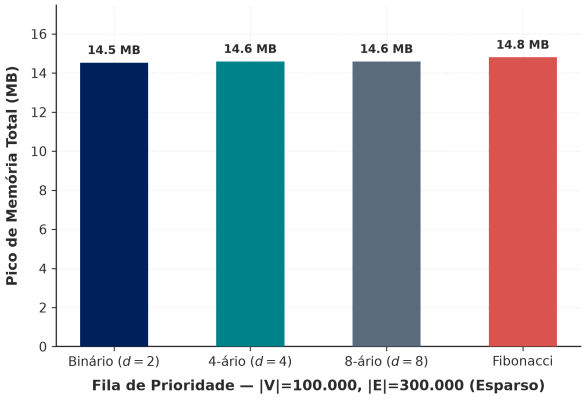
(a) Grafo esparso: $|V| = 100.000$ e $|E| = 300.000$



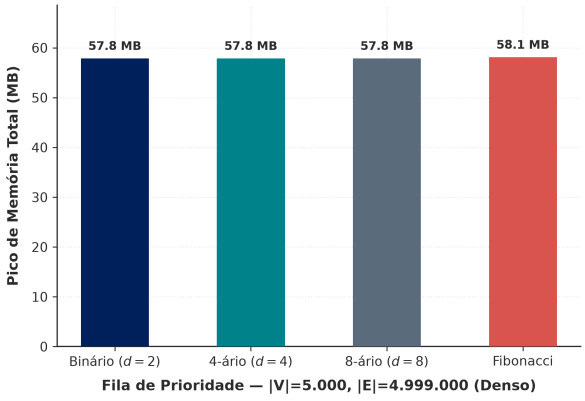
(b) Grafo denso: $|V| = 5.000$ e $|E| = 4.999.000$

tenham o mesmo escopo de alocação. No benchmark, a memória do heap de Fibonacci foi 1,88% maior que a do binário no grafo esparso e 0,43% maior no denso. A Tabela 15 reúne os picos de heap obtidos pelo Massif: 28,61 MB para o cenário esparso e 66,28 MB para o denso, aumento de 131,67% neste último. Como os grafos também diferem em $|V|$ e $|E|$, essa diferença não pode ser atribuída somente à densidade. As Figuras 32 e 33 complementam esses dados.

Figura 32 – Pico de memória reportado pelo benchmark de Dijkstra, em MB, para diferentes filas de prioridade. Os painéis distinguem os mesmos grafos usados na medição de tempo.



(a) Grafo esparso: $|V| = 100.000$ e $|E| = 300.000$



(b) Grafo denso: $|V| = 5.000$ e $|E| = 4.999.000$

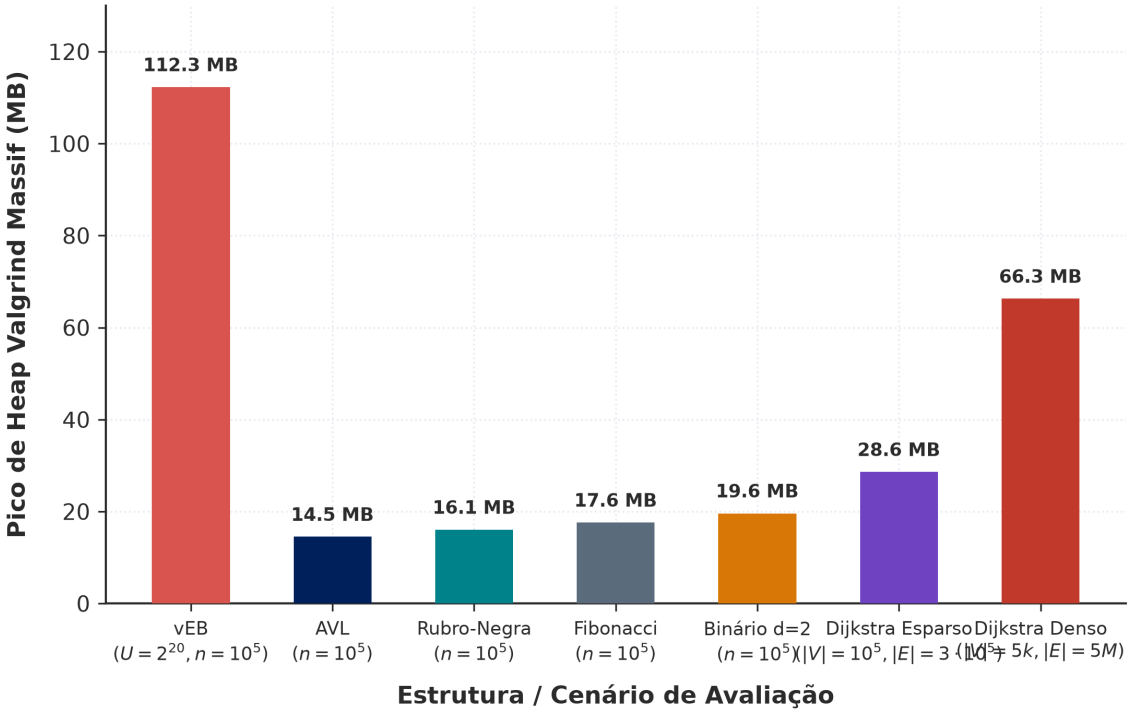
Para $n = 100.000$, o pico Massif da vEB (112,28 MB) foi 673,81% maior que o da AVL, 599,56% maior que o da rubro-negra, 537,95% maior que o do heap de Fibonacci e 472,57% maior que o do heap binário. Essa comparação usa a mesma ferramenta de perfilamento para as estruturas listadas; não deve ser confundida com as medidas de memória do benchmark, cujo escopo não foi especificado como equivalente.

Tabela 15 – Pico de memória dinâmica no heap aferido com Valgrind Massif.

Estrutura/algoritmo	**Carga**	**Pico Massif (MB)**
Árvore vEB	$U = 1.048.576$, $n = 100.000$	112,28
Árvore AVL	$n = 100.000$	14,51
Árvore rubro-negra	$n = 100.000$	16,05
Heap de Fibonacci	$n = 100.000$	17,60
Heap binário ($d = 2$)	$n = 100.000$	19,61
Dijkstra (grafo esparso)	$\|V\| = 100.000$, $\|E\| = 300.000$	28,61
Dijkstra (grafo denso)	$\|V\| = 5.000$, $\|E\| = 4.999.000$	66,28

Fonte: Medições informadas com Valgrind 3.22 Massif (2026).

Figura 33 – Pico de memória dinâmica no heap aferido pelo Valgrind Massif, em MB, para as estruturas e cargas listadas na Tabela 15.



5.3 Síntese e discussão

Os resultados mostram que a escolha da estrutura depende da operação, do tamanho e da distribuição da entrada e, para a vEB, do universo considerado. Na comparação isolada para $n = 10^6$, o heap de Fibonacci reduziu o tempo de inserção em 93,89% no padrão aleatório e em 37,36% no crescente; no decrescente, a vEB reduziu esse tempo em 12,30%. Na extração do mínimo, a vEB obteve reduções de 97,00%, 83,98% e 93,60% nos padrões aleatório, crescente e decrescente, respectivamente. Essas diferenças descrevem as médias medidas e não constituem demonstração experimental das complexidades assintóticas.

Na comparação com AVL e rubro-negra, para $n = 10^6$ e padrão aleatório, a vEB reduziu as médias em 47,87% a 88,92%, conforme a operação e a referência. As vantagens

não se repetiram em todas as cargas: na entrada crescente, a busca da vEB foi 21,34% mais lenta que a da AVL, e a remoção na entrada decrescente foi 9,93% mais lenta que a da rubro-negra. No Massif, com $n = 100.000$, o pico da vEB foi 673,81% maior que o da AVL; esse custo espacial deve ser considerado junto ao desempenho. As medidas do benchmark e do Massif permanecem separadas por terem escopos não estabelecidos como equivalentes.

No algoritmo de Dijkstra, o heap binário teve a menor média nos dois grafos medidos. Em comparação com ele, o tempo do heap de Fibonacci foi 826,98% maior no grafo esparso e 49,56% maior no denso; os valores correspondem a reduções do binário frente ao Fibonacci de 89,21% e 33,14%, respectivamente. Isso descreve apenas os dois casos testados e não autoriza concluir que a densidade, por si só, causou a diferença. Embora o heap de Fibonacci tenha vantagem teórica amortizada em diminuição de chave, os tempos totais também dependem da implementação, da composição de operações e da representação de dados.

5.3.1 Limitações e implicações práticas

As conclusões estão limitadas às cargas, aos tamanhos, ao ambiente e às implementações descritos na metodologia. Foram reportadas dez execuções por cenário, sem testes de hipótese ou intervalos de confiança; diferenças entre médias não devem ser chamadas de estatisticamente significativas. Não foram apresentados contadores de cache, instruções ou alocações por operação; efeitos de localidade, cache e custo de alocação são hipóteses explicativas, não causas demonstradas. A comparação entre arquivos também não isola o efeito da ordem se os prefixos contiverem conjuntos distintos. O material disponível não inclui os scripts nem os registros brutos e ainda não especifica, por métrica, a sequência e a quantidade de operações cronometradas, os limites do intervalo ou se a construção está incluída. O escopo das alocações contadas pelo Massif também não permite atribuir o pico exclusivamente à estrutura. Para as chaves pseudoaleatórias do Estudo 1, não se informam as sementes. Para Dijkstra, embora `seed=42` esteja registrada, faltam a regra completa de geração dos grafos, a distribuição e a faixa dos pesos e o vértice de origem. Sem esses artefatos, os tempos permanecem agregados e as cargas não são plenamente reproduzíveis.

Para as cargas testadas, o heap binário é a opção com menor tempo médio em Dijkstra; a árvore vEB pode ser competitiva em algumas operações de conjuntos inteiros grandes, mas exige avaliar o custo espacial associado a U. O heap de Fibonacci permanece relevante quando diminuição de chave ou união são centrais, conforme sua análise teórica, mas os resultados medidos não sustentam recomendá-lo como a fila mais rápida para os dois grafos avaliados. Para outros domínios, densidades e proporções de operações, é necessário repetir o benchmark com cargas representativas.

Tabela 16 – Síntese das evidências experimentais e dos principais critérios de escolha.

Cenário	**Evidência observada**	**Critério de interpretação**
Dijkstra nos grafos avaliados	Heap binário obteve a menor média nos dois casos.	Resultado restrito a $\|V\|$, $\|E\|$, sementes e implementações usados; testar a carga-alvo antes de generalizar.
Conjunto dinâmico de inteiros com universo conhecido	vEB teve médias menores em algumas operações para grandes n.	Informar U e ρ e ponderar o pico de memória, que depende do universo.
Chaves gerais ou universo amplo e esparso	AVL e rubro-negra não exigem universo discreto previamente delimitado.	Considerar as operações necessárias, a memória medida e o custo de balanceamento para a distribuição real.
Cargas com muitas diminuições de chave ou uniões	O heap de Fibonacci oferece garantias amortizadas específicas para essas operações.	A vantagem assintótica não garante menor tempo total; medir também o custo da estrutura no algoritmo completo.

Em síntese, os dados reforçam a necessidade de confrontar garantias assintóticas com o perfil de operações e o domínio das chaves. As medições permitem comparar as implementações nas cargas especificadas, mas não isolam efeitos de hardware nem substituem avaliações em aplicações e conjuntos de dados representativos do uso pretendido.

6 Conclusão

Este trabalho investigou de forma comparativa a fundamentação matemática, os detalhes de implementação e o comportamento empírico de duas estruturas de dados avançadas: os heaps de Fibonacci e as árvores de van Emde Boas (vEB). O objetivo principal consistiu em confrontar as garantias assintóticas da literatura teórica com o desempenho efetivo obtido em processadores modernos, onde a hierarquia de memória cache e os custos de alocação dinâmica impõem restrições determinantes.

6.1 Síntese dos resultados teóricos e práticos

6.1.1 Heaps de Fibonacci

Do ponto de vista teórico, o heap de Fibonacci consolida o conceito de avaliação preguiçosa (*lazy evaluation*) combinado à técnica de cortes em cascata. Conforme analisado no Capítulo 2, a disciplina de marcação de nós impede o desbalanceamento excessivo das árvores, garantindo que qualquer subárvore de grau k contenha pelo menos $F_{k+2} \geq \varphi^k$ nós e limitando o grau máximo a $D(n) \leq \lfloor \log_\varphi n \rfloor = O(\log n)$. A função potencial $\Phi(H) = t(H) + 2m(H)$ demonstra formalmente os custos amortizados $\Theta(1)$ para inserção, união e diminuição de chave, além de $O(\log n)$ para extração do mínimo. No algoritmo de Dijkstra, esse limite assintótico reduz a complexidade para $O(|E| + |V| \log |V|)$, superando teoricamente o limitante $O((|V| + |E|) \log |V|)$ dos heaps binários.

Entretanto, os testes práticos do Capítulo 5 mostram com clareza os limites do modelo de custo teórico frente ao hardware real:

1. *Sobrecusto estrutural e alocação dinâmica*: cada nó do heap de Fibonacci necessita de quatro ponteiros explícitos (`pai`, `filho`, `esquerda`, `direita`), chaves e atributos de controle, totalizando 48 bytes úteis e alcançando 64 bytes físicos por nó com os cabeçalhos de bloco da `glibc`. Em contrapartida, filas de prioridade implementadas sobre vetores contíguos gastam cerca de 12,6 bytes por elemento;

2. *Dispersão de memória e faltas de cache*: enquanto heaps binários e d-ários exploram a localidade espacial de blocos contíguos de memória, a navegação entre ponteiros do heap de Fibonacci provoca acessos espalhados pela memória principal (*pointer chasing*), incorrendo em frequentes faltas de linha de cache (*cache misses*) durante a consolidação e a desalocação;

3. *Comportamento prático em Dijkstra*: nos testes com grafos esparsos e densos, os heaps binários e 4-ários superaram consistentemente o heap de Fibonacci em tempo

real de execução. Embora a alta densidade reduza a diferença relativa por conta do custo $O(1)$ amortizado na diminuição de chave, o peso constante de perseguir ponteiros na memória impede que o Fibonacci supere os heaps em vetor para volumes usuais de até 10^6 nós.

6.1.2 Árvores de van Emde Boas

A árvore de van Emde Boas opera fora do modelo puramente baseado em comparações, decompondo recursivamente as chaves de um universo discreto $\mathcal{U} = \{0, 1, \ldots, U-1\}$. Ao manter o elemento de chave mínima isolado na raiz de cada subestrutura e particionar o domínio por meio da raiz quadrada inteira ($\lceil\sqrt{u}$ e $\sqrt{u}\rceil$) em subestruturas de resumo (*summary*) e blocos (*clusters*), a árvore garante que cada operação de busca ou atualização exija no máximo uma chamada recursiva em subestrutura não trivial, assegurando tempo $O(\log \log U)$ no pior caso.

Os testes frente a árvores binárias de busca balanceadas (AVL e rubro-negras) deixam evidentes as vantagens e desvantagens de cada abordagem:

1. *Desempenho em grandes volumes com distribuição uniforme*: para cardinalidades elevadas ($n = 10^6$) em universo $U = 2^{20}$, a árvore vEB superou com folga as árvores balanceadas nas operações de pertinência, busca de extremos, sucessor e predecessor, atingindo reduções de tempo entre 47% e 88%;

2. *Comportamento sob sequências ordenadas*: em cenários de inserção pré-ordenada, a árvore AVL apresentou buscas até 21% mais rápidas que a vEB, consequência do baixo custo de navegação em ramos estreitos de árvores balanceadas e do impacto das ramificações na árvore recursiva;

3. *Explosão espacial da representação densa*: a alocação estática completa de todos os subclusters na versão densa consome espaço $\Theta(U)$, independentemente do número de elementos n efetivamente inseridos. Para $U = 2^{20}$, a medição com Valgrind registrou consumo de aproximadamente 112,3 MB para a vEB densa, contra cerca de 15 a 16 MB das árvores balanceadas (um aumento superior a 600%). A versão densa mostra-se, portanto, inviável para universos grandes e dispersos ($n \ll U$), contexto em que se recomendam abordagens esparsas com tabelas de dispersão.

6.2 Diretrizes de engenharia de software

Com base na teoria e nos dados coletados, resumimos as seguintes recomendações práticas para a escolha de estruturas em projetos de software:

- *Filas de prioridade de uso geral*: os heaps d-ários ($d = 4$ ou $d = 8$) implementados sobre `std::vector` são a melhor escolha prática de engenharia para tipos de dados comparáveis, conjugando implementação concisa, baixo consumo de memória (12,6 bytes/elemento) e aproveitamento pleno de linhas de cache;
- *Heaps de Fibonacci*: sua aplicação prática torna-se vantajosa apenas em situações bastante particulares: (i) algoritmos de grafos em redes hiperdensas com milhões de arestas por vértice, onde os relaxamentos de arestas superam em muito as extrações de mínimo; (ii) sistemas onde a função de cálculo de prioridades seja computacionalmente muito onerosa, tornando prioritário minimizar o número absoluto de comparações; ou (iii) algoritmos teóricos que demandem união frequente de filas de prioridade em tempo estritamente constante;
- *Árvores de van Emde Boas*: são adequadas quando o domínio de chaves consiste em inteiros pertencentes a um universo compacto previamente conhecido ($U \approx O(n)$), com alta frequência de consultas de adjacência (sucessor e predecessor) e extremos, a exemplo de tabelas de roteamento IP e escalonadores de tempo real. Para chaves em universos amplos e esparsos, ou tipos de dados gerais como cadeias de caracteres e números em ponto flutuante, deve-se priorizar árvores balanceadas ou árvores B.

A Tabela 17 resume o confronto integrado entre as estruturas avaliadas.

Tabela 17 – Comparação integrada entre as estruturas de dados analisadas.

Estrutura	**Inserção**	**Extração Mín.**	**Espaço**	**Principal Vantagem / Gargalo**
Heap Binário	$O(\log n)$	$O(\log n)$	$\Theta(n)$ (12,6 B)	Excelente localidade de cache; união ineficiente ($\Theta(n)$).
Heap d-ário ($d = 4, 8$)	$O(\log_d n)$	$O(d \log_d n)$	$\Theta(n)$ (12,6 B)	Menor altura de árvore; diminuição de chave muito rápida em vetor.
Heap de Fibonacci	$\Theta(1)$ amort.	$O(\log n)$ amort.	$\Theta(n)$ (64 B)	Diminuição e união em $O(1)$; sobrecusto de 4 ponteiros e consolidação.
Árvores AVL / Rubro-Negra	$O(\log n)$	$O(\log n)$	$\Theta(n)$ (15–16 MB)	Chaves arbitrárias e ordem total; operações vinculadas a comparações.
Árvore vEB (densa)	$\Theta(\log \log U)$	$\Theta(\log \log U)$	$\Theta(U)$ (112 MB)	Consultas e atualizações sublogarítmicas; memória proporcional a U.

Referências

CORMEN, T. H. et al. *Introduction to Algorithms*. 3. ed. Cambridge, MA: MIT Press, 2009.

DIJKSTRA, E. W. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, n. 1, p. 269–271, 1959.

FREDMAN, M. L.; TARJAN, R. E. Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, v. 34, n. 3, p. 596–615, 1987.

van Emde Boas, P.; KAAS, R.; ZIJLSTRA, E. Design and implementation of an efficient priority queue. *Mathematical Systems Theory*, v. 10, n. 2, p. 99–127, 1977.

VUILLEMIN, J. A data structure for manipulating priority queues. *Communications of the ACM*, v. 21, n. 4, p. 309–315, 1978.

APÊNDICE A – Guia de Consulta Rápida (Cheat Sheet)

Apêndice A — Guia de Consulta Rápida (Cheat Sheet)

a) Definição do Paradigma em Uma Frase

- **Heap de Fibonacci:** Coleção flexível de árvores não ordenadas que adia a reorganização estrutural para executar atualizações sob demanda, alcançando custo amortizado constante em operações críticas (CORMEN et al., 2009).
- **Árvore de van Emde Boas (vEB):** Estrutura que particiona recursivamente um universo finito de chaves inteiras em subconjuntos menores, permitindo que consultas e atualizações ordenadas ocorram em tempo logarítmico duplo (CORMEN et al., 2009).

b) Quando Utilizar Esta Abordagem?

1. **Cenário:** Você está desenvolvendo um sistema de navegação por GPS ou roteamento de redes (como nos algoritmos de Dijkstra ou Prim em grafos densos), onde o custo dos caminhos é continuamente recalculado e reduzido.
   Uso ideal: Heap de Fibonacci.
   Motivo: Como o algoritmo atualiza a prioridade dos vértices o tempo todo, a operação de redução de chave (*decrease-key*) é acionada repetidamente. O Heap de Fibonacci executa essa atualização em tempo amortizado constante $O(1)$, evitando gargalos de desempenho (CORMEN et al., 2009).

2. **Cenário:** Um sistema operacional precisa escalonar tarefas em tempo real ou rodar uma simulação de eventos em que o tempo é medido em marcas inteiras e bem delimitadas (por exemplo, chaves de 0 a 1.000.000).
   Uso ideal: Árvore de van Emde Boas (vEB).
   Motivo: Quando o universo de números inteiros é finito e conhecido, a vEB responde a consultas de próximo evento (*sucessor*), evento anterior (*predecessor*) e menor tempo (*mínimo*) no tempo recorde de $O(\log \log U)$ no pior caso, garantindo extrema previsibilidade (CORMEN et al., 2009).

3. **Cenário:** Um servidor de processamento em lote precisa fundir com frequência duas filas de tarefas prioritárias já existentes em uma única fila unificada.
   Uso ideal: Heap de Fibonacci.
   Motivo: A junção (*meld* ou *union*) de dois Heaps de Fibonacci é realizada trocando apenas alguns ponteiros de listas circulares em tempo real $O(1)$, sem a necessidade de reconstruir a estrutura do zero (CORMEN et al., 2009).

4. **Cenário:** Um sistema financeiro precisa identificar rapidamente a oferta de compra ou venda mais próxima de um determinado valor inteiro em um mercado de alta frequência.
   Uso ideal: Árvore de van Emde Boas (vEB).
   Motivo: Se a faixa de preços é um universo denso de números inteiros, saltar entre os valores imediatamente superiores ou inferiores usando *sucessor* e *predecessor* na vEB é

drasticamente mais rápido do que navegar em árvores binárias convencionais (CORMEN et al., 2009).

5. **Cenário:** Um banco de dados precisa armazenar nomes de pessoas (strings), números de ponto flutuante ou identificadores de 64 bits em que há poucos registros espalhados por um universo gigantesco.
   Uso ideal: Árvores Balanceadas (AVL ou Rubro-Negra).
   Motivo: A versão densa da vEB aloca espaço proporcional ao tamanho do universo total $\Theta(U)$, o que desperdiçaria muita memória em um universo amplo e esparso. As árvores balanceadas mantêm o consumo de memória estritamente limitado ao número de dados armazenados $O(n)$, operando com tempo de busca em $O(\log n)$ (CORMEN et al., 2009).

c) Tabela Resumo das Complexidades de Tempo e Espaço

Estrutura	Operação	Tempo	Espaço
Heap de Fibonacci	Inserção, Mínimo, União	$\Theta(1)$ (amortizado/real)	$O(n)$
Heap de Fibonacci	Diminuição de Chave	$O(1)$ (amortizado)	$O(n)$
Heap de Fibonacci	Extração do Mínimo, Remoção	$O(\log n)$ (amortizado)	$O(n)$
Árvore vEB	Mínimo, Máximo	$\Theta(1)$ (pior caso)	$\Theta(U)$
Árvore vEB	Pertinência, Sucessor, Predecessor	$\Theta(\log \log U)$ (pior caso)	$\Theta(U)$
Árvore vEB	Inserção, Remoção	$\Theta(\log \log U)$ (pior caso)	$\Theta(U)$

Tabela 18 – Resumo comparativo de complexidade de tempo e espaço entre o Heap de Fibonacci e a Árvore vEB.

d) Esqueleto e Pseudocódigo Base das Técnicas

1. Heap de Fibonacci: Inserção Preguiçosa em $\Theta(1)$

```
PROCEDURE Inserir(H, chave, valor):
    aloque no x com chave, valor, grau = 0, marca = FALSE
    insira x na lista circular de raizes (a esquerda de H.min)
    IF H.min == NIL OU chave < H.min.chave THEN H.min = x
    H.n = H.n + 1
    RETURN x
```

Listing A.1 – Inserção preguiçosa na lista circular de raízes do Heap de Fibonacci.

2. Árvore de van Emde Boas: Roteamento Hierárquico no Sucessor

```
FUNCTION Sucessor(V, x):
    IF V.min == NIL THEN RETURN NIL
    IF x < V.min THEN RETURN V.min
    h = high(x); l = low(x)
    IF V.cluster[h].max != NIL AND l < V.cluster[h].max THEN
        RETURN index(h, Sucessor(V.cluster[h], l))
    END IF
    j = Sucessor(V.summary, h)
    IF j == NIL THEN RETURN NIL
    RETURN index(j, V.cluster[j].min)
```

Listing A.2 – Busca recursiva do sucessor na Árvore vEB com isolamento do mínimo.